

ASP .NET 6.0 MVC

COURS C#



- Introduction
- Anatomie de l'application
- Conventions en ASP .Net MVC
- Notion de route
- Les contrôleurs
- Les vues
- Les formulaires
- Html Helpers
- Tags Helpers
- Validation des formulaires
- Les variables de sessions
- Références

TABLE DES MATIÈRES

Asp .Net 6.0 MVC



INTRODUCTION

ASP .NET 6.0 MVC

- Historique
- Fonctionnement d'un site web
- Design Pattern MVC
- Rôle de chaque élément
- Pourquoi utiliser ASP .Net 6.0 MVC

INTRODUCTION

ASP .Net 5.0 MVC

HISTORIQUE

06/2016	• ASP .Net Core 1.0
11/2016	• ASP .Net Core 1.1
08/2017	• ASP .Net Core 2.0
05/2018	• ASP .Net Core 2.1 (LTS)
12/2018	• ASP .Net Core 2.2
09/2019	• ASP .Net Core 3.0
12/2019	• ASP .Net Core 3.1 (LTS)
11/2020	• ASP .Net 5.0
11/2021	• ASP .Net 6.0 (LTS)

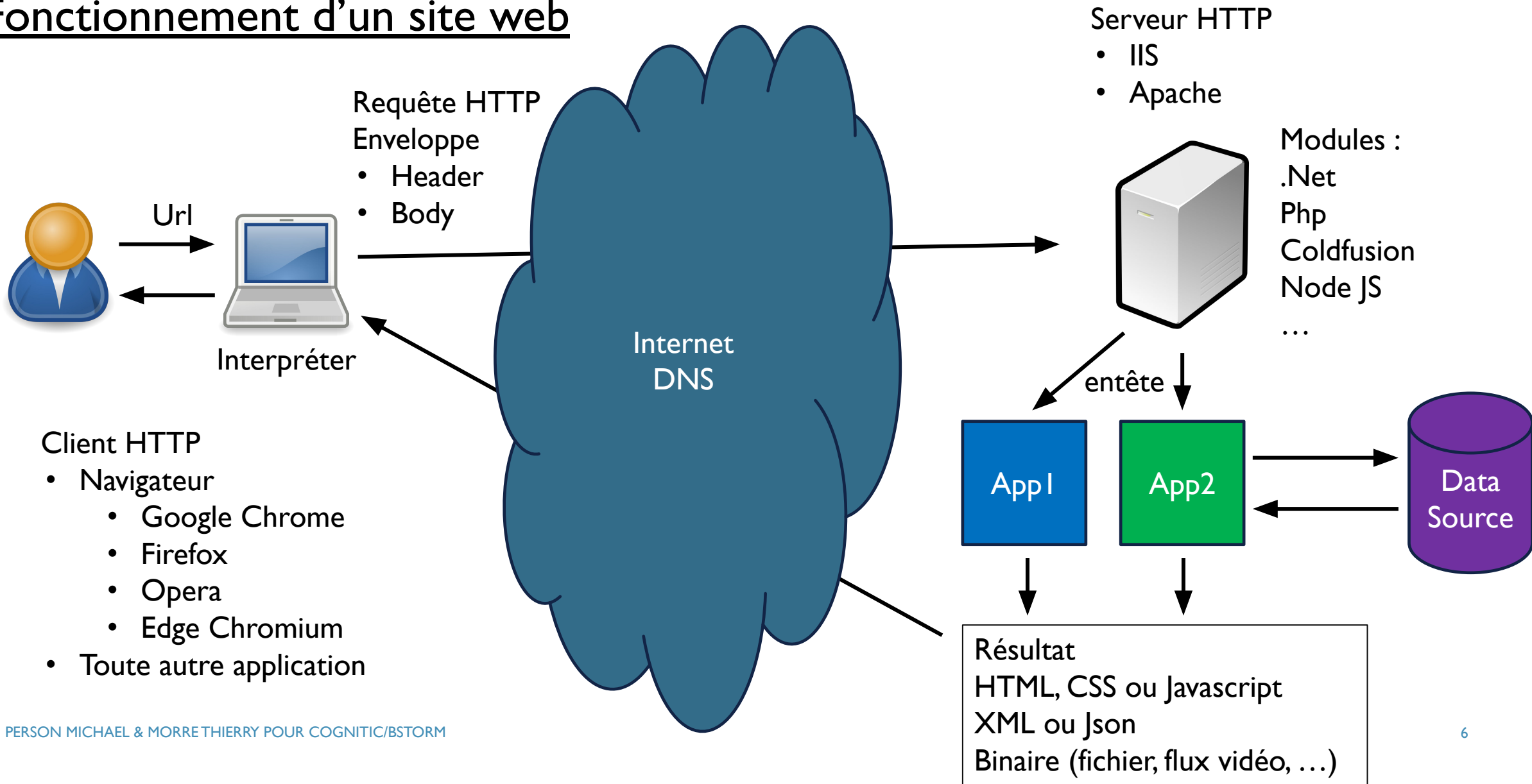
.NET Core est une plateforme de développement généraliste open source qui est tenue à jour par Microsoft et la communauté .NET sur GitHub.

Il est multiplateforme et prend en charge Windows, MacOS et Linux. Elle permet de générer des applications destinées à des appareils, au cloud et à l'IOT.

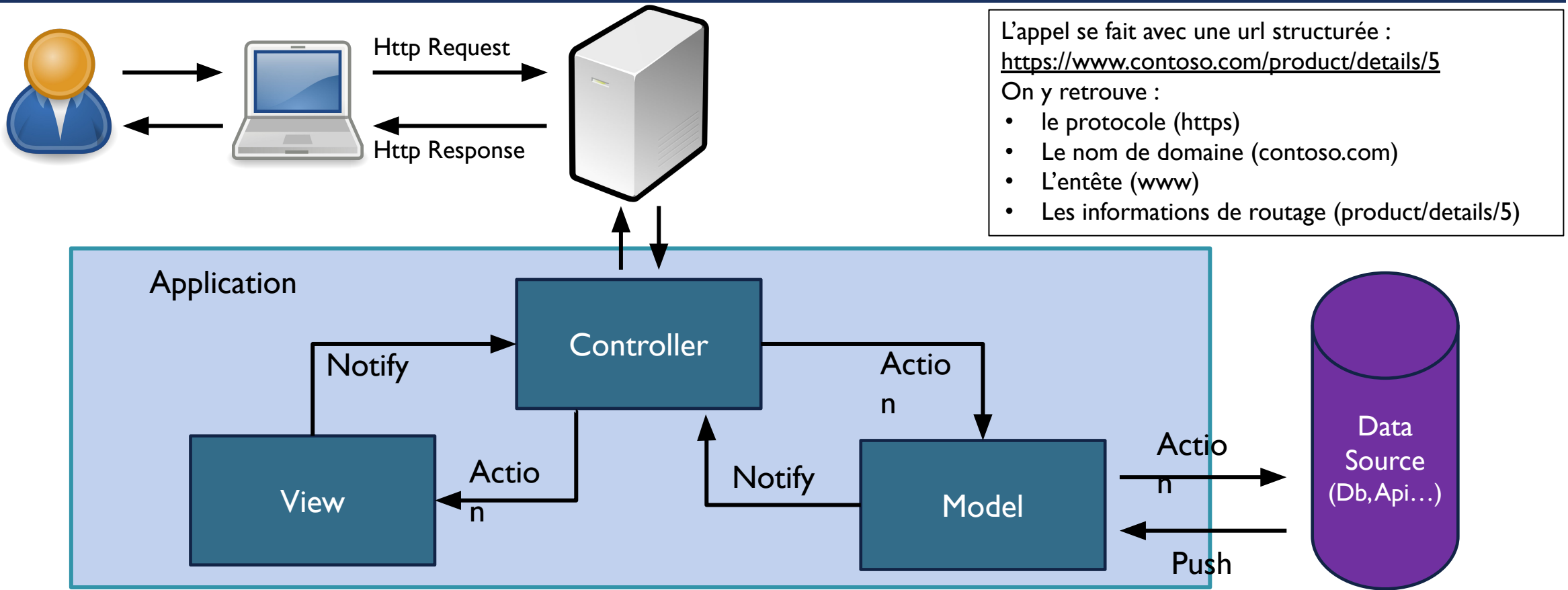
En 2020, l'appellation .Net Core a été remplacée par .Net, actuellement en version 6.0, dont le but est d'unifier l'univers .Net et rendant accessible pour Windows, Linux, MacOS, iOS, Android, tvOS, watchOS, WebAssembly et plus encore.

ASP .Net Core et maintenant ASP .Net 6.0 sont les successeurs d'ASP .Net, ils se veulent gratuit, open source et multiplateforme.

Fonctionnement d'un site web



DESIGN PATTERN MVC



DESIGN PATTERN MVC - RÔLES

Rôle du contrôleur

Dans le modèle MVC, le contrôleur est le point d'entrée initial de toutes requêtes faites par un client. Il est le garant de la logique métier de votre application et il est responsable de la sélection des types de modèle à utiliser et de la vue à afficher.

Ou en d'autres termes, ils gèrent l'interaction avec l'utilisateur, fonctionnent avec le modèle et, au final, si besoin sélectionnent une vue à afficher.

Rôle du modèle

Le modèle est la couche travaillant sur les données.

Ses rôles premiers sont de décrire les données et de fournir un accès à ces dernières (qu'il s'agisse d'une base de données, d'une api, ou d'un fichier plat).

Le cas échéant, le modèle définira les méthodes de conversions servant à transformer nos données avant l'envoi ou après réception afin que ces dernières appliquent les règles métier imposée par le corps business de l'applicatif.

Enfin, nous aurons parfois besoin de modèles de vue, ce type de modèle permettra l'affichage et/ou la gestion de formulaires (facilitant la validation de ces derniers).

DESIGN PATTERN MVC - RÔLES

Rôle de la vue

Les vues sont responsables de la présentation du contenu via l'interface utilisateur.

Elles utilisent le moteur d'affichage « Razor » pour incorporer le code .Net dans le balisage HTML.

Cependant, contrairement à d'autres Framework basés sur les fichiers, les vues ne sont pas directement accessibles mais fournissent un résultat à une demande spécifique du contrôleur.

POURQUOI UTILISER ASP .NET 6.0 MVC

- Prise en charge de plusieurs plates-formes :
Windows, Mac ou encore Linux
- Rapide:
Il ne dépend plus de *System.Web.dll* pour la communication Client-Server. Cela réduit les délais des requêtes et améliore les performances
- IOC Container :
Il intègre un IOC Container et l'injection de dépendances ce qui facilite la maintenance et les tests
- Intégration avec les Framework UI :
Il permet de d'intégrer et de gérer plus facilement les Frameworks tels Angular, React, Umber, Bootstrap,...
à l'aide de Bower (Gestionnaire de paquets web)
- Hébergement :
Il ne dépend plus d'IIS comme ses prédécesseurs et peut être hébergé sur n'importe quel serveur web tel Apache, ...



ANATOMIE DE L'APPLICATION

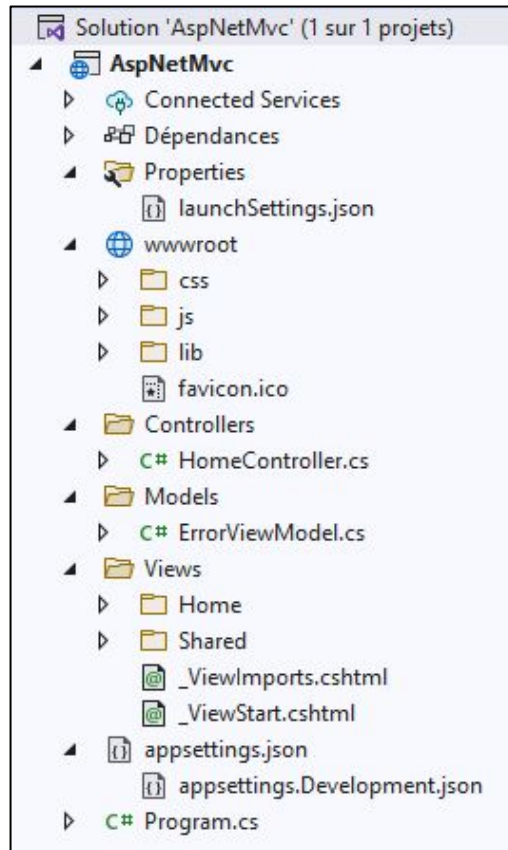
ASP .NET 6.0 MVC

- Structure de base
- Références vs Dépendances
- Le fichier « launchsettings.json »
- Le fichier « appsettings.json »
- Le fichier « program.cs »
- Notion de « Middleware »
- Le dossier « wwwroot »
- Le dossier « Controllers »
- Le dossier « Models »
- Le dossier « Views »

ANATOMIE DE L'APPLICATION

Asp .Net 6.0 MVC

STRUCTURE DE BASE



L'architecture MVC sur laquelle reposent les projets ASP .NET MVC offre une disposition par défaut aux fichiers du projet.

Ainsi, tous les contrôleurs de l'application sont créés par défaut dans un dossier « Controllers », les vues dans un dossier « Views ».

Concernant le modèle, bien qu'il soit possible de le créer dans le dossier « Models », en règle générale ce dossier contiendra que les modèles utilisables uniquement que par l'application tel que « les modèle de vues » liés aux formulaires ou à l'affichage.

Préférant ainsi la création de librairies de classes pour le modèle d'accès aux données.

RÉFÉRENCES VS DÉPENDANCES

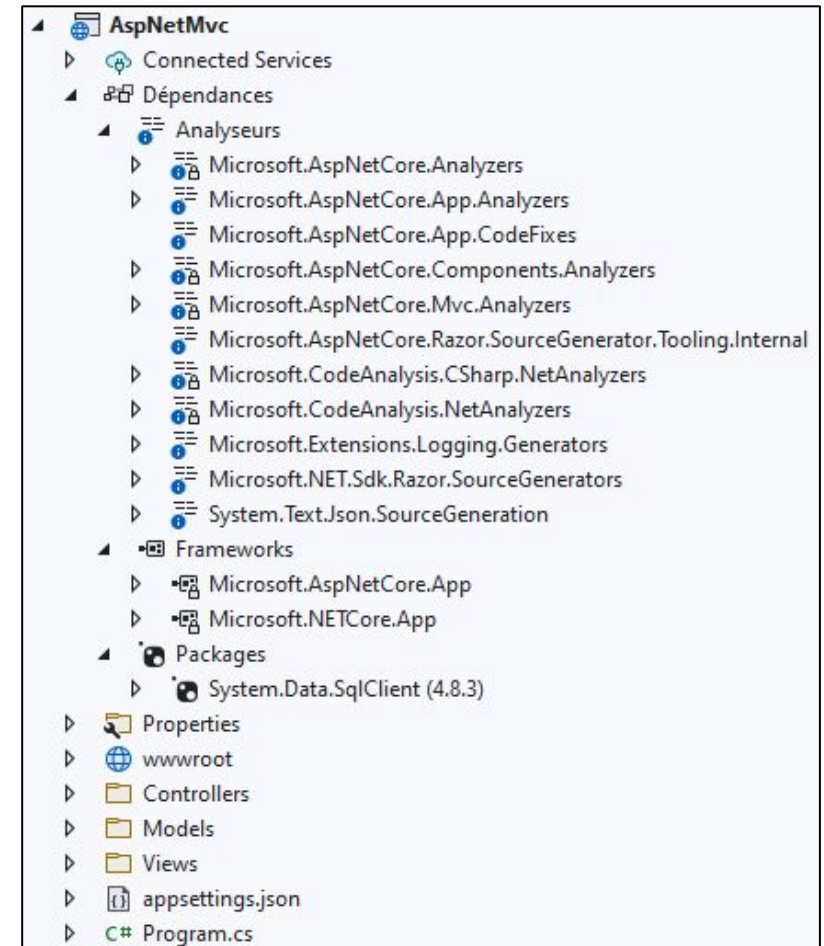
Depuis la version .Net Core, le terme « Références » a été remplacé par « Dépendances ».

En effet, par le passé, le Framework .Net étant installé intégralement sur la machine, nous ajoutions des références sur ces éléments.

Le Framework .Net Core et maintenant .Net 6.0, eux utilisent un Framework minimal sur lequel nous venons ajouter des dépendances en fonction du type de projet.

Les dépendances installées sont définies par le Sdk dans le fichier « csproj » et les packages nécessaires y seront ajoutés également.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net6.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="System.Data.SqlClient" Version="4.8.3" />
  </ItemGroup>
</Project>
```



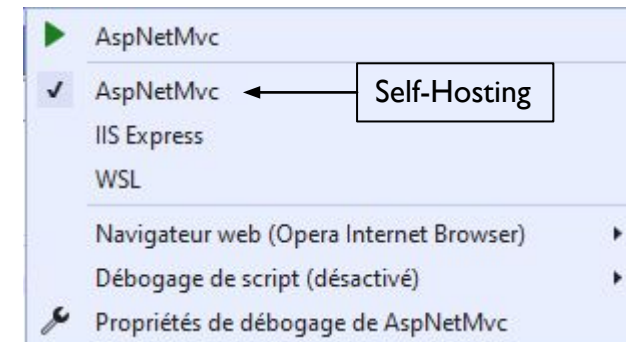
LE FICHIER « LAUNCHSETTINGS.JSON »

```
{
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:18852",
      "sslPort": 44327
    }
  },
  "profiles": {
    "AspNetMvc": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7082;http://localhost:5082",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Ce fichier contient les informations de profile et de debugages.

Par défaut il contient une configuration pour « IIS Express » et une pour son auto-hébergement (« self-hosting »).

En effet, il est possible d'utiliser « IIS Express » pour déboguer notre application mais aussi de lancer notre projet depuis une invite de commandes (sous linux par exemple).



LE FICHIER « APPSETTINGS.JSON »

Ce fichier va être utilisé pour la configuration de notre application.

Par défaut, il contient la configuration du service de logs, mais nous pourrons y ajouter nos chaînes de connexion à nos bases de données comme des variables d'environnement.

Ces informations pourront être récupérée lors de la configuration (par défaut dans le fichier « Program.cs ») comme ci-dessous par exemple.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DbConnection": "Data Source={0};Initial Catalog={1};User Id={2};Password={3}"
  },
  "AllowedHosts": "*",
  "SmtpServer" : "smtp.myserver.be"
}
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
string smtpServer = builder.Configuration["SmtpServer"];
```

```
string connectionString = builder.Configuration.GetConnectionString("DbConnection");
```

▣ Masqué pour des raisons de mise en page

LE FICHIER « PROGRAM.CS »

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Au démarrage, une application ASP .NET Core crée un hôte. L'hôte encapsule toutes les ressources de l'application, telles que :

- Une implémentation serveur HTTP
- Configuration du middleware
- Système de logs
- Services d'injection de dépendance (DI)
- Configuration (appsettings.json, Azure, ..)

Il existe deux hôtes différents :

- Hôte générique .NET
- Hôte web ASP .NET Core

L'hôte Web ASP .NET Core est déprécié et n'est disponible qu'à des fins de compatibilité descendante.



Notons que le fichier « Program.cs » applique le nouveau template du .Net 6.0
<https://aka.ms/new-console-template>

LE FICHIER « PROGRAM.CS »

Ce fichier contient également tout ce qui touche à la configuration de notre application.

Dans le cadre d'une application « MVC » on y retrouve, par défaut, l'ajout des contrôleurs avec vue sous forme de services.

En effet, ceux-ci sont nativement intégrés dans le processus d'injection de dépendances que le moteur MVC utilise au moment d'instancier un contrôleur.

Ensuite, nous y retrouvons la configuration du middleware, c'est-à-dire les fonctionnalités que notre application activera tel que :

- La redirection automatique vers le protocole https
- L'utilisation du système de routage et sa configuration
- La page d'erreur à afficher en cas d'exception
- ...

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

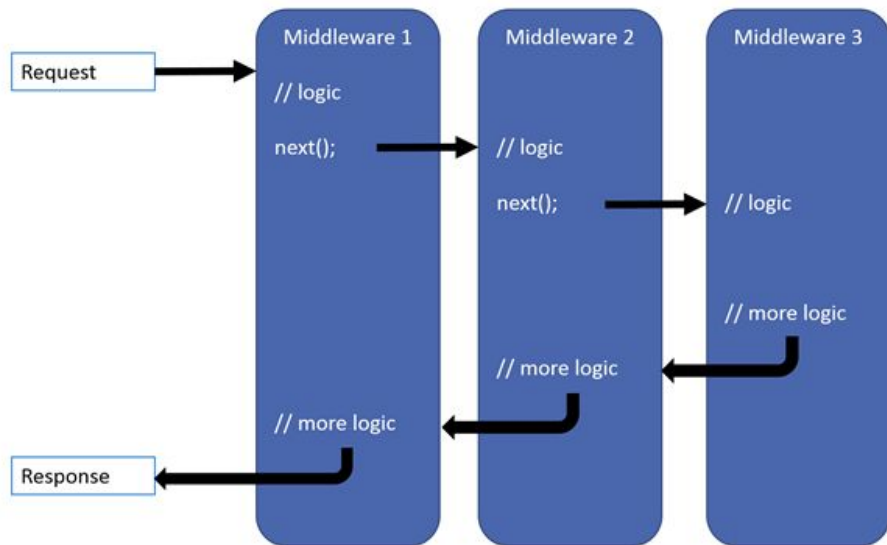
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

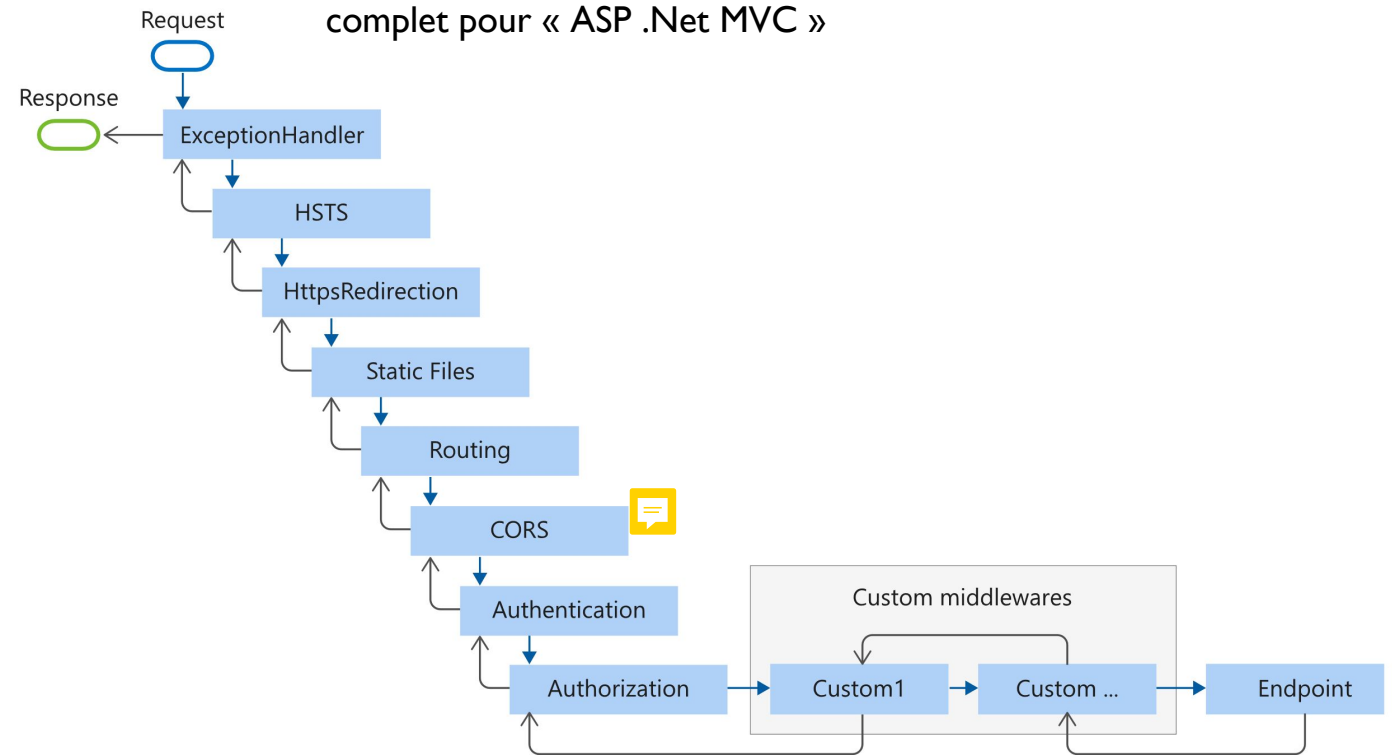
app.Run();
```

NOTION DE « MIDDLEWARE »

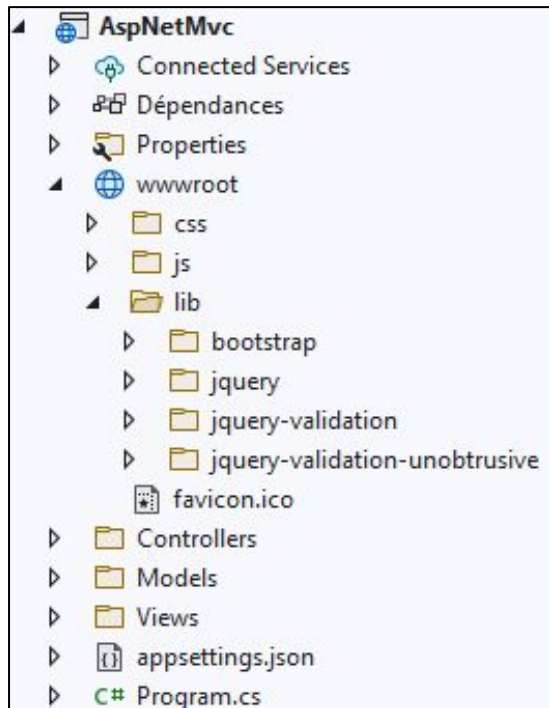
Le pipeline de requêtes « ASP .Net MVC » est composé d'une séquence de traitements, appelés l'un après l'autre. Chaque élément, appelé middleware, peut effectuer des opérations avant et après l'élément suivant éventuel.



Ce schéma illustre le pipeline de traitement des demandes complet pour « ASP .Net MVC »



LE DOSSIER « WWWROOT »



Le dossier « wwwroot » du projet ASP .Net MVC est traité comme le dossier racine de notre application.

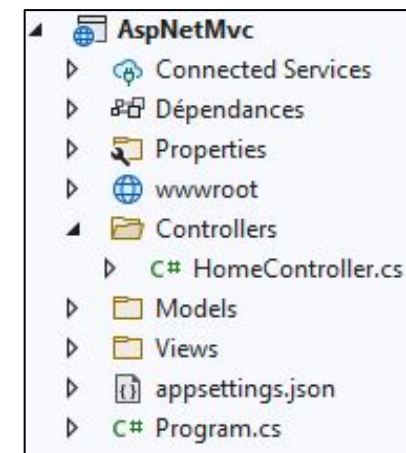
Les fichiers statiques tels que les bannières, les icônes, le styles css, les scripts javascript peuvent y être stockés dans n'importe quel sous-dossier et seront accessibles via un chemin relatif.

On y retrouve, par défaut, la présence de « Bootstrap » et de « JQuery ».

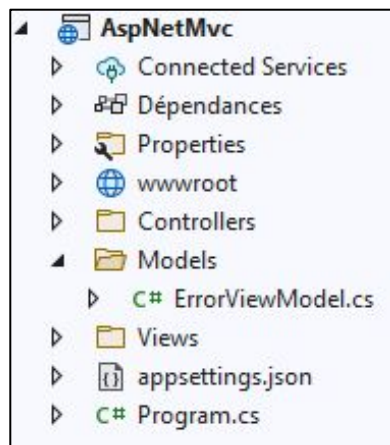
LE DOSSIER « CONTROLLERS »

Par définitions, le répertoire « Controllers » contiendra nos contrôleurs.

Ceux-ci contiendront la logique métier de notre application.



LE DOSSIER « MODELS »



Le répertoire « Models » contiendra notre modèle.

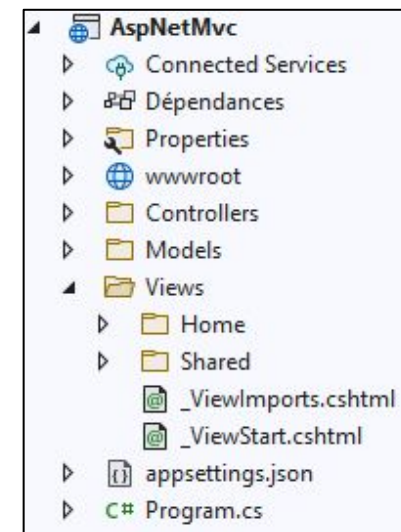
Cependant, dans la bonne pratique et afin d'améliorer la maintenance de notre application, nous n'y trouverons que les éléments qui ne peuvent être utilisés que par l'application tel que les modèles de vue pour les formulaires.

La couche d'accès aux données étant reprise dans des « dll » séparées.

LE DOSSIER « VIEWS »

Le répertoire « Views » contiendra tout ce qui a trait à l’affichage

- Les vues
- Les vues partielles
- Les composants de vue
- Les pages de disposition « Layout »





CONVENTION EN ASP .NET MVC

ASP .NET 6.0 MVC

CONVENTIONS EN ASP .NET MVC

MVC est conçu autour de valeurs par défaut sur la base de conventions qui peuvent être redéfinies au besoin.

Ce concept est communément appelé « convention over configuration »

*« We know, by now, how to build a web application.
Let's roll that experience into the framework so we don't have to
configure absolutely everything again. »*

Ruby On Rails

*« Nous savons , désormais, comment construire une application web.
Partageons cette expérience de sorte que nous ne devons pas
configurer à nouveau absolument tout »*

Le concept de « Conventions over Configuration » est illustré par la présence des trois dossiers qui font que l'application peut fonctionner :

- Controllers
- Models
- Views

Ces dossiers sont juste là par convention afin que le moteur « MVC » puisse retrouver ces éléments.

En d'autre termes, nous ne devons pas spécifier au moteur « MVC » que nos vues se trouvent dans le dossier « Views » puisque par convention, il le sait déjà.

CONVENTIONS EN ASP .NET MVC

Les conventions ASP.NET MVC sont plutôt simples :

- Chaque nom de contrôleur, se termine par « **Controller** » (HomeController, AccountController, ProductController, etc.)
- Il n'a qu'un dossier « **Views** » pour toutes les vues de l'application
- Les vues utilisées par chaque contrôleur doivent se trouver dans un sous-dossier de « **Views** » qui portera le nom du Contrôleur sans le suffixe. (ProductController aura ses vues dans /Views/Product)
- Les interfaces utilisateurs réutilisables (partial view, layout, component, etc.) se retrouvent dans le dossier « Views/Shared » et sont préfixées d'un « _ » (« _Layout.cshtml »).



NOTION DE ROUTE

ASP .NET 6.0 MVC

- Avant de commencer
- Introduction
- Route conventionnelle par défaut
- Ajouter ses propres routes
- Noms de routage réservés

INTRODUCTION

Asp .Net 5.0 MVC

AVANT DE COMMENCER

```
public static IActionResult GetInfo(this ControllerContext controllerContext)
{
    ControllerActionDescriptor actionDescriptor = controllerContext.ActionDescriptor;
    HttpRequest httpRequest = controllerContext.HttpContext.Request;
    IList<ParameterDescriptor> parameters = controllerContext.ActionDescriptor.Parameters;
    var info = new {
        HttpVerb = httpRequest.Method,
        Protocol = httpRequest.Protocol,
        Scheme = httpRequest.Scheme,
        HostUrls = new {
            Value = httpRequest.Host.Value,
            Host = httpRequest.Host.Host,
            Port = httpRequest.Host.Port
        },
        Controller = actionDescriptor.ControllerName,
        Action = actionDescriptor.ActionName,
        Parameters = parameters.Select(p => new {
            p.Name,
            ParameterType = p.ParameterType.Name,
            Value = parameters[parameters.IndexOf(parameters.Single(p => p.Name == p.Name))].ToString()
        }).ToArray(),
        QueryStringParameters = !httpRequest.QueryString.HasValue ?
            string.Empty : httpRequest.QueryString.Value
    };
    return new JsonResult(info, new JsonSerializerOptions() { WriteIndented = true });
}
```

PERSON MICHAEL & MORRE THIERRY POUR COGNITIC/BSTORM

Durant la présentation des routes et des contrôleurs, je ferai appel à une méthode qui s'appelle « GetInfo() ».

Cette méthode n'existe pas nativement au sein du moteur « MVC ».

Il s'agit en réalité d'une méthode d'extension créée pour illustrer comment le moteur interprète une requête et, cette dernière, renvoi un contenu « Json » avec quelques données nécessaires à la compréhension.

INTRODUCTION

Toute requête envoyée à une application utilisant le moteur MVC doit être structurée, cette structure est placée derrière l'url et définit la route que l'on veut emprunter.

Les contrôleurs de ASP .NET Core utilisent le middleware de routage pour faire correspondre les URL aux actions.

Une action est toute méthode publique présente dans un contrôleur sauf si cette dernière possède l'attribut « NonAction ».

Il existe deux modèles de routage :

- Le routage conventionnel (Application Web MVC)
- Le routage par attribut (Api)

L'appel se fait avec une url structurée :

<https://www.contoso.com/product/details/5>

On y retrouve :

- le protocole (https)
- Le nom de domaine (contoso.com)
- L'entête (www)
- Les informations de routage (product/details/5)

ROUTE CONVENTIONNELLE PAR DÉFAUT

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Lorsque nous créons notre projet MVC, une route conventionnelle par défaut est définie dans le fichier « Program.cs », celle-ci définit la structure classique d'URL attendue.

Dans le cas présent, celle-ci demande, après l'URL de base, de fournir dans les informations de routage le nom du contrôleur suivie du nom de l'action suivie son tour par un identifiant.

De plus, elle fournit les valeurs par défaut si ces informations sont manquantes :

- « Home » pour le contrôleur
- « Index » pour l'action
- « id » optionnel

ROUTE CONVENTIONNELLE PAR DÉFAUT

Url	Informations
https://www.contonso.com	Contrôleur : home Action : index
https://www.contonso.com/product	Contrôleur : product Action : index
https://www.contonso.com/product/details/5	Contrôleur : product Action : details Id : 5
https://www.contonso.com/admin/index	Contrôleur : admin Action : index
https://www.contonso.com/index/admin	Contrôleur : index Action : admin

Url	Informations
https://www.contonso.com/admin/index/soccer	Contrôleur : admin Action : index Id : soccer (string)
https://www.contonso.com/?test=5	Contrôleur : home Action : index
https://www.contonso.com/admin/17	Erreur
https://www.contonso.com/5	Erreur

AJOUTER SES PROPRES ROUTES

Bien entendu, nous pouvons modifier/supprimer la route par défaut et spécifier nos propres routes.

Cependant le nom de la route doit être unique et pattern doit être dissociable des autres.

En cas d'ambiguïté, le système se verra contraint de :

- Choisir le meilleur candidat (voir gestion de formulaires).
- Lever d'une exception.

Si les mots « controller » et « action » ne sont pas repris par le pattern, des valeurs par défauts doivent être spécifiées pour ces derniers.

L'astérisque « * » ou le double « ** » signifie « catch-all ». C'est-à-dire que nous prenons le reste de l'url et l'affectons à l'information qui suit.

(ex : « article » dans la route « Blog »)

```
app.MapControllerRoute(  
    name: "blog",  
    pattern: "blog/{*article}",  
    defaults: new { Controller = "Blog", Action = "Index" });
```

```
app.MapControllerRoute(  
    name: "admin",  
    pattern: "Admin/{action=Index}/{id?}",  
    defaults: new { Controller = "Admin" });
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```



L'ordre d'ajout des routes est important!
De la plus spécifique à la plus générale.

NOMS DE ROUTAGE RÉSERVÉS

Les noms suivants sont réservés par le middleware de routage.

Une mauvaise utilisation de ces derniers, pourrait engendrer des problèmes de générations de routes et conduire à des erreurs.

- action
- area
- controller
- handler
- page

Le middleware les utilise pour déterminer si nous faisons référence à une page statique ou à un contrôleur.



LES CONTRÔLEURS

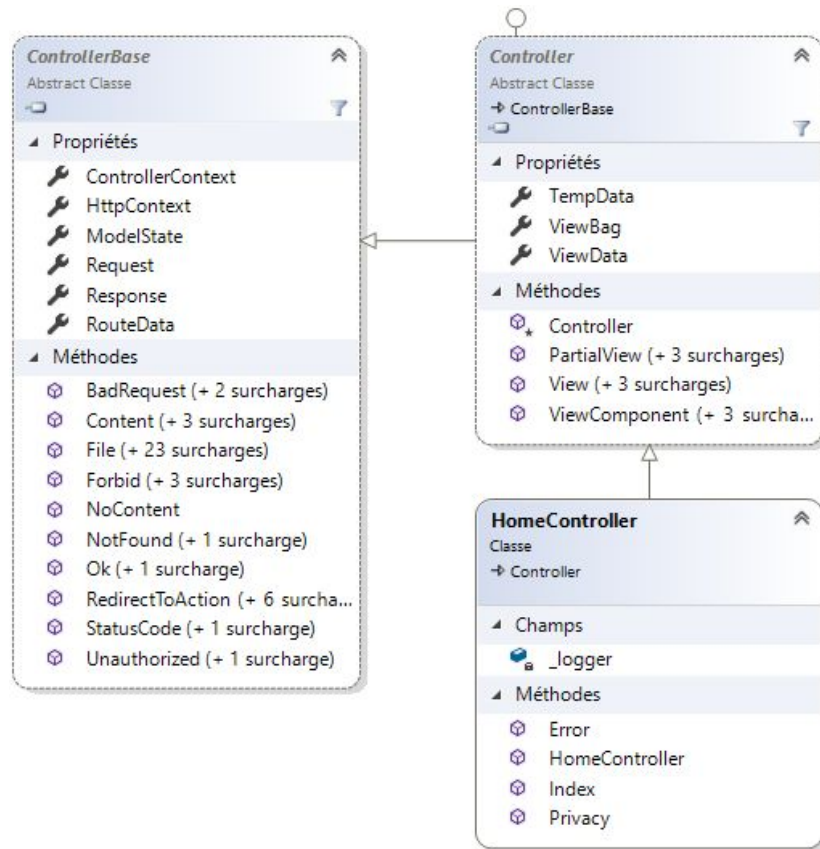
ASP .NET 6.0 MVC

- Avant de commencer
- À savoir sur les contrôleurs
- « HomeController » par défaut
- La classe « Controller »
- Créer ses contrôleurs
- L'interface « IActionResult »
- Les types courants implémentant « IActionResult »
- Spécifier une route particulière dans le contrôleur

LES CONTRÔLEURS

Asp .Net 6.0 MVC

À SAVOIR SUR LES CONTRÔLEURS



En ASP .Net 6.0 MVC, qu'il s'agisse d'une application de type Web ou Api, les contrôleurs héritent tous d'au moins la classe « ControllerBase ».

Il n'y a que dans le cadre d'une application « Web » que nous hériterons de la classe « Controller ».

En effet, celle-ci ajoute des propriétés dédiées au passage de données entre les contrôleurs et les vues et des méthodes spécifiques à la génération de ces dernières.



Il existe bien plus de propriétés et de méthodes dans les classes « ControllerBase » et « Controller »

LA CLASSE « CONTROLLERBASE »

Propriétés	
ControllerContext	Donne accès au contexte associé à la demande actuelle d'un contrôleur.
HttpContext	Donne accès aux informations spécifiques de la requête HTTP courante
ModelState	Représente l'état d'une tentative de liaison de valeurs entre une requête HTTP et une action
Request	Donne accès aux informations de la requête
Response	Donne accès à l'objet qui contient la réponse
RouteData	Contient les informations de routage

Méthodes	
BadRequest	Retourne un code d'erreur 400
Content	Retourne un code 200 avec les données
File	Retourne un code 200 et le contenu d'un fichier
Forbidden	Retourne un code d'erreur 403
NoContent	Retourne un code 204
NotFound	Retourne un code erreur 404
Ok	Retourne un code 200
RedirectToAction	Permet de rediriger vers une autre action
StatusCode	Permet de retourner n'importe quel code
Unauthorized	Retourne un code erreur 401



LA CLASSE « CONTROLLER »

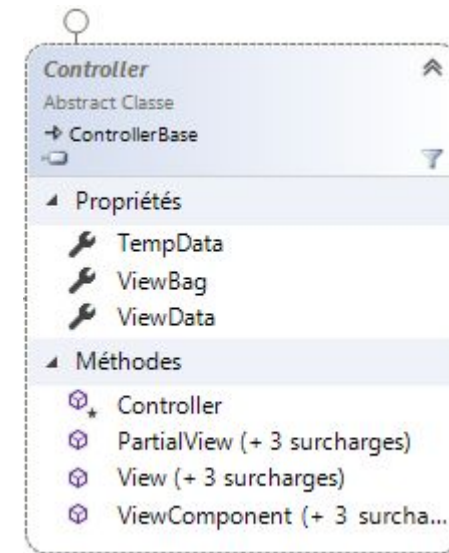
La classe « Controller », quant à elle, est héritée par les contrôleurs des applications web.

Les méthodes, « PartialView », « View » et « ViewComponent » ont pour but de générer un affichage basé sur les fichiers de vue que nous définirons plus tard.

Les propriétés « TempData », « ViewBag » et « ViewData » sont là pour passer des données primitives aux vues.



Nous les verrons plus en détail
un peu plus loin dans la matière



« HOMECONTROLLER » PAR DÉFAUT

```
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }

    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
    0 references
    public IActionResult Error()
    {
        return View(new ErrorViewModel {
            RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }
}
PERSON MICHAEL & MORRE THIERRY POUR COGNITIC/BSTORM
```

La classe « HomeController » propose trois actions prédéfinies :

- Index : Retournant la page d'accueil de notre site
- Privacy : Retournant la page pour détailler la politique de confidentialité de notre site
- Error : Retournant la vue dédiée aux erreurs en cas d'exception non gérée comme spécifiée dans la configuration

```
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

Dans le fichier « program.cs »



Pour la page « Error », le mode « Development » doit être désactivé dans le fichier « launchSettings.json »

« HOMECONTROLLER » PAR DÉFAUT

Il est à noter cependant que, contrairement à sa version « Asp .Net Mvc Classic » utilisant le Framework .Net 4.8, l'instanciation est gérée par le moteur d'injection de dépendances.

Il nous est donc possible d'injecter des ressources en paramètre du constructeur de notre contrôleur.



Nous verrons l'injection de dépendances un peu plus loin dans la matière

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 références
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 références
    public IActionResult Index()...
    0 références
    public IActionResult Privacy()...
    Caché pour des raisons de mise en page
    0 références
    public IActionResult Error()...
}
```

CRÉER SES CONTRÔLEURS

```
using Microsoft.AspNetCore.Mvc;

namespace AspNetMvc.Controllers
{
    0 références
    public class AdminController : Controller
    {
        0 références
        public IActionResult Index()
        {
            return ControllerContext.GetInfo();
        }
    }
}
```

Pour créer ses contrôleurs, rien de plus simple!

Soit manuellement, en créant une classe et en lui faisant hériter de la classe « Controller ».

Soit en utilisant un modèle de Visual Studio.

Il existe 3 modèles :

- Contrôleur vide
- Contrôleur avec des actions de lecture / écriture
- Contrôleur avec vues, utilisant « Entity Framework »



N'oublions pas les conventions!
Le nom d'un contrôleur se termine par
« Controller »

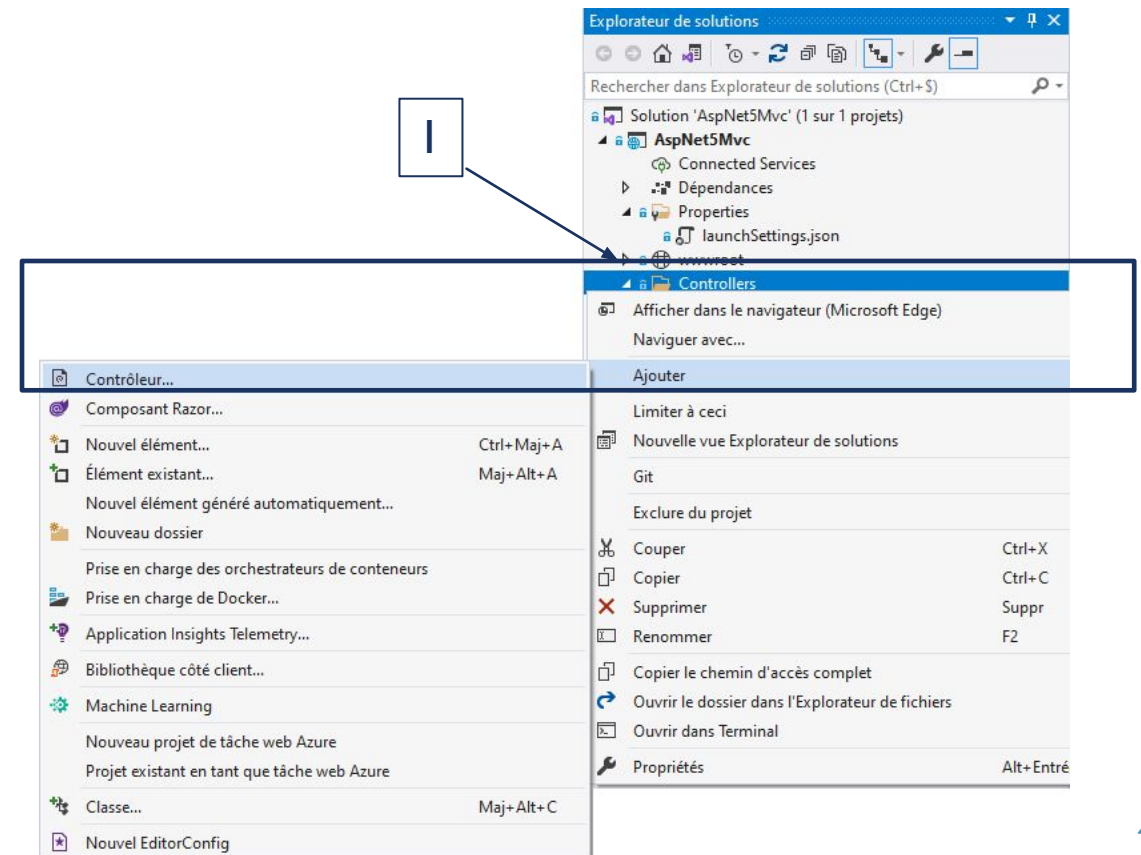
CRÉER SES CONTRÔLEURS

Pour utiliser un modèle de « Controller », nous devons :

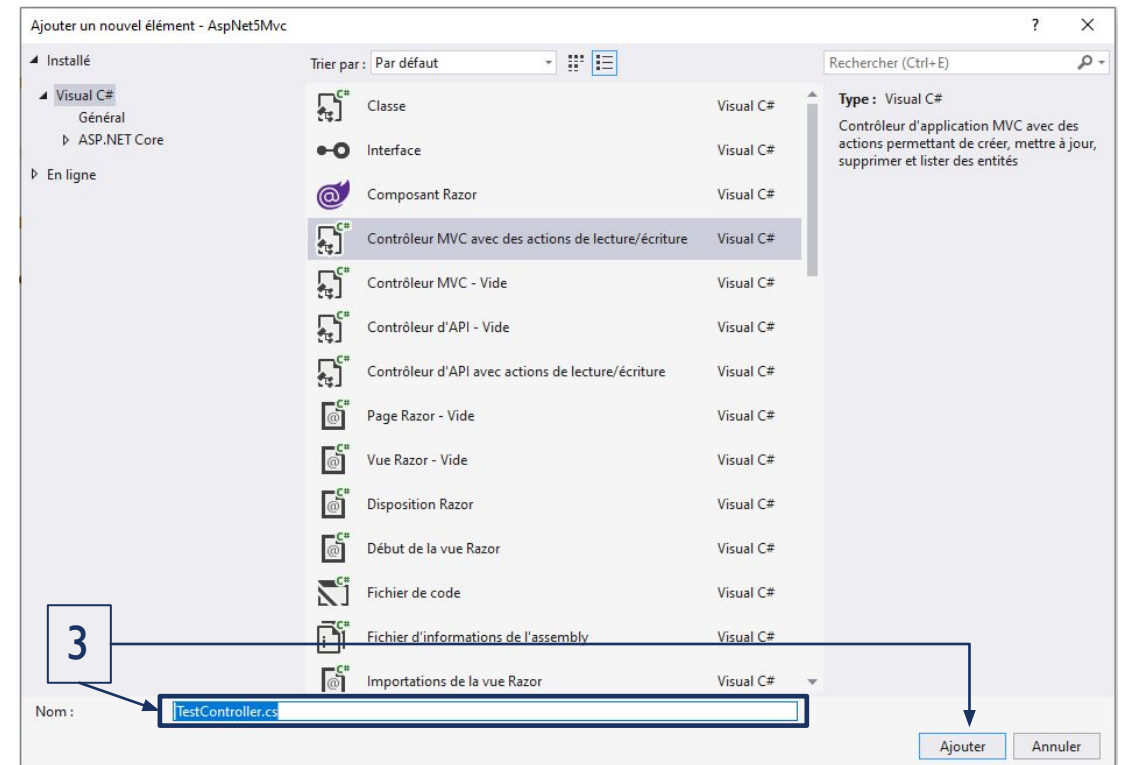
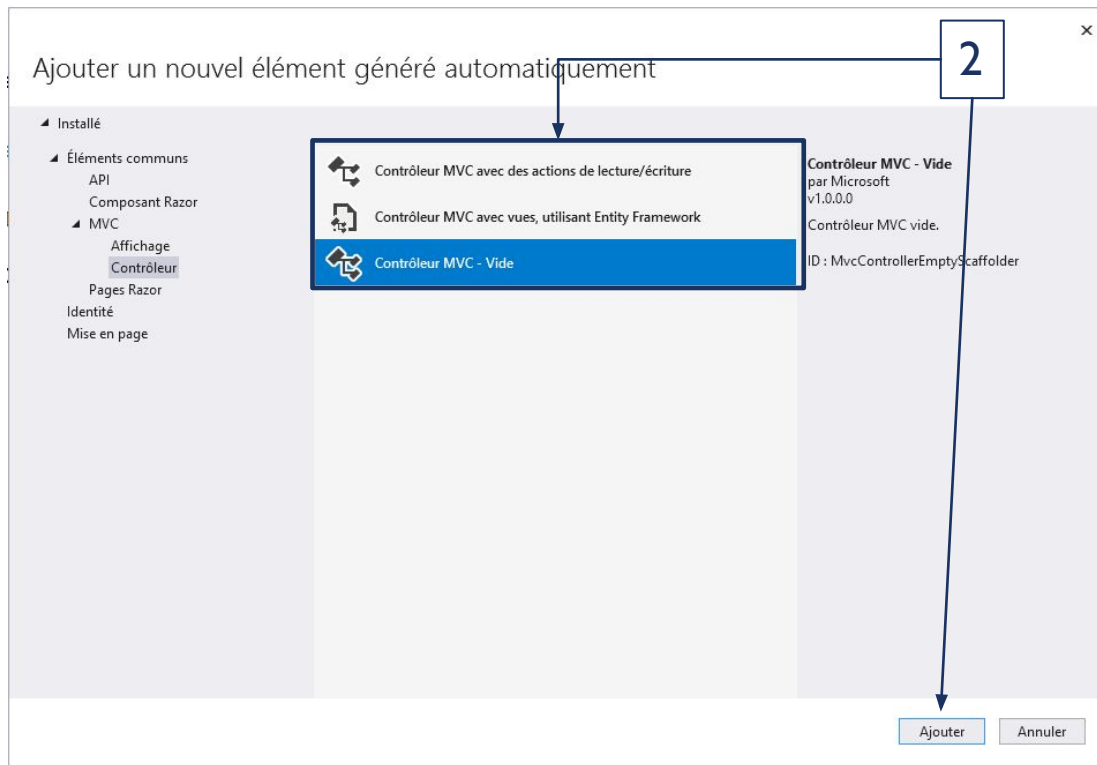
1. Faire un clic droit sur le répertoire « Controllers »
 - Sélectionner « Ajouter »
 - Cliquer sur « Contrôleur »
2. Choisir le modèle et cliquer sur « Ajouter »
3. Lui donner un nom et cliquer sur « Ajouter »



N'oublions pas les conventions!



CRÉER SES CONTRÔLEURS



L'INTERFACE « IACTIONRESULT »

```
using System.Threading.Tasks;

namespace Microsoft.AspNetCore.Mvc
{
    ...public interface IActionResult
    {
        ...Task ExecuteResultAsync(ActionContext context);
    }
}
```

Cette interface est implémentée par toutes les classes permettant de générer un résultat par et pour le moteur « MVC ».

La méthode « ExecuteResultAsync » est appelée par le moteur « Mvc » afin de fournir le résultat et retourner ce dernier.

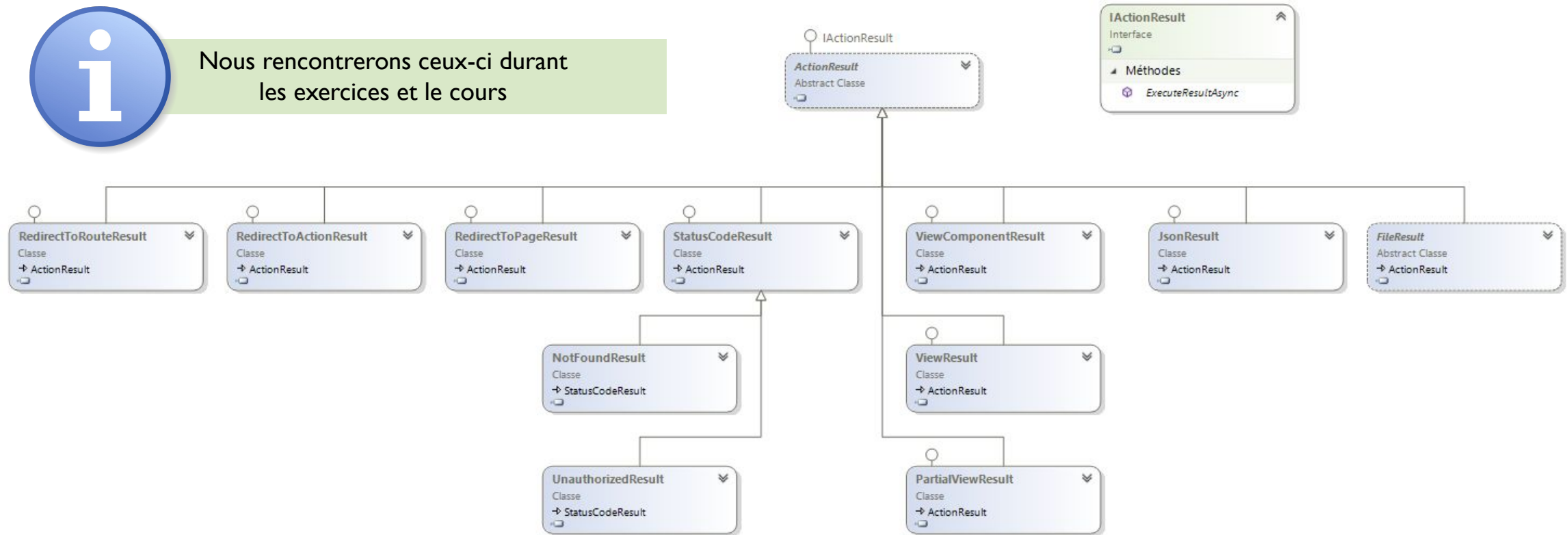
Les classes « Controller » et « ControllerBase » fournissent des méthodes instanciant ces classes pour nous.

Exemples	Type de résultat instancié
return View();	ViewResult
return PartialView(...);	PartialViewResult
return Json(...)	JsonResult
return RedirectToAction(...)	RedirectToActionResult
return File(...)	FileStreamResult
...	

LES TYPES COURANTS IMPLÉMENTANT « IACTIONRESULT »



Nous rencontrerons ceux-ci durant les exercices et le cours



SPÉCIFIER UNE ROUTE PARTICULIÈRE DANS LE CONTRÔLEUR

Bien que dans la bonne pratique, et comme vu précédemment, il soit vivement conseillé de mettre la déclaration des routes dans le fichier « Program.cs », il est possible de faire une déclaration spécifique dans le contrôleur directement.



PERSON MICHAEL & MORRE THIERRY POUR COGNITIC/BSTORM

```
public class BlogController : Controller
{
    0 references
    public IActionResult Index(string article)
    {
        return ControllerContext.GetInfo();
    }

    [Route("Blog/Test/{id}")]
    0 references
    public IActionResult Exemple(int id)
    {
        return ControllerContext.GetInfo();
    }
}
```



Nous pouvons spécifier plusieurs routes sur la même action de manière à lui fournir plusieurs « endpoint »



LES VUES

ASP .NET 6.0 MVC

- Les pages « razor »
- Le répertoire « Shared »
- Ordre de priorité
- La syntaxe « razor »
- Passer des données du contrôleur à la vue

LES PAGES « RAZOR »

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;
    public HomeController(ILogger<HomeController> logger) ...
    public IActionResult Index()
    {
        return View();
    }
    public IActionResult Privacy() ...
    public IActionResult Error() ...
}
```

Masqué pour des raisons de mise en page



Les pages « Razor », communément appelée « View », sont placées dans le répertoire « Views » du projet.

Avant toute chose, nous devons spécifier au moteur « MVC » que notre action retourne une vue. Pour cela, nous spécifierons au niveau de l'action « return View(); »

Ensuite, par défaut, une vue doit se retrouver dans un répertoire portant le nom du contrôleur sans son suffixe et elle doit elle-même porter le nom de l'action à laquelle elle est associée et elle a pour extension « .cshtml ».

Par exemple, la vue liée à l'action « Details » du « ProductController » devrait se trouver dans le répertoire « Views\Product\ » et s'appeler « Details.cshtml ».



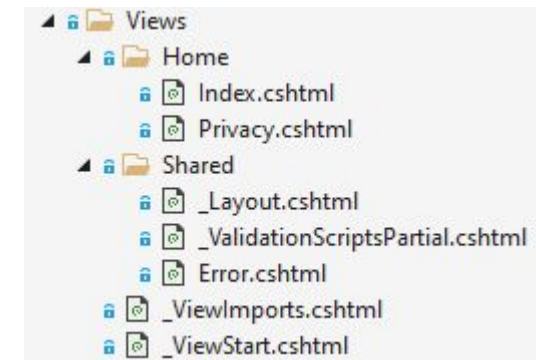
Dans le cadre des composants,
l'extension est « .razor »

LE RÉPERTOIRE « SHARED »

Comme son nom l'indique, il s'agit d'un répertoire partagé.

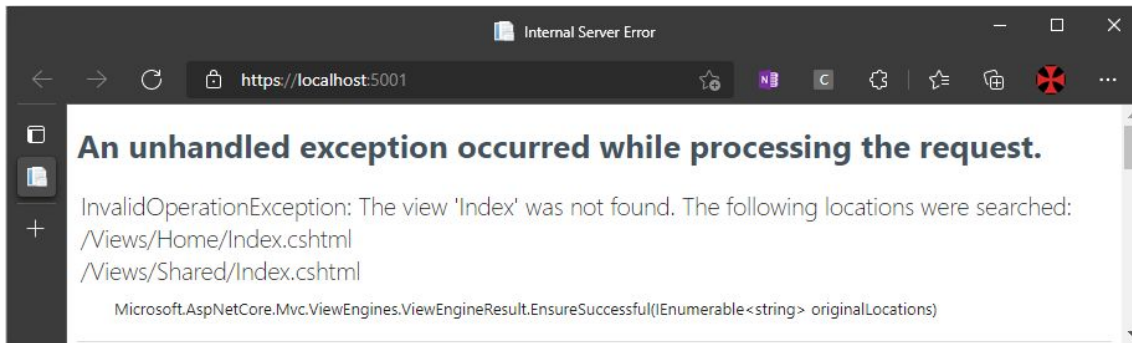
Il contiendra des vues réutilisables par plusieurs contrôleurs, des vues dîtes « partielles », les « composants » et nos « pages de disposition ».

Par défaut nous y retrouvons la page « _Layout.cshtml » qui sera la structure de page par défaut de notre site, une vue partielle « _ValidationScriptsPartial.cshtml » ainsi que la page « Error.cshtml ».



Les « Layouts », les « Components » et les « Partial Views » seront vues un peu plus loin

ORDRE DE PRIORITÉ



Le moteur « MVC » respectera un ordre de priorité pour aller rechercher les vues dans les répertoires.

1. Le répertoire associé contrôleur
2. Le répertoire « Shared »

Si aucun de ces répertoires ne contient la vue, ce message apparaîtra.



Il existe plusieurs surcharges de la méthode « View » dont une qui prend en argument le nom de la vue à retourner

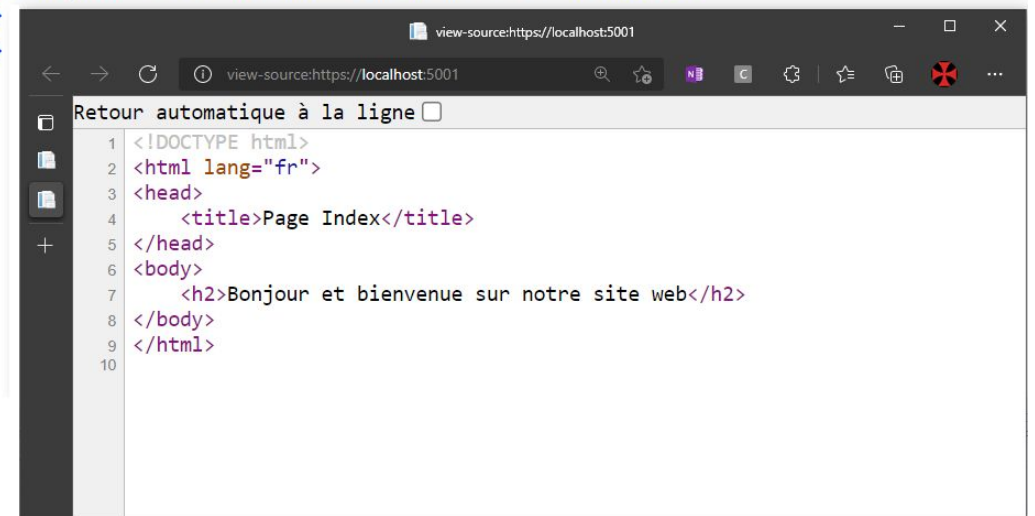
LA SYNTAXE « RAZOR »

La syntaxe « Razor » comprend le balisage « Razor », « C# » et « HTML ».

Le langage par défaut de « Razor » est « HTML ». Le rendu « HTML » à partir du balisage « Razor » n'est pas différent du rendu « HTML » à partir d'un fichier « HTML ».

Lorsque nous définissons du « HTML » dans nos pages « Razor », celui-ci est retourné inchangé.

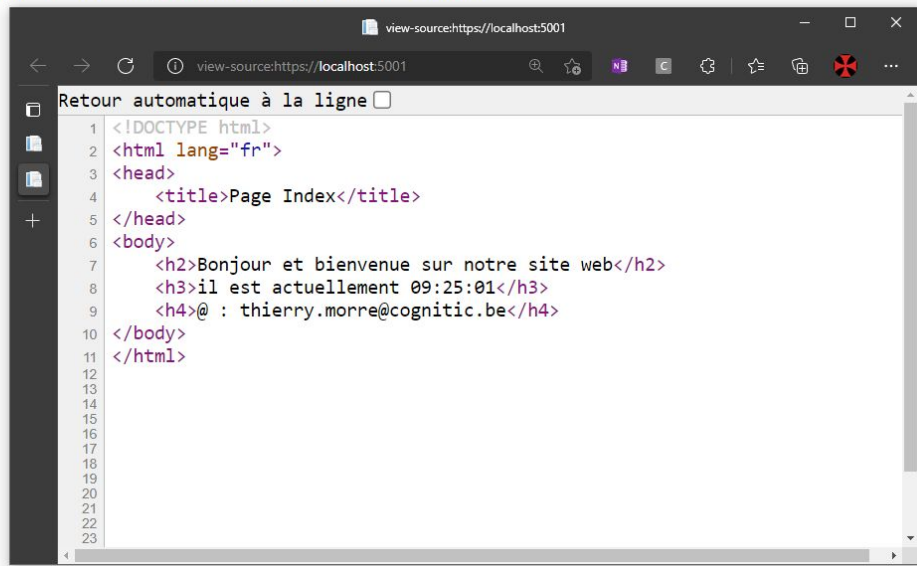
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <h2>Bonjour et bienvenue sur notre site web</h2>
</body>
</html>
```



LA SYNTAXE « RAZOR »

LE CARACTÈRE « @ »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <h2>Bonjour et bienvenue sur notre site web</h2>
  <h3>il est actuellement @DateTime.Now.ToString("hh:mm:ss")</h3>
  <h4>@@ : thierry.morre@cognitic.be</h4>
</body>
</html>
```



La syntaxe « Razor » supporte le « C# » et pour passer de l'un à l'autre, nous utiliserons le caractère « @ ».

Lorsque celui-ci est utilisé, le « Razor » évalue l'expression « C# » et ajoute le résultat à la page.

Pour échapper le caractère « @ » il suffit de le doubler.

De plus, le « @ » n'est pas traité dans le contenu « HTML », en tant que texte ou valeur d'attribut, lorsqu'il s'agit d'une adresse email.

LA SYNTAXE « RAZOR »

LES EXPRESSIONS IMPLICITES

Ce que l'on appelle, les expressions implicite, sont l'appel de fonctionnalité directement après le caractère « @ ».

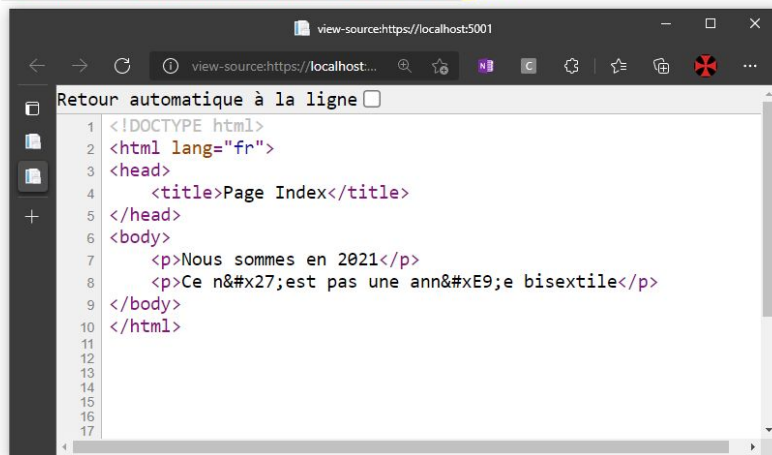
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <h2>Bonjour et bienvenue sur notre site web</h2>
  <p>Pi : @Math.PI</p>
  <p>2 exposant 3 : @Math.Pow(2, 3)</p>
</body>
</html>
```



LA SYNTAXE « RAZOR »

LES EXPRESSIONS EXPLICITES

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <p>Nous sommes en @DateTime.Now.Year</p>
  <p>@(DateTime.IsLeapYear(DateTime.Now.Year) ?
    "C'est une année bisextile" :
    "Ce n'est pas une année bisextile")</p>
</body>
</html>
```



Les expressions explicites, se différencie par la présence de parenthèses juste après le caractère « @ ».

L'expression est évaluée avant d'être rendue par « Razor »

LA SYNTAXE « RAZOR »

LES BLOCS D'INSTRUCTIONS

À l'inverse des expressions, implicites ou explicites, les blocs d'instructions ne sont pas rendu par « Razor ».

Ils servent à déclarer des variables, des méthodes locales et d'accéder aux propriétés de la vue (« Layout », « ViewBag », « ViewData », etc.),

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    string value = "Salut tout le monde!";
    int Addition(int x, int y)
    {
      return x + y;
    }
  }
  <h1>@value</h1>
  <h3>5 + 7 = @Addition(5, 7)</h3>
</body>
</html>
```



LA SYNTAXE « RAZOR »

TRANSITION DE LIGNE EXPLICITE

Pour afficher le reste d'une ligne entière au format HTML à l'intérieur d'un bloc de code, nous utiliserons la « @: ».



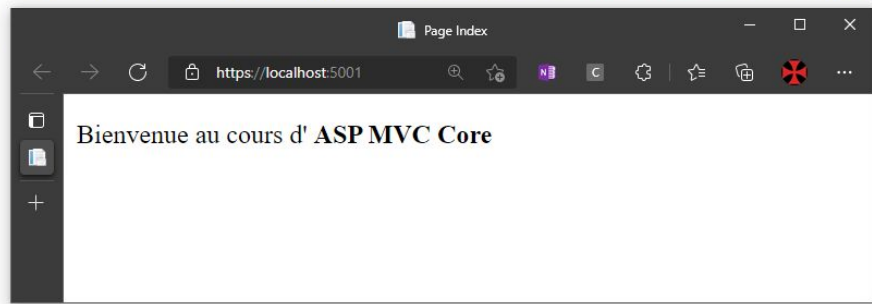
```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Page Index</title>
5 </head>
6 <body>
7   <h1>Salut tout le monde!</h1>
8   <h3>Nouvelle addition demandée 5 + 7 =
9 12</h3>
10 </body>
11 </html>
12
13
14
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    string value = "Salut tout le monde!";
    int Addition(int x, int y)
    {
      @:Nouvelle addition demandée @x + @y =
        return x + y;
    }
    <h1>@value</h1>
    <h3>@Addition(5, 7)</h3>
  }
</body>
</html>
```

LA SYNTAXE « RAZOR »

TEMPLATING METHODS

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    void AfficheEnGras(string value)
    {
      <strong>@value</strong>
    }
  }
  <p>Bienvenue au cours d'@{ AfficheEnGras("ASP MVC Core"); }</p>
</body>
</html>
```



En « Razor », le fait de mettre du code « HTML » dans une méthode local s'appelle le « templating method ».

Au moment d'appeler la méthode, « Razor » générera le code « HTML » à l'emplacement défini.

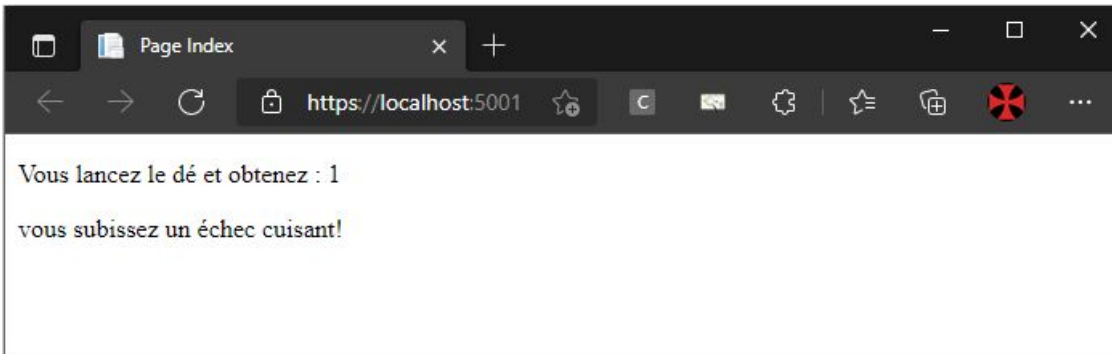
LA SYNTAXE « RAZOR »

LES INSTRUCTIONS CONDITIONNELLES

En « Razor », nous retrouvons les blocs d'instructions conditionnels du C#. À la différence qu'en « Razor », les accolades sont toujours obligatoires.

Nous retrouvons donc, les blocs « if » « else if », « else » et « switch ».

Les blocs « else if » et « else » restent facultatifs.



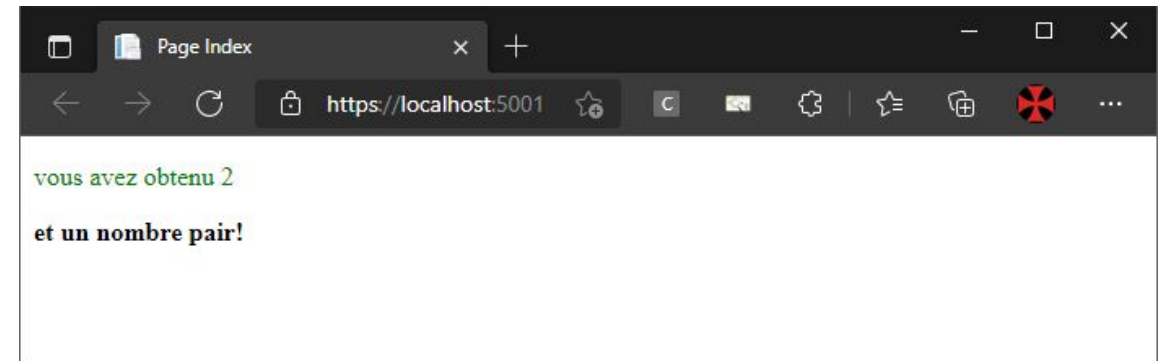
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    Random random = new Random();
    int value = random.Next(20) + 1;
  }

  <p>Vous lancez le dé et obtenez : @value</p>
  @if (value == 20) {
    <p>vous obtenez une réussite critique!</p>
  }
  else if (value >= 10) {
    <p>vous réussissez votre action!</p>
  }
  else if (value == 1) {
    <p>vous subissez un échec cuisant!</p>
  }
  else {
    <p>vous ratez votre action!</p>
  }
}
</body>
</html>
```

LA SYNTAXE « RAZOR »

LES INSTRUCTIONS CONDITIONNELLES

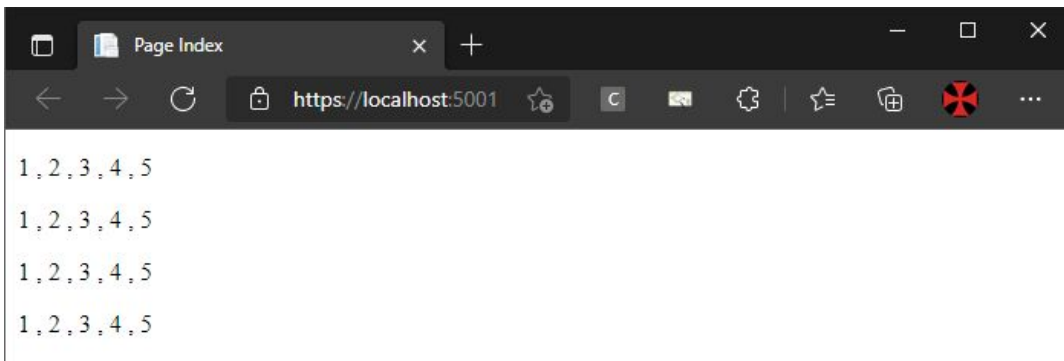
```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    Random random = new Random();
    int value = random.Next(4) + 1;
  }
  @switch (value)
  {
    case 1:
      <p style="color:blue">vous avez obtenu @value</p>
      break;
    case 2:
      <p style="color:green">vous avez obtenu @value</p>
      goto default;
    case 3:
      <p style="color:red">vous avez obtenu @value</p>
      break;
    case 4:
      <p style="color:orange">vous avez obtenu @value</p>
      goto default;
    default:
      <p style="color:black"><strong>et un nombre pair!</strong></p>
      break;
  }
</body>
</html>
```



LA SYNTAXE « RAZOR »

LES INSTRUCTIONS ITÉRATIVES

Il va de même pour les boucles « for », « while », « do while » et « foreach »



```
<!DOCTYPE html>
<html lang="fr">
<head>...</head>
<body>
    @{
        int[] tab = { 1, 2, 3, 4, 5 };
        int j = 0, k = 0;
    }

    <p>
        @for (int i = 0; i < 5; i++) {
            <span>@(i != 0 ? $", {tab[i]}" : $"{tab[i]}")</span>
        }
    </p>
    <p>
        @while (j < 5) {
            <span>@(j != 0 ? $", {tab[j++]}" : $"{tab[j++]}")</span>
        }
    </p>
    <p>
        @do {
            <span>@(k != 0 ? $", {tab[k++]}" : $"{tab[k++]}")</span>
        } while (k < 5);
    </p>
    <p>
        @foreach (var item in tab) {
            <span>@(item != 1 ? $", {item}" : $"{item}")</span>
        }
    </p>
</body>
</html>
```

LA SYNTAXE « RAZOR »

LE BLOC « USING »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <!-- génération du formulaire à l'aide d'un html helper -->
  @using (Html.BeginForm())
  {
    <div>
      Email: <input type="email" id="Email" value="">
    </div>
    <div>
      <button>Register</button>
    </div>
  }
</body>
</html>
```

Comme en C#, le bloc d'instruction « using » est utilisé sur les instances d'objets implémentant l'interface « IDisposable ».

Le but étant qu'en arrivant à l'accolade fermante du bloc, la méthode « Dispose » soit appelée automatiquement.

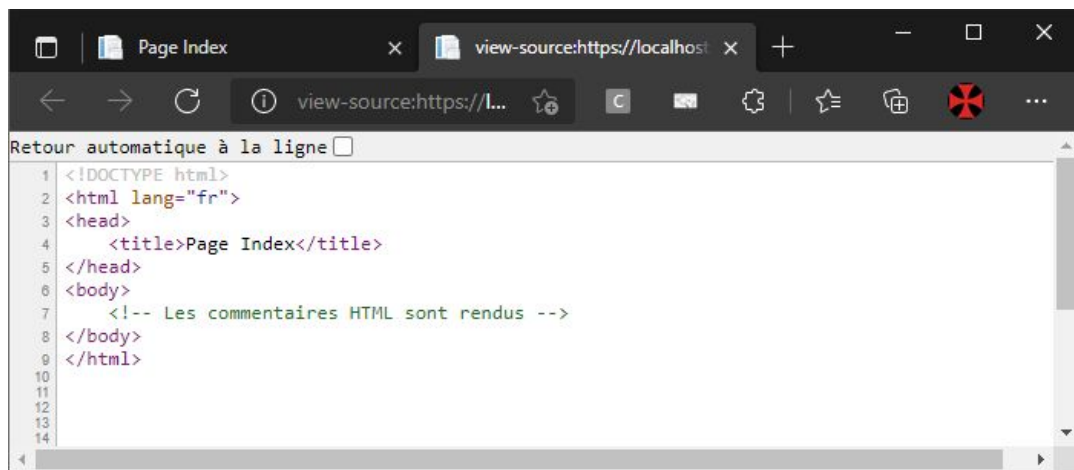


Les « Html Helpers » seront vus un peu plus loin

LA SYNTAXE « RAZOR »

LES COMMENTAIRES

La mise en commentaire en « Razor » se fait à l'aide des « @* » pour ouvrir et « */ » pour fermer.



```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Page Index</title>
5 </head>
6 <body>
7   <!-- Les commentaires HTML sont rendus -->
8 </body>
9 </html>
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @* Les commentaires Razor ne sont pas rendus */
  <!-- Les commentaires HTML sont rendus -->
</body>
</html>
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

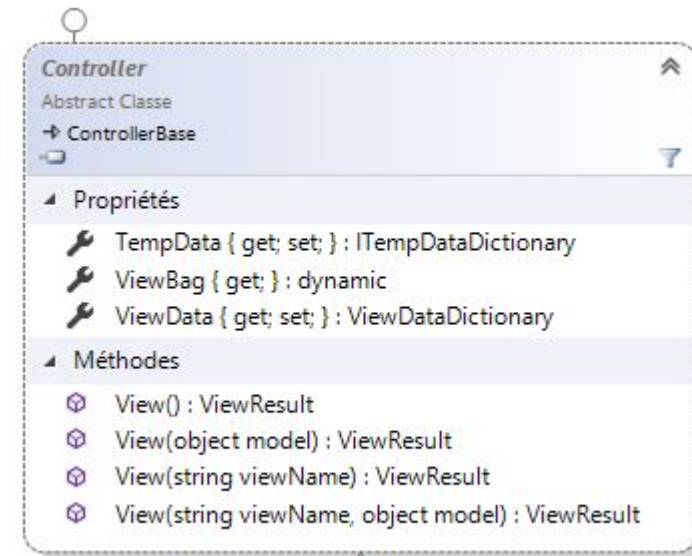
Il existe plusieurs moyens de passer des données d'un contrôleur à une vue, chacun ayant ses avantages et ses inconvénients.

Nous retrouvons :

- Le « ViewData » (Propriété ou Attribut)
- Le « ViewBag » (Propriété)
- Le « TempData » (Propriété)
- Le modèle de vue (Méthode)

Le choix sera principalement déterminé par les critères suivants :

- Donnée simple (ViewData / ViewBag)
- Complexe (Modèle de vue)
- Volatile (ViewData / ViewBag)
- Pseudo persistante (TempData)



PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

LE « VIEWDATA »

```
0 références
public class HomeController : Controller
{
    0 références
    public IActionResult Index()
    {
        ViewData["Value"] = 5;
        return View();
    }
    0 références
    public IActionResult Privacy()...
    Masqué pour des raisons de mise en page
    0 références
    public IActionResult Error()...
}
```

La propriété « ViewData » est de type « ViewDataDictionary » qui implémente l'interface « IDictionary<string, object> ».

Son utilisation reste donc celle d'un dictionnaire, attention que le type de valeur reste « object ».

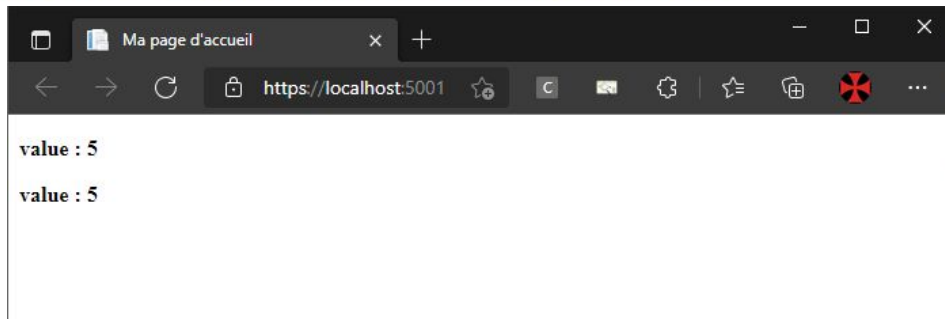
```
<!DOCTYPE html>
<html lang="fr">
<head>
    <title>Page Index</title>
</head>
<body>
    @* S'il n'y a pas de cast, il appelle la méthode ToString() par défaut *@
    <p><strong>value : @ViewData["Value"]</strong></p>
    <p><strong>value : @((int)ViewData["Value"])</strong></p>
</body>
</html>
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

L'ATTRIBUT « VIEWDATA »

Une autre manière de travailler avec le « ViewData » est d'utiliser un attribut sur une propriété qui fait en sorte que la valeur de la propriété est stockées et chargées automatiquement à partir du dictionnaire.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>@ViewData["Title"]</title>
</head>
<body>
  @* S'il n'y a pas de cast, il appelle la méthode ToString() par défaut *@
  <p><strong>value : @ViewData["Value"]</strong></p>
  <p><strong>value : @((int)ViewData["Value"])</strong></p>
</body>
</html>
```



```
public class HomeController : Controller
{
    [ViewData]
    public string Title { get; set; }

    public IActionResult Index()
    {
        Title = "Ma page d'accueil";
        ViewData["Value"] = 5;
        return View();
    }

    public IActionResult Privacy()...
    public IActionResult Error()...
}
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

LE « VIEWBAG »

```
public class HomeController : Controller
{
    [ViewData]
    public string Title { get; set; }

    public IActionResult Index()
    {
        Title = "Ma page d'accueil";
        ViewBag.Value = 5;
        return View();
    }

    public IActionResult Privacy()...
    Masqué pour des raisons de mise en page

    public IActionResult Error()...
}
```

La propriété « ViewBag » est, quant à elle, déclarée de type « Dynamic », cela simplifie l'écriture du code mais il n'y a plus d'intellisense.

Derrière cette propriété dynamique, se cache en réalité une valeur de type « DynamicViewData » qui hérite de « DynamicObject ».

Ce type, « DynamicViewData », fait le lien avec le dictionnaire « ViewData », ce qui explique pourquoi les données du « ViewBag » sont présentes dans le « ViewData » et inversement.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <title>@ViewBag.Title</title>
</head>
<body>
    <p><strong>value : @ViewBag.Value</strong></p>
    @* Derrière le ViewBag se trouve les données du ViewData *@
    <p><strong>value : @ViewData["Value"]</strong></p>
</body>
</html>
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

LE « TempData »

Le « TempData » est légèrement différent du « ViewData ».

Tout d'abord il est de type « TempDataDictionary » qui, tout comme pour le « ViewData », implémente l'interface « IDictionary<string, object> ».

Ensuite, le « ViewData » et le « ViewBag » sont limités à une seule requête « http ». Ce qui veut dire que s'il y a une redirection, ces derniers sont resettés et les données perdues.

Pourtant, dans certains processus, nous souhaiterons faire persister ces données comme par exemple l'url initiale ayant engendré la suite de requêtes.

Dans ce cas nous utiliserons le « TempData »

```
public class HomeController : Controller
{
    [ViewData]
    public string Title { get; set; }

    public IActionResult Index()
    {
        Title = "Ma page d'accueil";
        TempData["Value"] = 5;
        return RedirectToAction("Privacy");
    }

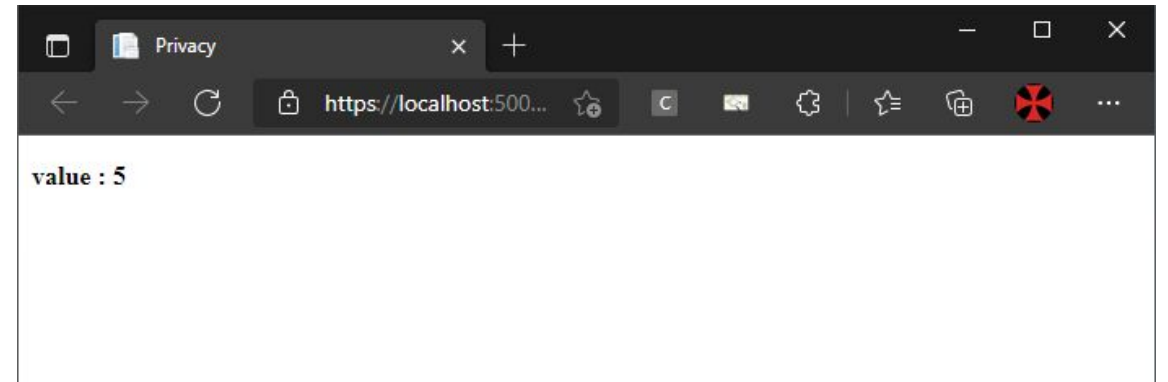
    public IActionResult Privacy()
    {
        Title = "Privacy";
        return View();
    }

    Masqué pour des raisons de mise en page
    public IActionResult Error()...
}
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

LE « TEMPDATA »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>@ViewBag.Title</title>
</head>
<body>
  <p><strong>value : @TempData["Value"]</strong></p>
</body>
</html>
```



PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

LE « TEMPDATA »

```
public class HomeController : Controller
{
    [ViewData]
    public string Title { get; set; }

    public IActionResult Index()
    {
        Title = "Ma page d'accueil";
        TempData["Value"] = 5;
        return RedirectToAction("Privacy");
    }

    public IActionResult Privacy()
    {
        Title = "Privacy";
        if (TempData.ContainsKey("Value"))
            TempData.Keep();

        return View();
    }

    public IActionResult Error()...
```

Dans le principe, la valeur disparaît du « TempData » si elle est récupérée.

Pour faire persister à plus d'une redirection, nous devons utiliser la méthode « Keep » du « TempData » ou nous pouvons également utiliser « Peek » pour lire la donnée sans la retirer du « TempData ».

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

TABLEAU RÉCAPITULATIF

	ViewData	ViewBag	TempData
Type	ViewDataDictionary	Dynamic	ITempDataDictionary
Fonctionnement	Dictionnaire	Dynamique	Dictionnaire
Durée de vie	Le temps de la requête « http »	Le temps de la requête « http »	Peut persister sur plusieurs appels

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

MODÈLE DEVUE

Pour l'envoi de données complexes à une vue, nous préférons plutôt l'utilisation d'un modèle de vue.

Ces modèles de vues sont en règle générale des classes possédant plusieurs propriétés ou des séquences d'objets.

Une fois instancié, nous passerons la valeur en paramètre de la méthode « View » utilisée par notre contrôleur.

```
namespace AspNet5Mvc.Models
{
    5 références
    public class ContactInformation
    {
        2 références
        public string Nom { get; set; }
        2 références
        public string Prenom { get; set; }
        2 références
        public string Adresse { get; set; }
        2 références
        public int CodePostal { get; set; }
        2 références
        public string Localite { get; set; }
    }
}
```

```
public class HomeController : Controller
{
    0 références
    public IActionResult Index()...

    0 références
    public IActionResult Contact()
    {
        ContactInformation contactInformation = new ContactInformation()
        {
            Nom = "Morre",
            Prenom = "Thierry",
            Adresse = "Rue du voyageur galactique, 42",
            CodePostal = 1000,
            Localite = "Bruxelles"
        };

        return View(contactInformation);
    }

    0 références
    public IActionResult Privacy()...
    Masqué pour des raisons de mise en page
    0 références
    public IActionResult Error()...
}
```

PASSER DES DONNÉES DU CONTRÔLEUR À LA VUE

MODÈLE DEVUE

```
@using AspNetMvc.Models
@model ContactInformation

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Contact</title>
</head>
<body>
    <address>
        <p>@Model.Nom @Model.Prenom</p>
        <p>@Model.Adresse à</p>
        <p>@Model.CodePostal @Model.Localite</p>
    </address>
</body>
</html>
```

Ensuite, au niveau de la vue, nous devons spécifier le type de model que notre vue va recevoir.

Pour cela, nous utiliserons « @model ... » en début de page.

Pour nous faciliter la tâche, nous pouvons aussi utiliser la syntaxe utilisée en C# pour utiliser un espace de noms « @using namespace », cependant comme il s'agit de « Razor » on doit le précéder d'un « @ ».

Pour terminer, la valeur de notre objet est accessible par la propriété « Model » à l'intérieur de la vue.



LES FORMULAIRES

ASP .NET 6.0 MVC

- Mode de fonctionnement
- Les verbes « http »
- Création du formulaire
- Traitement du formulaire
- Utilisation d'un modèle de vue
- Validation du formulaire
- Les attributs de validations
- Générer le formulaire avec les « HtmlHelpers »
- Générer le formulaire avec les « TagHelpers »
- Les attributs « Display » et « Edit »

LES FORMULAIRES

Asp .Net 6.0 MVC

MODE DE FONCTIONNEMENT

L'interaction avec l'utilisateur se fait principalement au travers de liens et de formulaires.

Cependant, il est bon de rappeler le fonctionnement d'un formulaire.

Il y a toujours deux appels du client vers l'application.

1. La demande du formulaire
2. Le traitement du formulaire

Lors de la déclaration de la balise « form » nous devons spécifier l'action (par défaut l'action courante) et la méthode utilisée « GET » ou « POST » (par défaut « GET »).

Le protocole « HTTP » définit un ensemble de méthodes de requête, dont « GET » et « POST » font partie, pour indiquer le type de requête à effectuer pour une ressource donnée.

Ces méthodes de requête sont aussi appelées verbes « HTTP »

```
{  
  "HttpVerb": "GET",  
  "Protocol": "HTTP/2",  
  "Scheme": "https",  
  "HostUrl": {  
    "Value": "localhost:5001",  
    "Host": "localhost",  
    "Port": 5001  
  },  
  "Controller": "Auth",  
  "Action": "Login",  
  "Parameters": []  
}
```

Informations de la première requête :
La demande de récupération du formulaire

```
{  
  "HttpVerb": "POST",  
  "Protocol": "HTTP/2",  
  "Scheme": "https",  
  "HostUrl": {  
    "Value": "localhost:5001",  
    "Host": "localhost",  
    "Port": 5001  
  },  
  "Controller": "Auth",  
  "Action": "Login",  
  "Parameters": [  
    {  
      "Name": "formCollection",  
      "ParameterType": "Microsoft.AspNetCore.Http.IFormCollection",  
      "Value": "Microsoft.AspNetCore.Mvc.Controllers.ControllerParameterDescriptor"  
    }  
  ]  
}
```

Informations de la deuxième requête :
La demande de traitement du formulaire

LES VERBES « HTTP »

HTTP Verb	Wep App	Web Api	Description	Attribut C#
GET	Oui	Oui	Envoi une requête de récupération identifiée par l'URI de la demande	[HttpGet]
POST	Oui	Oui	Envoi une requête de publication à un URI de demande.	[HttpPost]
PUT	Non	Oui	Envoi une requête de remplacement à un URI de demande.	[HttpPut]
DELETE	Non	Oui	Envoi une requête de suppression à un URI de demande.	[HttpDelete]
PATCH	Non	Oui	Envoi une requête pour appliquer un ensemble de changements décrits dans l'entité de demande pour une ressource identifiée par l'URI de demande.	[HttpPatch]

CRÉATION DU FORMULAIRE

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Login</title>
</head>
<body>
  <form method="post">
    <div>
      <label for="Email">Email : </label>
      <input type="email" name="Email" />
    </div>
    <div>
      <label for="Passwd">Password : </label>
      <input type="password" name="Passwd" />
    </div>
    <div>
      <input type="submit" value="Login" />
    </div>
  </form>
</body>
</html>
```

La création d'un formulaire ne diffère en rien de la création d'une vue classique, mise à part que dans le code « Razor » nous y retrouverons une balise « form » contenant les balises « label » et « input » de notre formulaire.

```
public class AuthController : Controller
{
    0 références
    public IActionResult Login()
    {
        return View();
    }
}
```

TRAITEMENT DU FORMULAIRE

Comme par défaut, l'attribut action de notre formulaire est l'action courante, au moment de cliquer sur le bouton de soumission du formulaire celui-ci sera envoyé à la même action du même contrôleur.

De ce fait nous rencontrons deux problèmes :

- Un problème lié au langage C#
- Un problème lié au verbe « HTTP »

Le problème C# est lié au principe de surcharge de méthode ce qui implique que notre méthode de traitement doit avoir une signature différente de celle qui renvoie le formulaire.

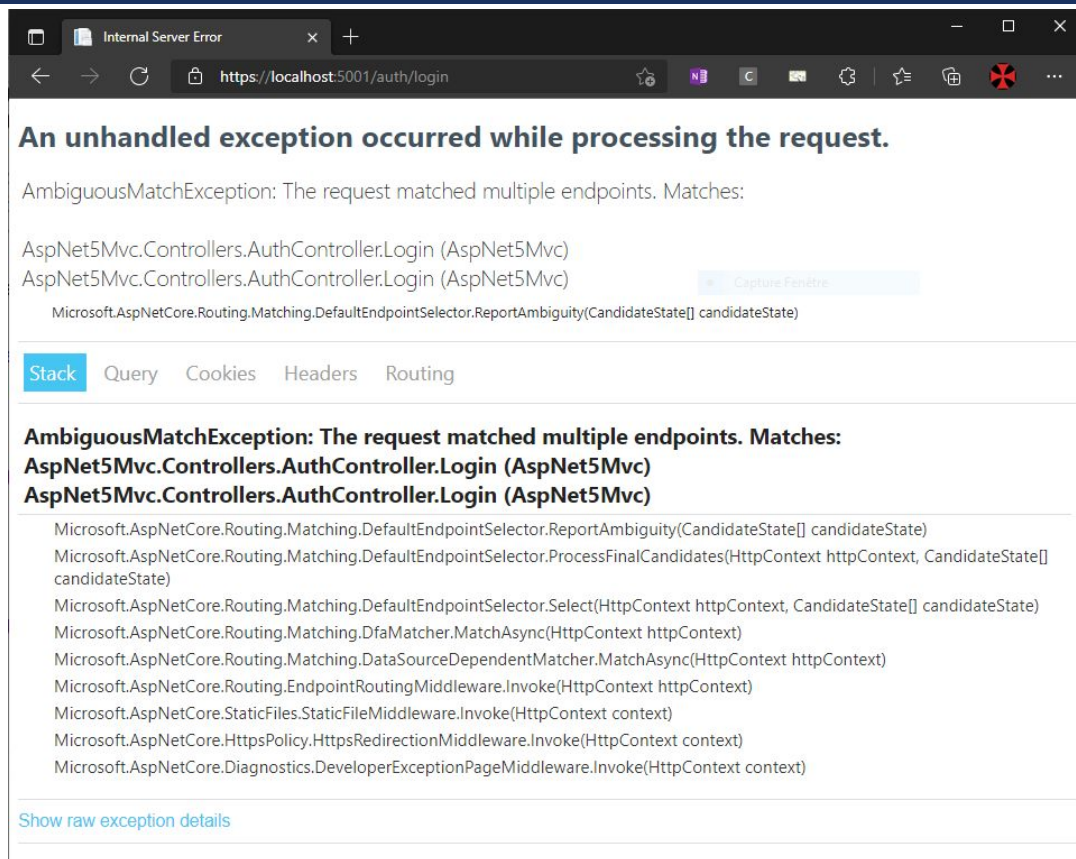


Le type « IFormCollection » sera vu dans quelques slides

```
public class AuthController : Controller
{
    0 références
    public IActionResult Login()
    {
        return View();
    }

    0 références
    public IActionResult Login(IFormCollection formCollection)
    {
        return View();
    }
}
```

TRAITEMENT DU FORMULAIRE



Le deuxième problème est lié au verbe « HTTP ».

En effet, le moteur « MVC » traite, par défaut, toute demande en « GET ».

Par conséquent, si je démarrais mon application telle quelle, je me retrouverais face à une erreur au moment de récupérer le formulaire.

Cette erreur nous indique que le moteur « MVC » se trouve face à une ambiguïté au moment de retourner le formulaire car deux actions portent le même nom et qu'elles sont toutes deux en « GET ».

Nous devons donc expliquer au moteur « MVC » laquelle gèrera le « GET » et laquelle gèrera le « POST ».

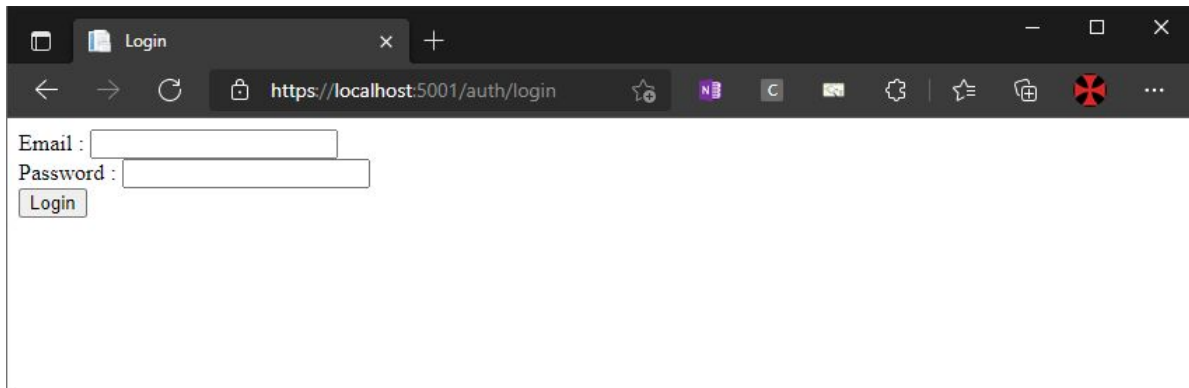
TRAITEMENT DU FORMULAIRE

Pour cela nous utiliserons les attributs « HttpGet » et « HttpPost ».

Ces attributs servent à spécifier à quel verbe « HTTP » est associée l'action.

L'attribut « HttpGet » est utilisé par défaut et est donc purement facultatif.

En règle générale, quand il est spécifié, il améliore la lecture du code.

A screenshot of a web browser window. The address bar shows the URL "https://localhost:5001/auth/login". The page content includes a form with two input fields: "Email :" and "Password :". Below these fields is a button labeled "Login". The browser's tab is titled "Login".

```
public class AuthController : Controller
{
    0 références
    public IActionResult Login()
    {
        return View();
    }

    [HttpPost]
    0 références
    public IActionResult Login(IFormCollection formCollection)
    {
        return View();
    }
}
```

RÉCUPÉRATION DES DONNÉES - IFORMCOLLECTION

Par défaut, le moteur « MVC » propose de récupérer les données du formulaire en ajoutant à la méthode un paramètre de type « `IFormCollection` » qui implémente l'interface « `IEnumerable<KeyValuePair<string, StringValues>>` ».

Dans lequel nous retrouverons nos valeurs de formulaires.

Attentions, il ne s'agit pas d'un dictionnaire car elle peut contenir plusieurs fois la même clé.



Utiliser le type « `FormCollection` » comme paramètre générera une exception de type « `InvalidOperationException` »

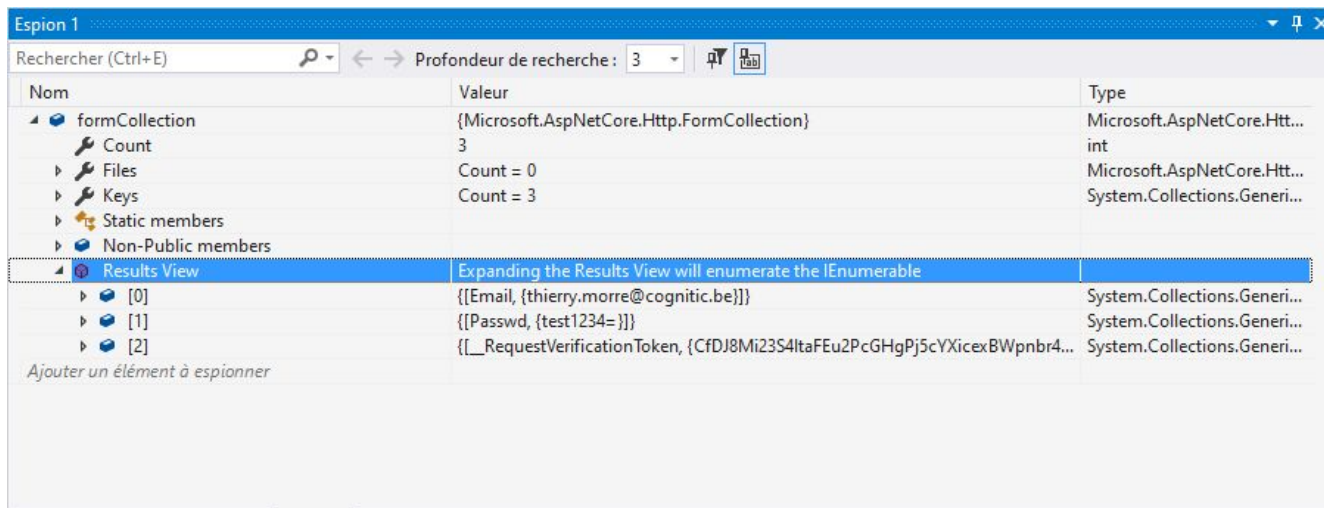
```
...public interface IFormCollection : IEnumerable<KeyValuePair<string, StringValues>>, IEnumerable
{
    ...StringValues this[string key] ...
    ...int Count { get; }
    ...ICollection<string> Keys { get; }
    ...IFormFileCollection Files { get; }

    ...bool ContainsKey(string key);
    ...bool TryGetValue(string key, out StringValues value);
}
```

RÉCUPÉRATION DES DONNÉES - IFORMCOLLECTION

Les valeurs reçues par l'indexeur est de type « StringValues » n'est rien d'autre qu'une séquence de « string » puisque ce type implémente « IEnumerable<string> ».

Nous sommes donc responsable de la récupération, de la conversion des données, de la validation de ces dernières.



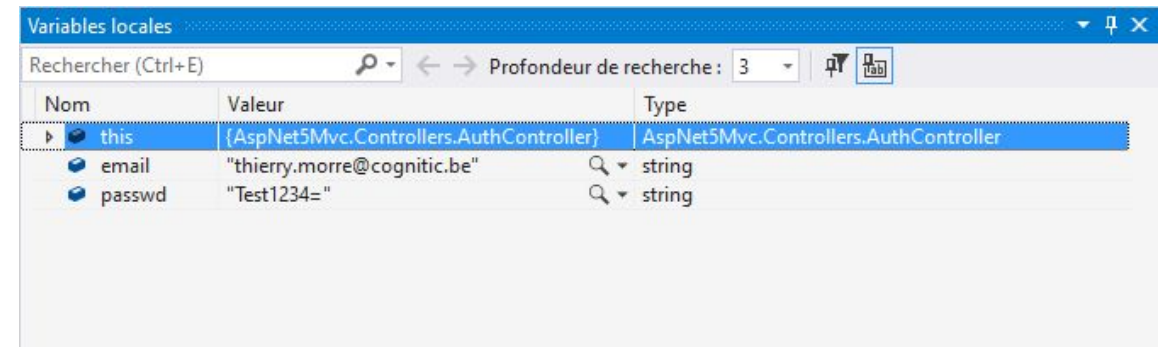
RÉCUPÉRATION DES DONNÉES - PAR PARAMÈTRES

Une autre possibilité est d'utiliser des paramètres au niveau de la méthode.

Ces paramètres devront avoir le même nom que dans notre formulaire indifféremment à la casse et le type devra être compatible au type de donnée du formulaire.

L'avantage par rapport au paramètre de type « `IFormCollection` » c'est que la conversion est gérée par le moteur « MVC » au moment de l'appelle de l'action.

```
[HttpPost]
0 références
public IActionResult Login(string email, string passwd)
{
    return View();
}
```



Nom	Valeur	Type
this	{AspNetMvc.Controllers.AuthController}	AspNetMvc.Controllers.AuthController
email	"thierry.morre@cognitic.be"	string
passwd	"Test1234="	string

UTILISATION D'UN MODÈLE DEVUE

```
public class LoginForm
{
    0 références
    public string Email { get; set; }
    0 références
    public string Passwd { get; set; }
}

public class AuthController : Controller
{
    0 références
    public IActionResult Login()
    {
        return View();
    }

    [HttpPost]
    0 références
    public IActionResult Login(LoginForm form)
    {
        return View();
    }
}
```

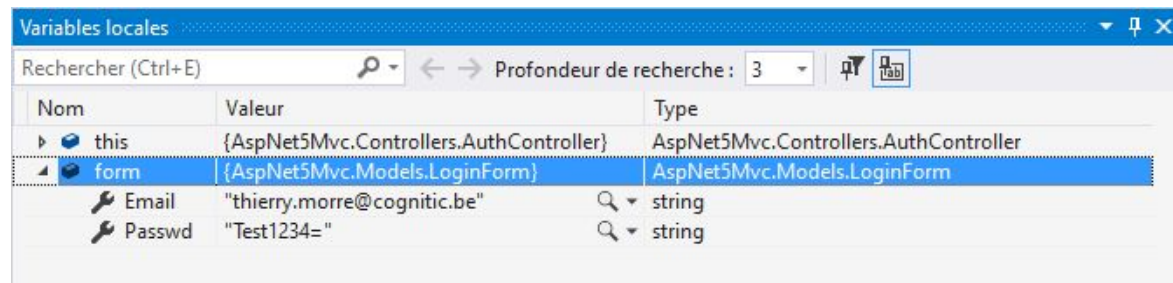
Utiliser des paramètres, bien que pratiques, peut rapidement devenir ingérable lorsque notre formulaire sera plus complexe que ce soit en nombre de champs et/ou la validation des données.

Raisons pour lesquelles nous aurons tendance à utiliser un modèle de vue pour récupérer les données.

Ce modèle contiendra des **propriétés** ayant pour nom le nom des champs de notre formulaire et devra avoir un constructeur sans paramètre (qu'il s'agisse du constructeur par défaut ou non).

Car c'est le moteur « MVC » qui instancie notre modèle de vue, se charge de la conversion des données.

En principe, nous nous contenterons d'auto-propriété.



Nom	Valeur	Type
this	{AspNet5Mvc.Controllers.AuthController}	AspNet5Mvc.Controllers.AuthController
form	{AspNet5Mvc.Models.LoginForm}	AspNet5Mvc.Models.LoginForm
Email	"thierry.morre@cognitic.be"	string
Passwd	"Test1234="	string

VALIDATION DU FORMULAIRE

La validation d'un formulaire passe par la propriété « ModelState » de notre contrôleur.

Cette propriété est de type « ModelStateDictionary ».

Le « ModelState » possède plusieurs propriétés dont :

- IsValid : retourne si le dictionnaire ne contient pas d'erreurs
- ErrorCount : retourne le nombre d'erreur ajouté au dictionnaire

Ainsi que des méthodes dont :

- AddModelError : Permet d'ajouter une erreur au « ModelState »
- Clear : Vide le ModelState

```
[HttpPost]
0 référence
public IActionResult Login(LoginForm form)
{
    ValidateLoginForm(form, ModelState);
    if(!ModelState.IsValid)
        return View();
    else
    {
        //Traitement à réaliser si le formulaire n'a généré aucune erreur
        return RedirectToAction("Index", "Home");
    }
}

1 référence
private static void ValidateLoginForm(LoginForm form, ModelStateDictionary modelState)
{
    if(string.IsNullOrEmpty(form.Email))
    {
        modelState.AddModelError(nameof(form.Email), "Email est obligatoire");
    }

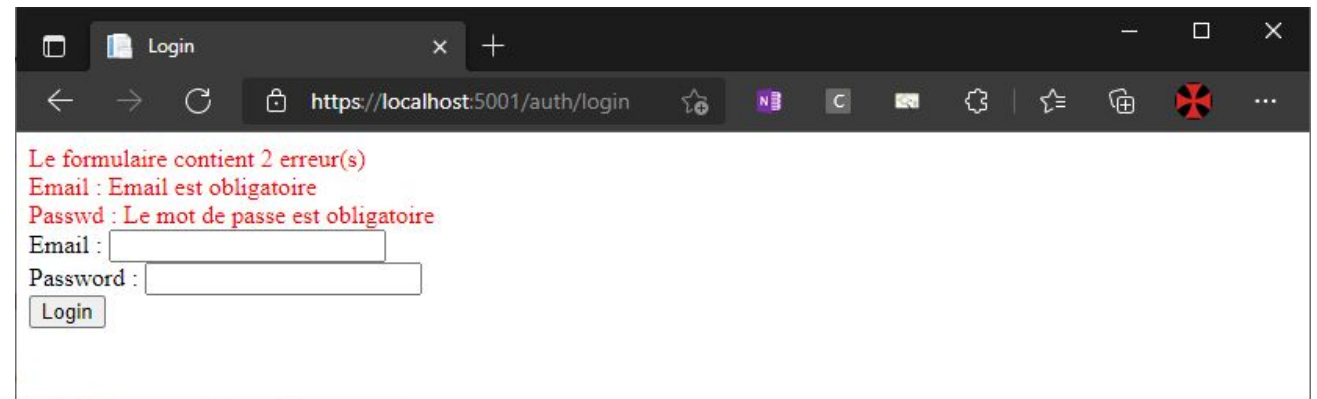
    if (string.IsNullOrEmpty(form.Passwd))
    {
        modelState.AddModelError(nameof(form.Passwd), "Le mot de passe est obligatoire");
    }

    if (form.Passwd is not null &&
        !Regex.IsMatch(form.Passwd,
            @"^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&-+=()])?(?=\S+$).{8, 20}$"))
    {
        modelState.AddModelError(nameof(form.Passwd), "le mot de passe n'est pas assez fort");
    }
}
```

VALIDATION DU FORMULAIRE

```
@using Microsoft.AspNetCore.Mvc.ModelBinding
<!DOCTYPE html>
<html lang="fr">
<head>
    <title>Login</title>
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)
    {
        <div>
            <span style="color:red;">
                Le formulaire contient @ViewContext.ModelState.ErrorCount erreur(s)
            </span><br />
            @foreach (KeyValuePair<string, ModelStateEntry> kvp in ViewContext.ModelState)
            {
                if (kvp.Value.Errors.Count() > 0)
                {
                    foreach (ModelError modelError in kvp.Value.Errors)
                    {
                        <span style="color:red;">
                            @kvp.Key : @modelError.ErrorMessage
                        </span><br />
                    }
                }
            }
        </div>
    }
    <form>...</form>
</body>
</html>
```

Ensuite pour afficher les erreurs dans la vue, nous pouvons retrouver notre « ModelState » dans la propriété « ViewContext » et parcourir ce dernier pour l'affichage des erreurs.

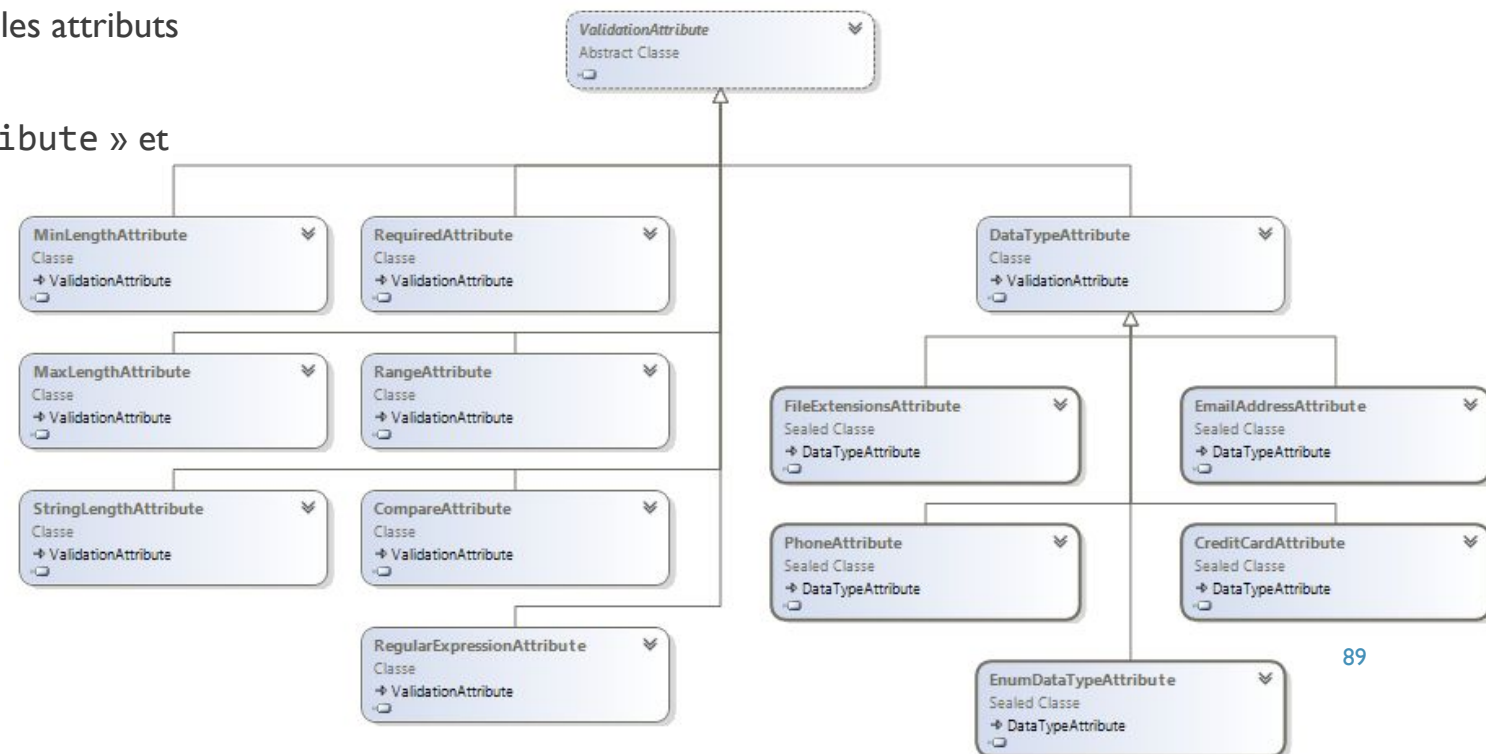


LES ATTRIBUTS DE VALIDATIONS

Faire de la validation sous cette forme, va rapidement s'avérer lourde et fastidieuse aussi bien pour la détection d'erreur et pour l'affichage de ces dernières.

Afin de faciliter notre développement, nous pouvons utiliser les attributs de validation.

Ces attributs héritent tous de la classe « ValidationAttribute » et ils s'appliquent à nos propriétés.



LES ATTRIBUTS DE VALIDATIONS

Validation Attribute	Description
Required	Spécifie que la propriété est obligatoire
MinLength	Spécifie une longueur minimale qu'une chaîne de caractères doit avoir
MaxLength	Spécifie une longueur maximale qu'une chaîne de caractères doit avoir
StringLength	Regroupe « MinLength » et « MaxLength » en un seul attribut
Compare	Compare la valeur d'une propriété à la valeur d'une autre
Range	Spécifie que la valeur doit se trouver dans une plage de valeurs données
RegularExpression	Spécifie que la valeur de la chaîne de caractères doit respecter une expression régulière
FileExtension	Spécifie les extensions de fichiers autorisée (ex : [FileExtensions(Extensions="jpg,gif,png")])
EmailAddress	Spécifie que la valeur de la chaîne de caractères doit être une adresse email
CreditCard	Spécifie que la valeur de la chaîne de caractères doit être une carte de crédit
EnumDataType	Spécifie que la valeur de la chaîne de caractères doit être présente dans l'énumération

LES ATTRIBUTS DE VALIDATIONS

Modèle de vue

```
public class LoginForm
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }
    [Required]
    [RegularExpression(@"^(?=[0-9])(?=[a-z])(?=[A-Z])(?=[@#$$%^&-+=()]).{8, 20}$")]
    public string Passwd { get; set; }
}
```

Dans l'action du contrôleur

```
[HttpPost]
public IActionResult Login(LoginForm form)
{
    if(!ModelState.IsValid)
        return View();
    else
    {
        //Traitement à réaliser si le formulaire n'a généré aucune erreur
        return RedirectToAction("Index", "Home");
    }
}
```

En utilisant les attributs de validations, nous venons de faciliter le traitement au niveau du contrôleur en une simple condition :
« if(!ModelState.IsValid) »

LES ATTRIBUTS DE VALIDATIONS

Il est bien entendu possible de customiser les messages d'erreurs simplement en ajoutant « ErrorMessage » après les paramètres obligatoires dans la déclaration de l'attribut.

```
public class LoginForm
{
    [Required(ErrorMessage="Le champs 'Email' est obligatoire")]
    [EmailAddress(ErrorMessage="Ce n'est pas une adresse email")]
    3 références
    public string Email { get; set; }
    [Required(ErrorMessage="Le champs 'Mot de passe' est obligatoire")]
    [RegularExpression(@"^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&-+=()]).{8, 20}$",
        ErrorMessage="Ce n'est pas un mot de passe fort")]
    3 références
    public string Passwd { get; set; }
}
```

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Les « HTML Helpers » sont utilisés dans les vues pour restituer le contenu HTML.

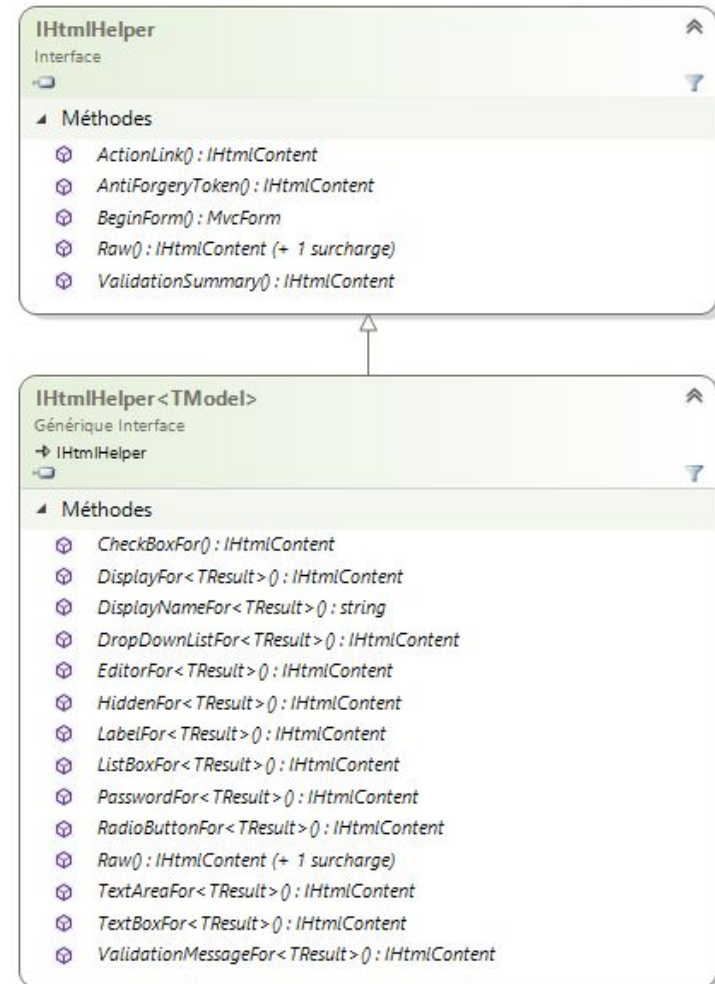
Celle-ci sont accessible par la propriété « Html » de la vue qui implémente « `IHtmlHelper` » par défaut et lorsque votre vue se voit associer un modèle elle implémente aussi « `IHtmlHelper<TModel>` ».

Ces deux interfaces proposent des méthodes ayant pour but de simplifier la création des vues.

En effet, celles-ci peuvent, entre autres choses, vérifier la présence des attributs de votre modèle par exemple, voir si la propriété de votre modèle a des erreurs dans le « `ModelState` », voir simplement créer du « HTML », etc.

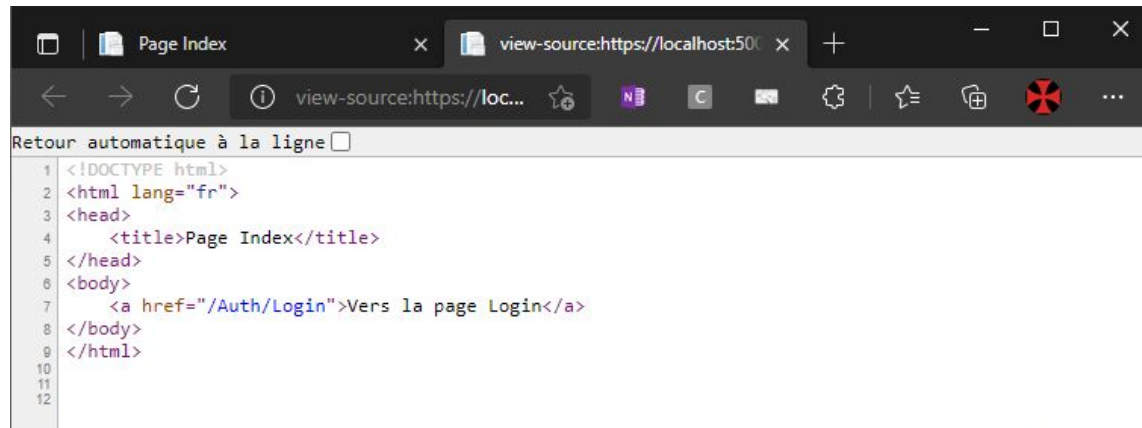


Ces deux interfaces proposent plus de choix en terme de méthodes et de propriétés.



GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @Html.ActionLink("Vers la page Login", "Login", "Auth")
</body>
</html>
```



```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Page Index</title>
5 </head>
6 <body>
7   <a href="/Auth/Login">Vers la page Login</a>
8 </body>
9 </html>
```

Avant de commencer la mise en place des « Html Helper » permettant de créer les formulaires, attardons nous un peu sur deux d'entre eux.

Tout d'abord, la méthode « ActionLink » qui permet de générer rapidement des liens de navigation pour la structure « MVC » et de garantir que les liens soient correctement générés.

Des méthodes d'extensions existent et sont reprises dans la classe « HtmlHelperLinkExtensions ».



« RouteLink » se base sur les routes

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Ensuite, la méthode « Raw » quant à elle permet d'encapsuler un balisage « HTML » contenu dans une variable afin qu'il soit interprété comme un balisage « HTML » normal.



```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Page Index</title>
5 </head>
6 <body>
7   <div>
8     &lt;h1&gt;ma page&lt;/h1&gt;
9     <h1>ma page</h1>
10  </div>
11  <div>
12    <a href="/Auth/Login">Vers la page Login</a>
13  </div>
14 </body>
15 </html>
```

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  @{
    string value = "<h1>ma page</h1>";
  }
  <div>
    @value
    @Html.Raw(value)
  </div>
  <div>
    @Html.ActionLink("Vers la page Login", "Login", "Auth")
  </div>
</body>
</html>
```



GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Reprenons notre formulaire précédent, auquel j'ai ajouté « Bootstrap » pour les exemples qui suivent au niveau de la gestion des classes css.

```
@using Microsoft.AspNetCore.Mvc.ModelBinding

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)
    {
        <div>
            <span class="text-danger">
                Le formulaire contient @ViewContext.ModelState.ErrorCount erreur(s)
            </span><br />
            @foreach(KeyValuePair<string, ModelStateEntry> kvp in ViewContext.ModelState)
            {
                if(kvp.Value.Errors.Count() > 0)
                {
                    foreach (ModelError modelError in kvp.Value.Errors)
                    {
                        <span class="text-danger">
                            @kvp.Key : @modelError.ErrorMessage
                        </span><br />
                    }
                }
            }
        </div>
    }
    <div class="row">...</div>
</body>
</html>
```

```
@using Microsoft.AspNetCore.Mvc.ModelBinding

<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)...
    <div class="row">
        <div class="col-md-6">
            <form method="post">
                <div class="form-group">
                    Email : <input type="email" class="form-control" />
                </div>
                <div class="form-group">
                    Passwd : <input type="password" class="form-control" />
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Commençons à retoucher notre formulaire en utilisant les « Html Helpers ». Nous pouvons commencer par remplacer le bloc d'affichage des erreurs en utilisant la méthode « ValidationSummary() » dont le rôle consiste à afficher un sommaire des erreurs du « ModelState ».

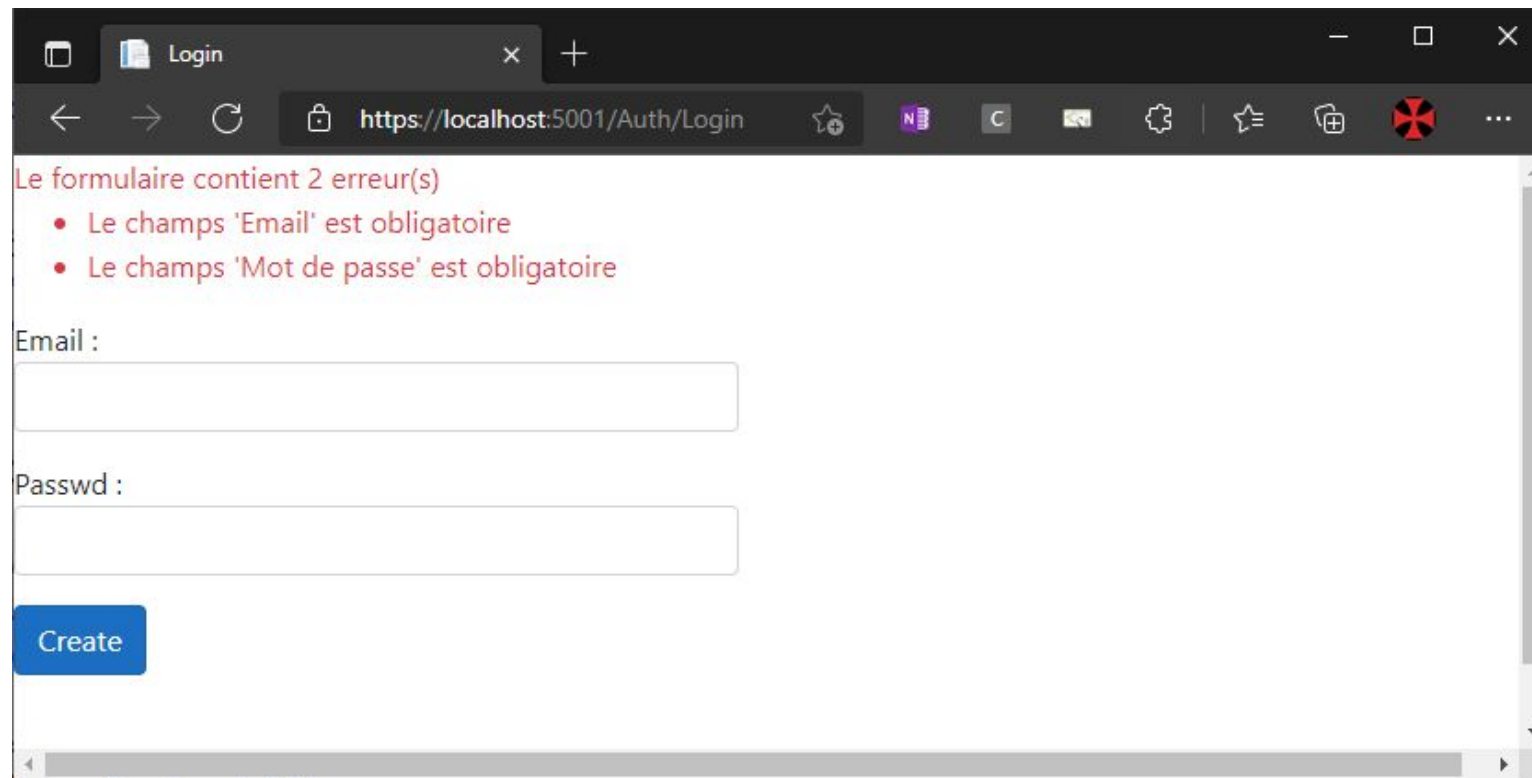
La valeur « false » signifie qu'on n'exclut pas les erreurs du modèle.

```
@if (!ViewContext.ModelState.IsValid)
{
    <div>
        <span class="text-danger">
            Le formulaire contient @ViewContext.ModelState.ErrorCount erreur(s)
        </span><br />
        @foreach(KeyValuePair<string, ModelStateEntry> kvp in ViewContext.ModelState)
        {
            if(kvp.Value.Errors.Count() > 0)
            {
                foreach (ModelError modelError in kvp.Value.Errors)
                {
                    <span class="text-danger">
                        @kvp.Key : @modelError.ErrorMessage
                    </span><br />
                }
            }
        }
    </div>
}
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)
    {
        @Html.ValidationSummary(false,
            $"Le formulaire contient {ViewContext.ModelState.ErrorCount} erreur(s)",
            new { @class = "text-danger" });
    }

    <div class="row">
        <div class="col-md-6">
            <form method="post">
                <div class="form-group">
                    Email : <input type="email" class="form-control" />
                </div>
                <div class="form-group">
                    Passwd : <input type="password" class="form-control" />
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »



The screenshot shows a web browser window with the title 'Login' and the address bar displaying 'https://localhost:5001/Auth/Login'. The page content includes a red error message at the top: 'Le formulaire contient 2 erreur(s)', followed by a bulleted list of errors: 'Le champs 'Email' est obligatoire' and 'Le champs 'Mot de passe' est obligatoire'. Below the errors are two input fields: 'Email :' and 'Passwd :'. A blue 'Create' button is positioned below the 'Passwd :' field.

Le formulaire contient 2 erreur(s)

- Le champs 'Email' est obligatoire
- Le champs 'Mot de passe' est obligatoire

Email :

Passwd :

Create

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  @if (!ViewContext.ModelState.IsValid)
  {
    @Html.ValidationSummary(true, null, new { @class = "text-danger" });
  }
  <div class="row">
    <div class="col-md-6">
      <form method="post">
        <div class="form-group">
          Email : <input type="email" class="form-control" />
          @Html.ValidationMessageFor(expression: model => model.Email,
            message: null,
            htmlAttributes: new { @class = "text-danger" });
        </div>
        <div class="form-group">
          Passwd : <input type="password" class="form-control" />
          @Html.ValidationMessageFor(model => model.Passwd, null, new { @class = "text-danger" })
        </div>
        <div class="form-group">
          <input type="submit" value="Create" class="btn btn-primary" />
        </div>
      </form>
    </div>
  </div>
</body>
</html>
```

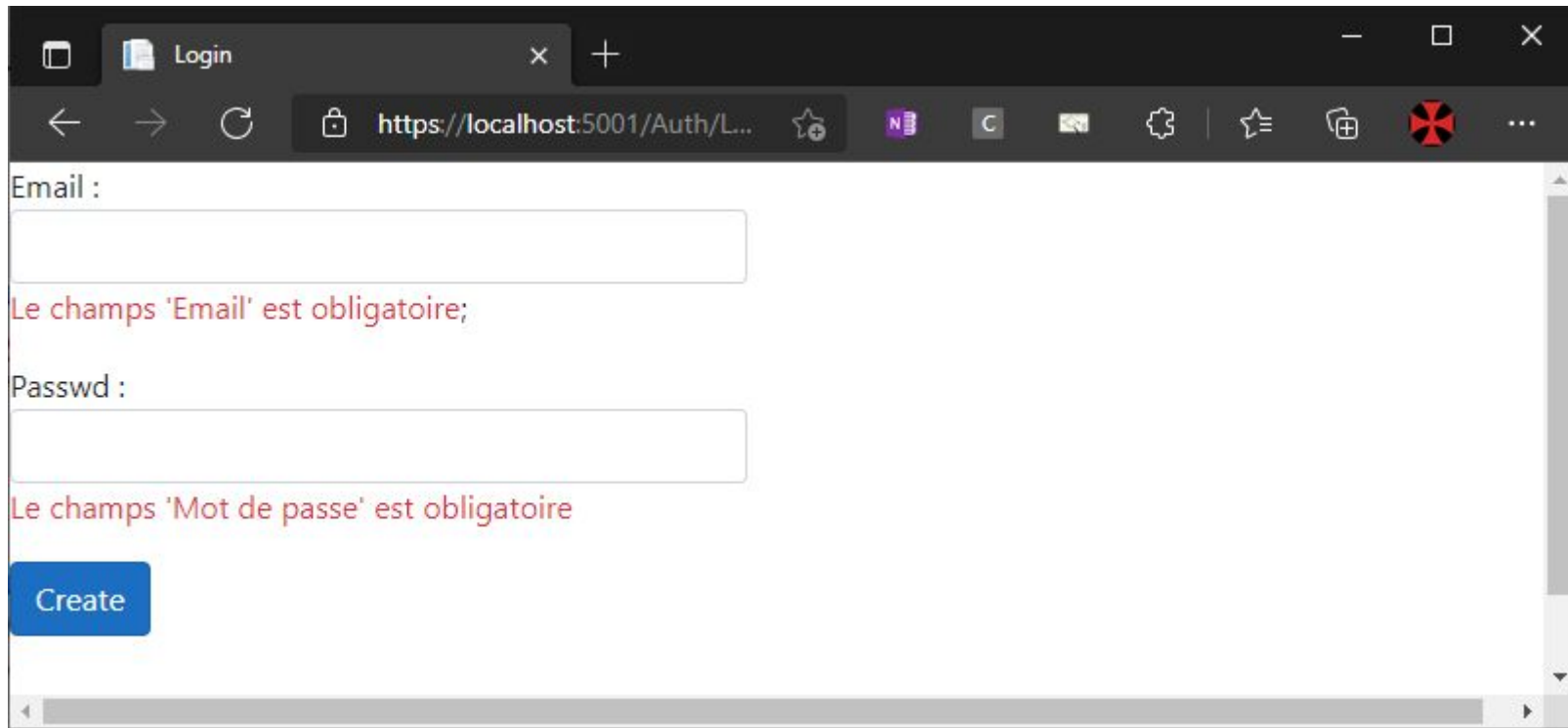
Ensuite, en complément au « ValidationSummary », nous pouvons ajouter la méthode « ValidationMessageFor ».

Son rôle consiste à afficher le message d'erreur d'une propriété du modèle.

Afin de pouvoir l'utiliser, nous allons ajouter « @model LoginForm » à notre page et mettre « true » à notre « ValidationSummary » pour éviter que les erreurs n'apparaissent en double.

- **expression** : définit la propriété pour laquelle nous voulons afficher le message.
- **message** : définit la chaîne de caractères que nous voudrions toujours afficher.
- **htmlAttributes** : permet de spécifier les attributs « HTML » que nous souhaitons ajouter à la balise « span » qui sera générée.

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

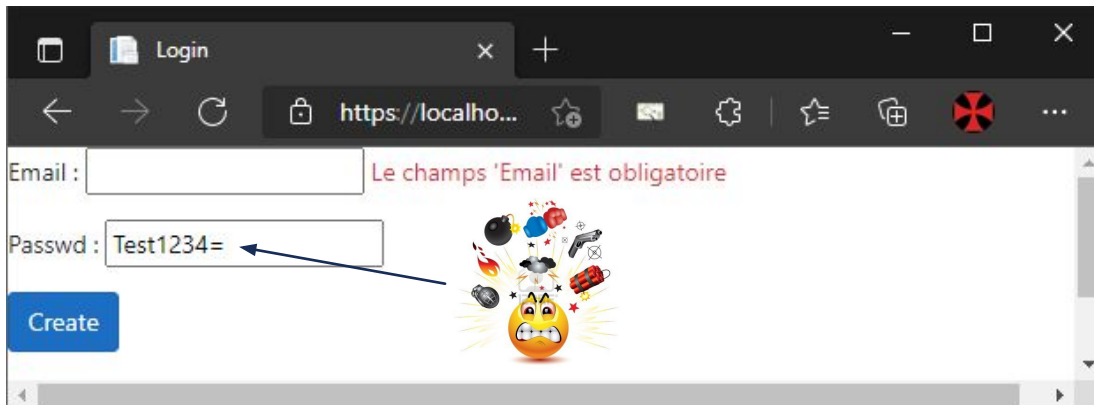


A screenshot of a web browser window displaying a login form. The browser's address bar shows the URL `https://localhost:5001/Auth/L...`. The form contains two input fields: "Email :" and "Passwd :". Below the "Email :" field, a red validation message reads "Le champs 'Email' est obligatoire;". Below the "Passwd :" field, another red validation message reads "Le champs 'Mot de passe' est obligatoire". At the bottom left of the form is a blue button labeled "Create". The browser window has a dark theme and includes standard navigation and window control buttons.

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Ensuite, nous pouvons remplacer nos balises « input » par des méthodes « EditorFor ».

Seule ombre au tableau est, que par défaut, la balise « input » générée sera de type « text ».



```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="/css/site.css" />
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)
    {
        @Html.ValidationSummary(true, null, new { @class = "text-danger" })
    }
    <div class="row">
        <div class="col-md-6">
            <form method="post">
                <div class="form-group">
                    Email : @Html.EditorFor(model => model.Email, new { @class="form-control" })
                    @Html.ValidationMessageFor(model => model.Email, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    Passwd : @Html.EditorFor(model => model.Passwd, new { @class = "form-control" })
                    @Html.ValidationMessageFor(model => model.Passwd, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

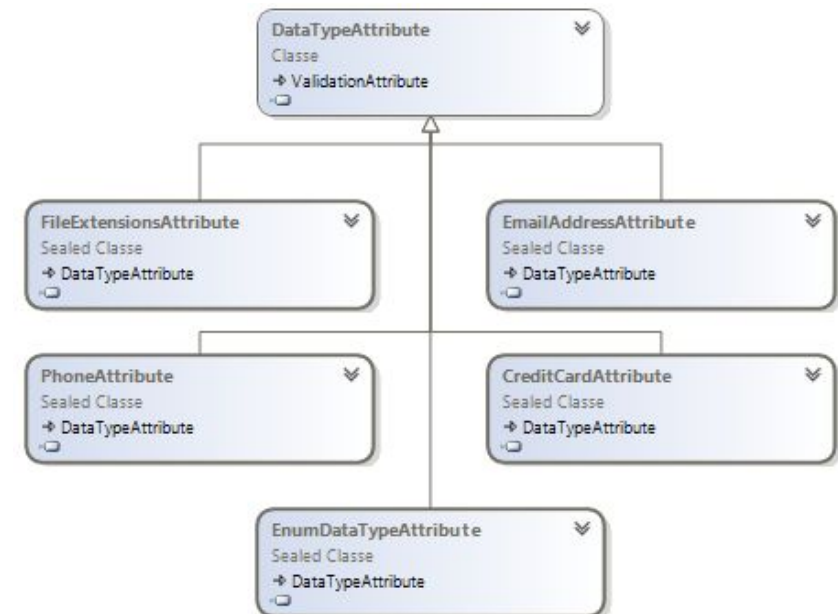
GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

```
public class LoginForm
{
    [Required(ErrorMessage="Le champs 'Email' est obligatoire")]
    [EmailAddress(ErrorMessage="Ce n'est pas une adresse email")]
    public string Email { get; set; }
    [Required(ErrorMessage="Le champs 'Mot de passe' est obligatoire")]
    [DataType(DataType.Password)]
    [RegularExpression(@"^.*(?=\{8,20\})(?=\.*\d)(?=\.*[a-z])(?=\.*[A-Z])(?=\.*[!@#$%^&+=]).*$",
        ErrorMessage="Ce n'est pas un mot de passe fort")]
    public string Passwd { get; set; }
}
```

Browser window showing a login form. The form has two input fields: 'Email' and 'Passwd'. The 'Email' field has a red error message: 'Le champs 'Email' est obligatoire'. The 'Passwd' field is masked with dots. There is a blue 'Create' button below the fields.

Pour palier à ce problème, nous pouvons utiliser l'attribut de validation « DataType » pour spécifier le type de la balise « input ».

Attention, que certains attributs de validation comme « EmailAddress » héritent déjà de l'attribut « DataType » et mettre les deux, générera une exception.



GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Html Helper
CheckBoxFor
DropDownListFor
HiddenFor
ListBoxFor
PasswordFor
RadioButtonFor
TextAreaFor
TextBoxFor

La méthode « EditorFor » est une méthode qui permet de générer presque tous les types de balise « Input ».

Cependant, il est possible d'utiliser des « Html Helpers » spécifiques en fonction de nos besoins.

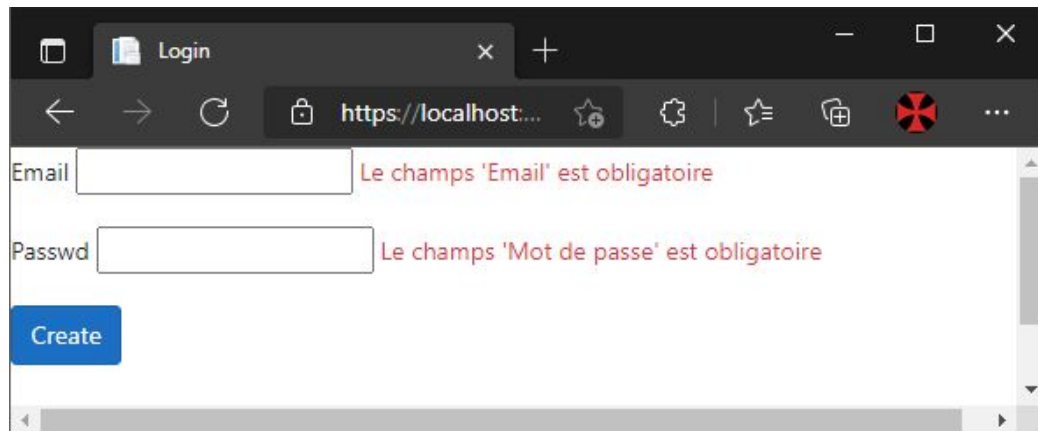


[Plus d'infos ici](#)

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Pour en terminer avec les champs, pour la génération de la balise « label », nous pouvons utiliser « LabelFor ».

Par défaut, cette méthode utilisera le nom de la propriété.



Le champs 'Email' est obligatoire

Le champs 'Mot de passe' est obligatoire

Create

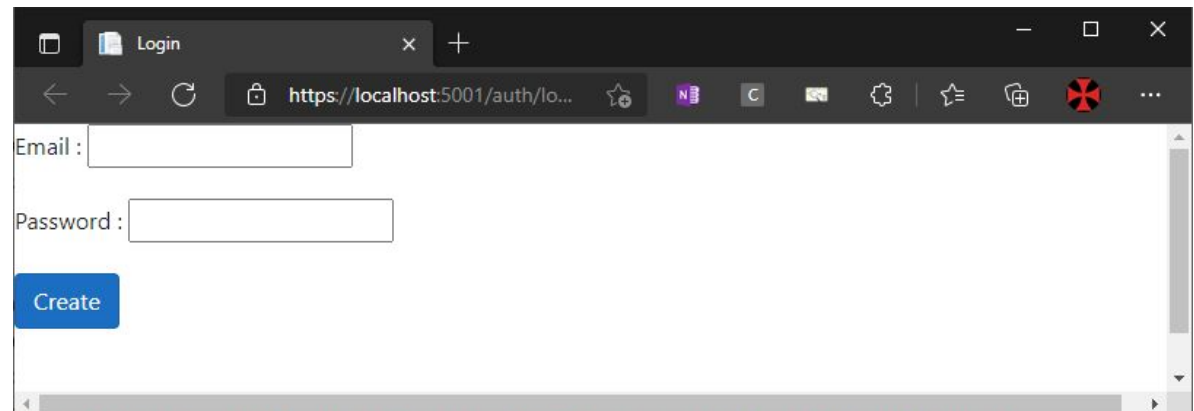
```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @if (!ViewContext.ModelState.IsValid)
    {
        @Html.ValidationSummary(true, null, new { @class = "text-danger" })
    }
    <div class="row">
        <div class="col-md-6">
            <form method="post">
                <div class="form-group">
                    @Html.LabelFor(model => model.Email)
                    @Html.EditorFor(model => model.Email, new { @class="form-control" })
                    @Html.ValidationMessageFor(model => model.Email, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    @Html.LabelFor(model => model.Passwd)
                    @Html.EditorFor(model => model.Passwd, new { @class = "form-control" })
                    @Html.ValidationMessageFor(model => model.Passwd, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

```
public class LoginForm
{
    [DisplayName("Email : ")]
    [Required(ErrorMessage="Le champs 'Email' est obligatoire")]
    [EmailAddress(ErrorMessage="Ce n'est pas une adresse email")]
    public string Email { get; set; }

    [DisplayName("Password : ")]
    [Required(ErrorMessage="Le champs 'Mot de passe' est obligatoire")]
    [DataType(DataType.Password)]
    [RegularExpression(@"^(?=.*{8,20})(?=.*\d)(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=]).*$",
        ErrorMessage="Ce n'est pas un mot de passe fort")]
    public string Passwd { get; set; }
}
```

Pour personnaliser, le contenu de la balise « label », nous pouvons utiliser l'attribut « DisplayName » de l'espace de noms « System.ComponentModel ».



The screenshot shows a web browser window with a single tab titled 'Login'. The address bar displays 'https://localhost:5001/auth/lo...'. The page content includes two input fields: 'Email :' followed by a text box, and 'Password :' followed by a password box. Below these fields is a blue button labeled 'Create'.

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

Enfin pour terminer avec le formulaire, nous pouvons remplacer la balise « Form » par la méthode « BeginForm ».

Cette méthode va générer un objet implémentant l'interface « IDisposable ».

Par conséquent, nous l'utiliserons avec un bloc « using ».

Cette méthode est bien sûr paramétrable, pour changer l'action, le contrôleur et/ou la méthode « GET » ou « POST ».

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @Html.ValidationSummary(true, null, new { @class = "text-danger" })
    <div class="row">
        <div class="col-md-6">
            @using (Html.BeginForm())
            {
                <div class="form-group">
                    @Html.LabelFor(model => model.Email)
                    @Html.EditorFor(model => model.Email, new { @class = "form-control" })
                    @Html.ValidationMessageFor(model => model.Email, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    @Html.LabelFor(model => model.Passwd)
                    @Html.EditorFor(model => model.Passwd, new { @class = "form-control" })
                    @Html.ValidationMessageFor(model => model.Passwd, null, new { @class = "text-danger" })
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            }
        </div>
    </div>
</body>
</html>
```

GÉNÉRER LE FORMULAIRE AVEC LES « HTML HELPERS »

```
@model IEnumerable<DisplayArticle>
<!DOCTYPE html>
<html>
<head>...</head>
<body>
<table class="table">
<thead>
<tr>
<th>
@Html.DisplayNameFor(model => model.Id)
</th>
<th>...</th>
<th>...</th>
</tr>
</thead>
<tbody>
@foreach (var item in Model)
{
<tr>
<td>...</td>
<td>
@Html.DisplayFor(modelItem => item.Nom)
</td>
<td>...</td>
</tr>
}
</tbody>
</table>
</body>
</html>
```

Il reste deux méthodes qui sont plus utilisées dans l'affichage de données comme pour les tableaux par exemple.

Il s'agit de « DisplayFor » et « DisplayNameFor ».

La méthode « DisplayNameFor » affichera le nom de la propriété et, comme pour le « LabelFor », il est sensible à la présence, au niveau de la propriété, de l'attribut « DisplayName ».

La méthode, « DisplayFor » quant à elle, affichera la valeur de la propriété sans la balise « label ».

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

Bien que les « Html Helpers » nous offrent un bon nombre de possibilités pour faciliter la création de nos pages et nos formulaires.

Cependant, ceux-ci lorsqu'ils sont présents en trop grand nombre, peuvent rendre nos pages difficilement lisible car nous remplaçons le code « HTML » par des appels de méthodes « C# ».

Pour palier à cela, les « tags helpers » ont été ajoutés.

Les Tag Helpers permettent au code côté serveur de participer à la création et au rendu des éléments « HTML » dans les fichiers « Razor ».

Il existe de nombreux « Tag Helpers » intégrés pour les tâches courantes telles que la création de formulaires, de liens, d'image, etc.

Ils sont créés en C# et ciblent les éléments HTML en fonction du nom de l'élément, du nom de l'attribut ou de la balise parent.

ASP.NET Core Tag Helpers	
Anchor Tag Helper	Label Tag Helper
Cache Tag Helper	Link Tag Helper
Component Tag Helper	Partial Tag Helper
Distributed Cache Tag Helper	Script Tag Helper
Environment Tag Helper	Select Tag Helper
Form Tag Helper	Textarea Tag Helper
Form Action Tag Helper	Validation Message Tag Helper
Image Tag Helper	Validation Summary Tag Helper
Input Tag Helper	



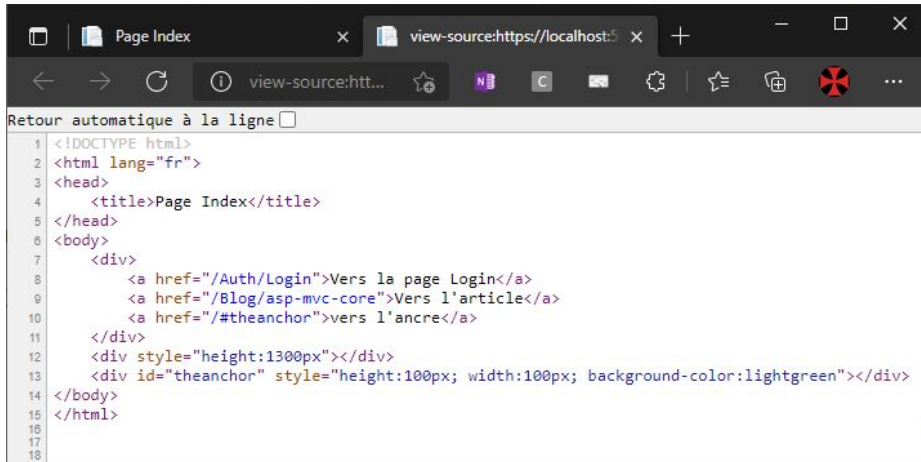
Lien : [ASP.NET Core built-in Tag Helpers | Microsoft Docs](#)



Attention tous les « Html Helper » n'ont pas de correspondance en « Tag Helper » comme pour « DisplayFor » et « DisplayNameFor »

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Page Index</title>
</head>
<body>
  <div>
    @* @Html.ActionLink("Vers la page Login", "Login", "Auth") *@
    <a asp-action="Login" asp-controller="Auth">Vers la page Login</a>
    <a asp-controller="blog" asp-route-article="asp-mvc-core">Vers l'article</a>
    <a asp-fragment="theanchor">vers l'ancre</a>
  </div>
  <div style="height:1300px"></div>
  <div id="theanchor" style="height:100px; width:100px; background-color:lightgreen"></div>
</body>
</html>
```



Pour la gestion des liens, nous retrouvons plusieurs attributs associés à la balise « a ».

- « asp-action » : spécifie le nom de l'action
- « asp-controller » : spécifie le nom du contrôleur
- « asp-route-id » : spécifie la valeur de l'id
- « asp-route-{value} » : spécifie toute autre valeur comme pour « asp-route-article » dans l'exemple
- « asp-fragment » : ajoute un # dans l'url pour spécifier l'ancre à atteindre

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

Le fait de déclarer notre balise « form » dans notre page est en soit déjà l'utilisation d'un « tag helper ».

Le simple fait de déclarer la balise « form » va entraîner la génération d'un token de vérification caché pour empêcher la falsification des demandes intersites.

Ensuite tout comme pour les liens, nous retrouvons « asp-action », « asp-controller », « asp-route-id », etc.

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    @Html.ValidationSummary(true, null, new { @class = "text-danger" })
    <div class="row">
        <div class="col-md-6">
            <form asp-action="Login" asp-controller="Auth" method="post">
                <div class="form-group">...</div>
                <div class="form-group">...</div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```



GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <div class="row">
    <div class="col-md-6">
      <form asp-action="Login" asp-controller="Auth" method="post">
        <div class="form-group">...</div>
        <div class="form-group">...</div>
        <div class="form-group">
          <input type="submit" value="Create" class="btn btn-primary" />
        </div>
      </form>
    </div>
  </div>
</body>
</html>
```

Concernant le « ValidationSummary », nous retrouvons un attribut « asp-validation-summary » associé à une balise « div ».

Cet attribut accepte trois valeurs :

- None : aucune erreur n'est affichée
- Model-Only : affiche les erreurs du modèle en excluant les erreurs liées aux propriétés
- All : toutes les erreurs sont affichées

Ces trois valeurs sont reprises dans l'énumération « ValidationSummary » de l'espace de noms « Microsoft.AspNetCore.Mvc.Rendering ».

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

Ensuite, au niveau des champs, on peut utiliser les attributs sur les balises « span » pour les messages d'erreurs par propriété.

Il s'agit de l'attribut « asp-validation-for » qui prend en paramètre le nom de la propriété à traiter.

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="row">
        <div class="col-md-6">
            <form asp-action="Login" asp-controller="Auth" method="post">
                <div class="form-group">
                    @Html.LabelFor(model => model.Email, new { @class = "control-label" })
                    @Html.EditorFor(model => model.Email, new { @class = "form-control" })
                    <span asp-validation-for="Email" class="text-danger"></span>
                </div>
                <div class="form-group">
                    @Html.LabelFor(model => model.Passwd, new { @class = "control-label" })
                    @Html.EditorFor(model => model.Passwd, new { @class = "form-control" })
                    <span asp-validation-for="Passwd" class="text-danger"></span>
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <div class="row">
    <div class="col-md-6">
      <form asp-action="Login" asp-controller="Auth" method="post">
        <div class="form-group">
          <label asp-for="Email" class="control-label"></label>
          @Html.EditorFor(model => model.Email, new { @class = "form-control" })
          <span asp-validation-for="Email" class="text-danger"></span>
        </div>
        <div class="form-group">
          <label asp-for="Passwd" class="control-label"></label>
          @Html.EditorFor(model => model.Passwd, new { @class = "form-control" })
          <span asp-validation-for="Passwd" class="text-danger"></span>
        </div>
        <div class="form-group">
          <input type="submit" value="Create" class="btn btn-primary" />
        </div>
      </form>
    </div>
  </div>
</body>
</html>
```

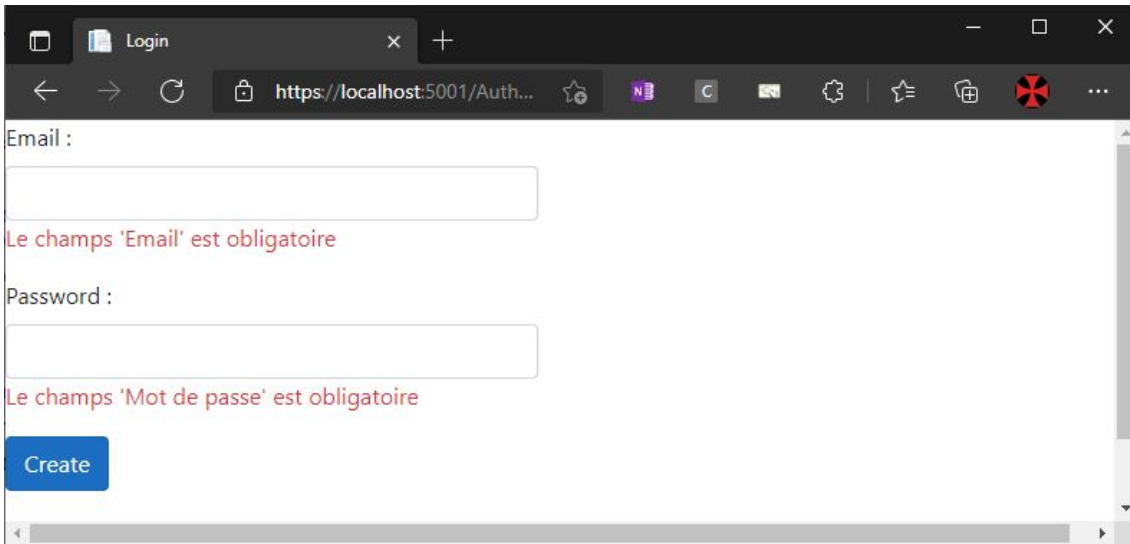
Pour les balises « label », on retrouve l'attribut « asp-for ».

Celle, tout comme la méthode « LabelFor » cet attribut tiendra compte de la présence de l'attribut « DisplayName » au niveau de la propriété du modèle.

GÉNÉRER LE FORMULAIRE AVEC LES « TAG HELPERS »

Enfin pour les balises « input », on retrouve aussi l'attribut « asp-for » qui fera le lien avec la propriété.

Cependant, contrairement au « Label Tag Helper », lui sera sensible à l'attribut « DataType » et ses dérivés.



```
@model LoginForm
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="row">
        <div class="col-md-6">
            <form asp-action="Login" asp-controller="Auth" method="post">
                <div class="form-group">
                    <label asp-for="Email" class="control-label"></label>
                    <input asp-for="Email" class="form-control" />
                    <span asp-validation-for="Email" class="text-danger"></span>
                </div>
                <div class="form-group">
                    <label asp-for="Passwd" class="control-label"></label>
                    <input asp-for="Passwd" class="form-control" />
                    <span asp-validation-for="Passwd" class="text-danger"></span>
                </div>
                <div class="form-group">
                    <input type="submit" value="Create" class="btn btn-primary" />
                </div>
            </form>
        </div>
    </div>
</body>
</html>
```

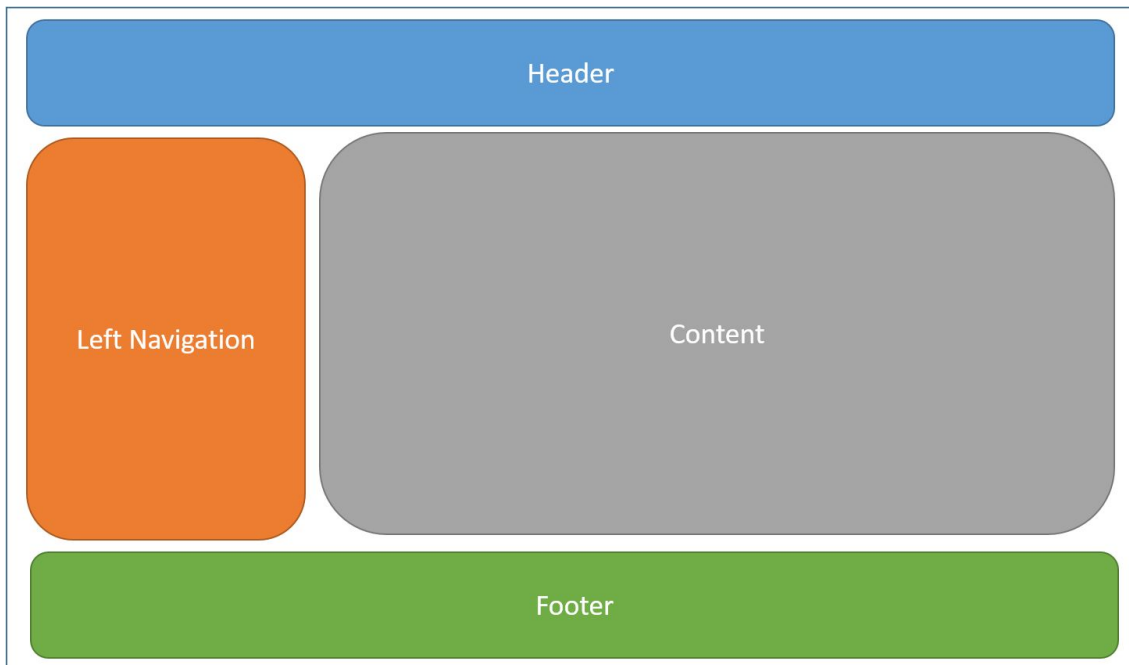


LES LAYOUTS

ASP .NET 6.0 MVC

- Qu'est qu'une page de disposition
- Le fichier « _layout.cshtml »
- L'instruction « @RenderBody »
- Le fichier « _viewimports.cshtml »
- Le fichier « _viewstart.cshtml »
- Utilisation des sections
- Le « scaffolding »

QU'EST QU'UNE PAGE DE DISPOSITION



La plupart des applications Web ont une disposition commune qui offre à l'utilisateur une expérience cohérente lorsqu'il navigue au sein de notre site.

La mise en page comprend généralement des éléments d'interface utilisateur courants tels que l'en-tête de l'application, les éléments de navigation ou de menu et le pied de page.

Les structures HTML courantes telles que les scripts et les feuilles de style sont également fréquemment utilisées par de nombreuses pages d'une application. Tous ces éléments partagés peuvent être définis dans une page de disposition, qui peut ensuite être référencé par n'importe quelle vue utilisée dans l'application.

Par convention, la page de disposition par défaut d'une application « ASP .NET 5 MVC » est nommée « `_Layout.cshtml` » et se trouve dans le répertoire « Shared » du répertoire « Views » de notre solution.

LE FICHIER « _LAYOUT.CSHTML »

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - AspNet5Mvc</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>
    <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white
      border-bottom box-shadow mb-3">
      <div class="container">...</div>
    </nav>
  </header>
  <div class="container">
    <main role="main" class="pb-3">
      @RenderBody()
    </main>
  </div>

  <footer class="border-top footer text-muted">
    <div class="container">
      &copy; 2021 - AspNet5Mvc - <a asp-controller="Home" asp-action="Privacy">Privacy</a>
    </div>
  </footer>
  <script src="~/lib/jquery/dist/jquery.min.js"></script>
  <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
  <script src="~/js/site.js" asp-append-version="true"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

Comme expliqué précédemment, cette page va contenir notre structure initiale de nos pages.

Nous y retrouvons :

- L'import de nos fichiers de style « css »
- L'en-tête de notre page contenant notre bar de navigation
- L'emplacement où sera placé notre contenu spécifique
- Le pied de page
- L'import des scripts

L'INSTRUCTION « @RENDERBODY »

Toute page « Layout » doit avoir l'appel à la méthode « RenderBody ».

Cet appel ne peut être présent qu'une seule fois et détermine l'emplacement où sera rendu le contenu de notre page.

Pour faire en sorte que notre page utilise la page « Layout », nous devons spécifier la propriété « Layout » dans cette dernière.



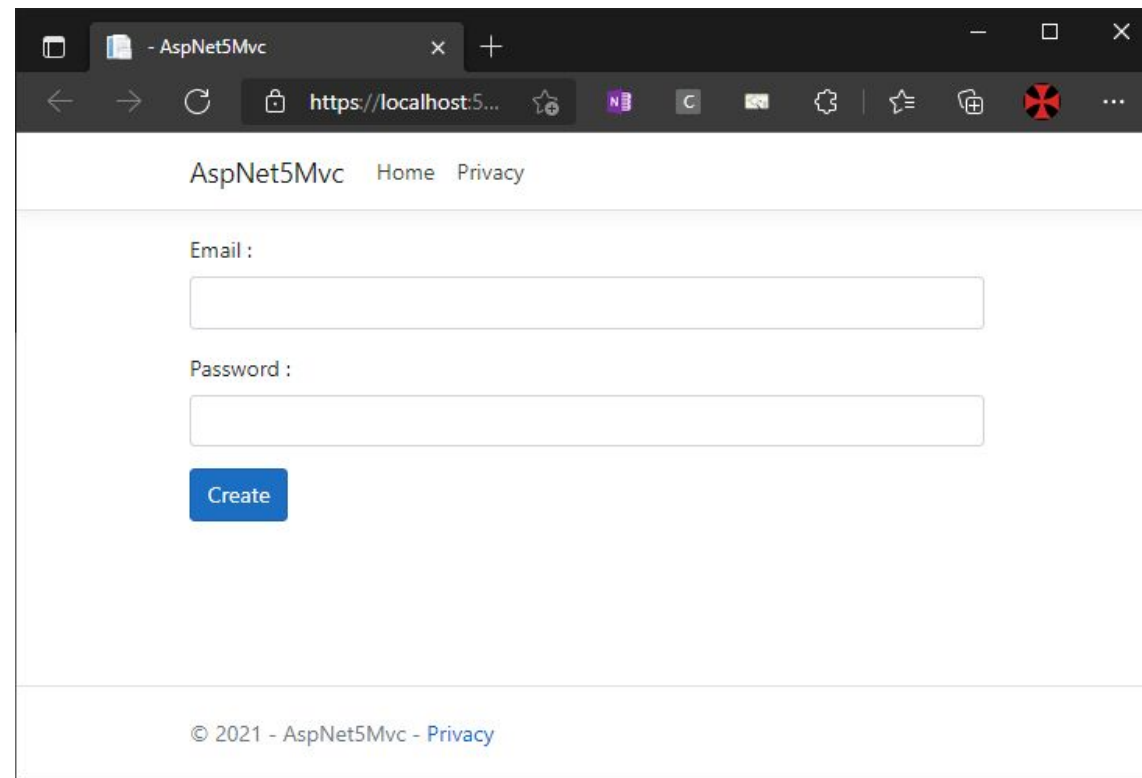
Contrairement aux versions précédentes, on ne spécifie que le nom du fichier sans le chemin complet ni l'extension

```
@model LoginForm

@{
    Layout = "_Layout";
}

<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="row">
    <div class="col-md-6">
        <form asp-action="Login" asp-controller="Auth" method="post">
            <div class="form-group">
                <label asp-for="Email" class="control-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Passwd" class="control-label"></label>
                <input asp-for="Passwd" class="form-control" />
                <span asp-validation-for="Passwd" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>
```

L'INSTRUCTION « @RENDERBODY »



AspNet5Mvc Home Privacy

Email :

Password :

Create

© 2021 - AspNet5Mvc - [Privacy](#)

LE FICHIER « _VIEWIMPORTS.CSHTML »

```
@using AspNetMvc  
@using AspNetMvc.Models
```

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

À la racine du répertoire « Views », nous retrouvons deux fichiers particuliers « _ViewImports.cshtml » et « _ViewStart.cshtml ».

Le fichier « _ViewImports.cshtml » sert à déclarer les espaces de noms et les « Tag Helpers » que nous utiliserons dans nos vues ainsi que les ressources à injecter dans toutes nos vues.

Ce qui, au final, nous évitera l'ajout récurrent des instructions « @using » et « @inject » au dessus de chacune de nos vues.



« @inject » sera vu dans la partie liée à l'injection de dépendances

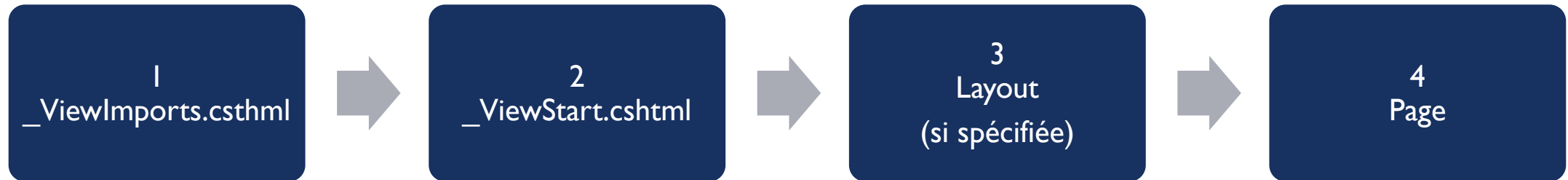
LE FICHIER « _VIEWSTART.CSHTML »

Le fichier « _ViewStart.cshtml » lui est prévu pour définir les propriétés par défaut de nos pages.

Nous y retrouvons, par ailleurs, la déclaration de la page « Layout » nous évitant de la déclarer à chaque page.

```
@{  
    Layout = "_Layout";  
}
```

Ordre d'appel des fichiers lorsqu'une vue est demandée par le moteur « MVC »



UTILISATION DES SECTIONS

```
@model LoginForm
@{
    ViewBag.Title = "Login";
}
@section CSS
{
    <link rel="stylesheet" href="~/css/specific.css" />
}
@section Scripts
{
    <script src="~/js/specific.js" asp-append-version="true"></script>
}
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="row">
    <div class="col-md-6">
        <form asp-action="Login" asp-controller="Auth" method="post">
            <div class="form-group">
                <label asp-for="Email" class="control-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Passwd" class="control-label"></label>
                <input asp-for="Passwd" class="form-control" />
                <span asp-validation-for="Passwd" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>
```

Une page « Layout » peut faire appel à une ou plusieurs section.

Ces sections sont définies dans les pages « Razor ».

Cas d'utilisation :

- Ajout d'un fichier de script
- Ajout d'une css
- Ajout d'un menu d'administration
- ...

UTILISATION DES SECTIONS

Tandis que dans la page de disposition, nous y ferons référence avec « RenderScript » ou « RenderScriptAsync ».

Si une section est déclarée dans la page « Razor », mais non utilisée par un « RenderScript » du « Layout », ceci génère une exception.

Pour l'éviter, nous devons utiliser la méthode « IgnoreSection » mais cela n'est pas une bonne pratique.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - AspNet5Mvc</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
  @RenderSection("CSS", false)
</head>
<body>
  <header>...</header>
  <div class="container">...</div>

  <footer class="border-top footer text-muted">...</footer>
  <script src="~/lib/jquery/dist/jquery.min.js"></script>
  <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
  <script src="~/js/site.js" asp-append-version="true"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

UTILISATION DES SECTIONS

```
...<!DOCTYPE html> == $0
<html lang="en">
  ▼ <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - AspNet5Mvc</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.min.css">
    <link rel="stylesheet" href="/css/site.css">
    <link rel="stylesheet" href="/css/specific.css">
    <meta id="ConnectiveDocSignExtentionInstalled" name="ConnectiveDocSignExtentionInstalled" data-extension-version="1.0.6">
  </head>
  ▼ <body>
    ▶ <header>...</header>
    ▶ <div class="container">...</div>
    ▶ <footer class="border-top footer text-muted">...</footer>
    <script src="/lib/jquery/dist/jquery.min.js"></script>
    <script src="/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="/js/site.js?v=4q1jwFhaPaZgr8WAUSrux6hAuh0XDg9kPS3xIVq36I0"></script>
    <script src="/js/specific.js?v=4q1jwFhaPaZgr8WAUSrux6hAuh0XDg9kPS3xIVq36I0"></script>
    <div id="eid-chrome-extension-is-installed"></div>
  </body>
</html>
```

LE « SCAFFOLDING »

L'« ASP.NET Scaffolding » est un outil de génération de code pour les applications « ASP.NET MVC Web App ».

Il nous permet d'ajouter rapidement du code qui interagit avec les modèles de données.

Son utilisation peut réduire le temps nécessaire au développement d'opérations répétitives sur nos données dans nos projets.

Le « scaffolding » permet, en suivant quelques étapes, de générer des contrôleurs et des vues.



Toute génération de code
entraîne une relecture attentive
du code généré!

```
public IActionResult Login()  
{  
    return View();  
}  
  
[HttpPost]  
public IActionResult Log  
{  
    if(!ModelState.IsValid)  
        return View();  
    else  
    {  
        //Traitement à r  
        return RedirectTo  
    }  
}
```

Add View...	
Go To View	Ctrl+M, Ctrl+G
Quick Actions and Refactorings...	Ctrl+;
Rename...	F2
Remove and Sort Usings	Ctrl+R, Ctrl+G
Peek Definition	Alt+F12
Go To Definition	F12
Go To Base	Alt+Home
Go To Implementation	Ctrl+F12
Find All References	Ctrl+K, R
View Call Hierarchy	Ctrl+K, Ctrl+T
Create Unit Tests	
Breakpoint	
Run To Cursor	Ctrl+F10
Execute in Interactive	Ctrl+E, Ctrl+E
Snippet	
Cut	Ctrl+X
Copy	Ctrl+C
Paste	Ctrl+V
Annotation	
Outlining	
Git	



DÉFINIR LE MODÈLE D'ACCÈS AUX DONNÉES

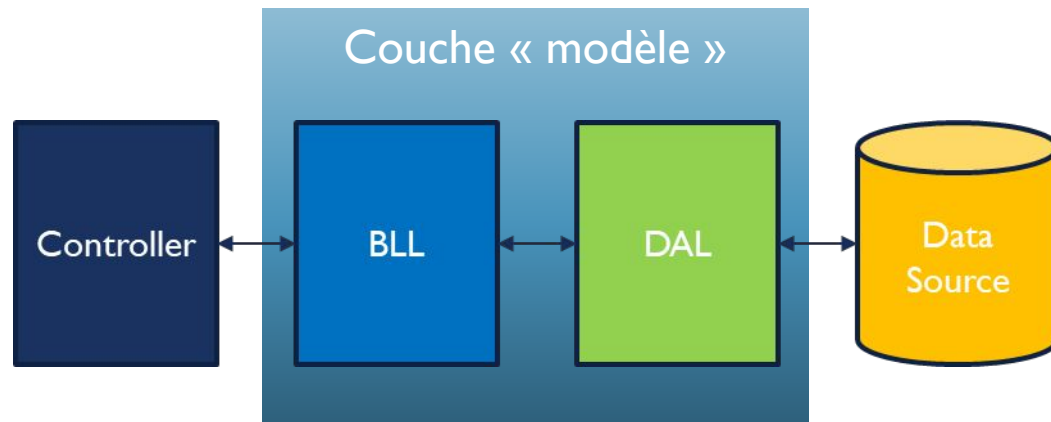
ASP .NET 6.0 MVC

- Structure du modèle
- Le pattern « Repository »
- Le principe d'injection de dépendances

DÉFINIR LE MODÈLE D'ACCÈS AUX DONNÉES

ASP .Net 6.0 MVC

STRUCTURE DU MODÈLE



La couche « modèle » est la couche qui représente la définition et l'accès aux données.

Celle-ci se divise en deux grandes parties

DAL : Data Access Layer (appelée modèle global)

BLL : Business Logic Layer (appelée modèle client)

STRUCTURE DU MODÈLE

La « DAL » est divisée en trois parties :

Les entités sont des classes qui représentent l'information provenant de la source de données. Ces classes sont des « POCO (Plain Old CLR Object) », c'est-à-dire qu'elles possèdent des propriétés en lecture/écriture « public » et le constructeur par défaut mais pas de méthodes et n'utilise pas le concept d'héritage.

Les mappeurs, sont là pour traduire l'information provenant de la source de données en instance d'objet par l'entremise de méthodes d'extension, de mappeurs génériques « home made » ou trouvés sur « Nugget ».

Les services sont des classes qui contiennent les requêtes à exécuter sur la source de données (requête sql, appel d'une vers une Web Api, etc.). Dans la bonne pratique celle-ci utilise le pattern « Repository »

Data Access Layer

Entities

Mappers

Services

STRUCTURE DU MODÈLE

Business Logic Layer

Entities

Mappers

Services

Tout comme la « DAL », la « BLL » contient les trois mêmes parties :

Les entités sont des classes qui représentent l'information utilisable par l'application. Ces classes sont sécurisées et ont des constructeurs et des méthodes et peuvent utiliser le concept d'héritage. De plus elle ne doivent pas être obligatoirement fidèle aux données reçues et peuvent donc filtrer ou ajouter des propriétés (comme les propriétés de navigation).

Les mappeurs, sont là pour traduire l'information provenant de la « DAL » en « BLL » et de la « BLL » vers la « DAL ».

Les services fournissent une façade pour les services de la « DAL » et font appels aux mappeurs pour la transformation des données.

En bon service qui se respecte, celui-ci utilise aussi le pattern « Repository ».

LE PATTERN « REPOSITORY »

Le design pattern « Repository » est très simple à implémenter.

Celui-ci consiste à déclarer une interface contenant les fonctionnalités que nos services devront implémenter.

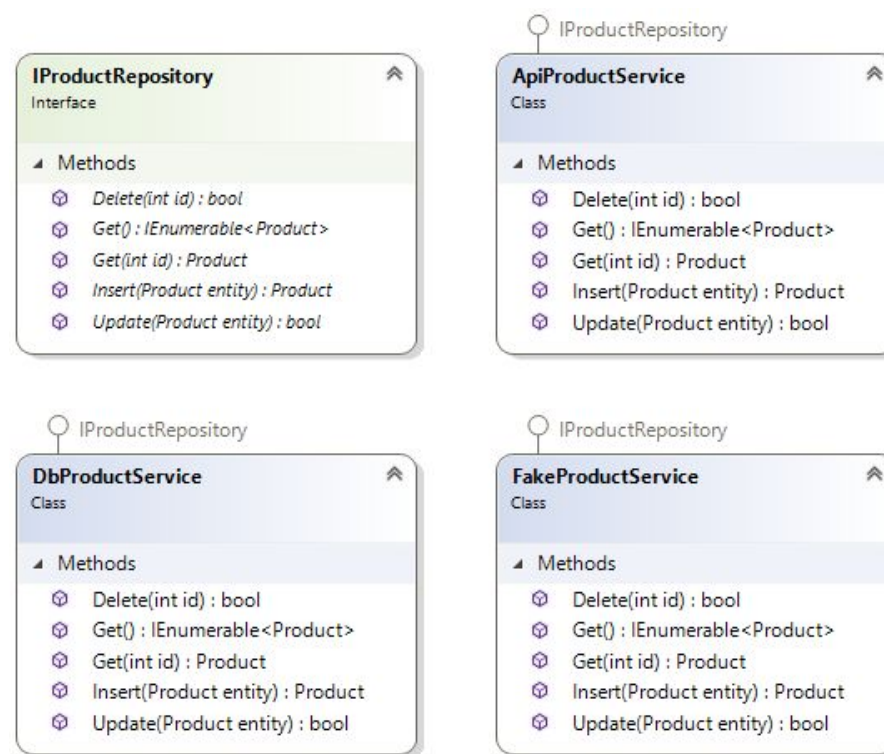
Le but de ce pattern est de pouvoir remplacer un service par un autre, en utilisant le concept de polymorphisme, très facilement.

Par exemple, remplacer le service d'accès à une base de données par un service d'accès à une api.

Il est également très utile dans le cadre des tests unitaires car nous pouvons déclarer des faux services et les utiliser sans changer le type, utilisant l'interface, de la variable.



Couplé à l'injection de dépendances qui va suivre, ce pattern est un incontournable



LE PRINCIPE D'INJECTION DE DÉPENDANCES

```
public class Startup
{
    1 reference
    public IConfiguration Configuration { get; }
    0 references
    public Startup(IConfiguration configuration) {...}

    0 references
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddControllersWithViews();
    }

    0 references
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env) {...}
}
```

Depuis le Framework Core 3.1, Microsoft a ajouté sa propre bibliothèque de classes pour l'injection de dépendances et l'a publiée sur « Nuget » sous le nom : « Microsoft.Extensions.DependencyInjection ».

Dans le cadre des applications web .Net Core (Asp et Api) celui-ci est déjà intégré dans le fichiers « Startup.cs » au travers de la méthode « ConfigureServices ».



Exemple complet :
[TMCognitic/DemoDependencyInjection \(github.com\)](https://github.com/TMCognitic/DemoDependencyInjection)

LE PRINCIPE D'INJECTION DE DÉPENDANCES

Le container de type « IServiceCollection » nous propose trois méthodes de base :

1. AddSingleton :
Retourne toujours la même instance d'objet
2. AddScoped :
Dans le même scope, retourne toujours la même instance d'objet
3. AddTransient :
Retourne toujours une nouvelle instance d'objet

Quant au type «(IServiceProvider) » il propose la méthode « GetService ».

```
public class Startup
{
    1 reference
    public IConfiguration Configuration { get; }
    0 references
    public Startup(IConfiguration configuration) ...
    0 references
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddControllersWithViews();
        services.AddSingleton<ISingletonRepository, SingletonService>();
        services.AddScoped<IScopedRepository, ScopedService>();
        services.AddTransient<ITransientRepository, TransientService>();
    }
    0 references
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env) ...
}
```

LE PRINCIPE D'INJECTION DE DÉPENDANCES

```
public class HomeController : Controller
{
    private readonly ISingletonRepository _singletonRepository;
    private readonly IScopedRepository _scopedRepository;
    private readonly ITransientRepository _transientRepository;

    0 references
    public HomeController(ISingletonRepository singletonRepository,
                        IScopedRepository scopedRepository,
                        ITransientRepository transientRepository)
    {
        _singletonRepository = singletonRepository;
        _scopedRepository = scopedRepository;
        _transientRepository = transientRepository;
    }

    0 references
    public IActionResult Index()
    {
        ViewBag.SingletonHashCode = _singletonRepository.GetHashCode();
        ViewBag.ScopedHashCode = _scopedRepository.GetHashCode();
        ViewBag.TransientHashCode = _transientRepository.GetHashCode();
        return View();
    }

    0 references
    public IActionResult Privacy()...
    Masqué pour des raisons de mise en page
    0 references
    public IActionResult Error()...
}
```

Pour injecter une ressource à un contrôleur, nous passerons par son constructeur.

LE PRINCIPE D'INJECTION DE DÉPENDANCES

Pour injecter une ressource à un vue, nous utiliserons « @inject ».

Pour rappel, si notre ressource doit être utilisée dans toute nos pages, nous pouvons le déclarer dans le fichier « _ViewImports.cshtml ».

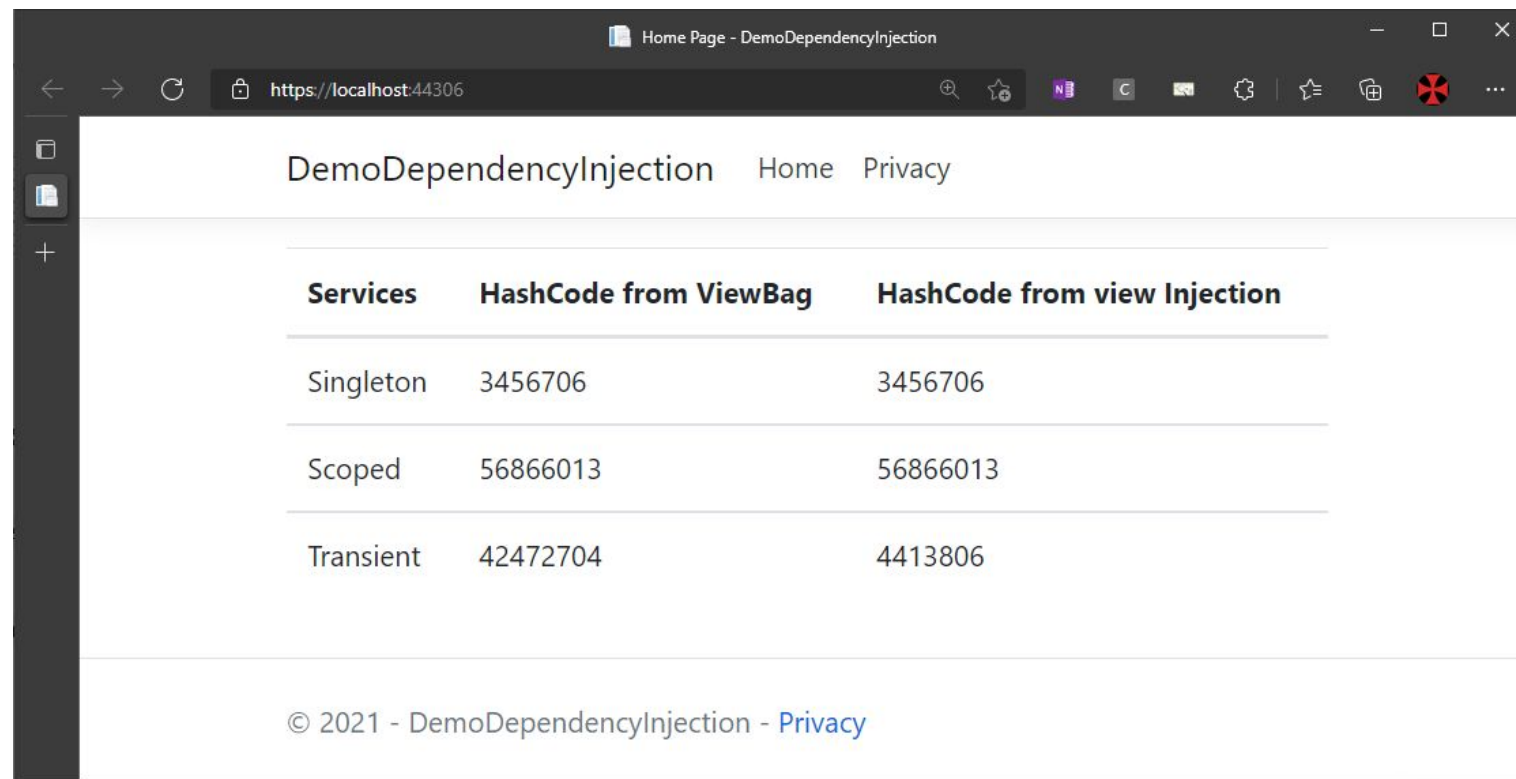
Il reste à noter qu'injecter un service d'accès au données directement dans une vue va à l'encontre de tout ce que nous avons vu en terme de séparation de couche du « MVC ».

```
@inject IRepository singletonRepository
@inject IRepository scopedRepository
@inject IRepository transientRepository

@{
    ViewData["Title"] = "Home Page";
}

<table class="table">
  <thead>
    <tr>
      <th>...</th>
      <th>...</th>
      <th>...</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>
        Singleton
      </td>
      <td>
        @ViewBag.SingletonHashCode
      </td>
      <td>
        @singletonRepository.GetHashCode()
      </td>
    </tr>
    <tr>...</tr>
    <tr>...</tr>
  </tbody>
</table>
```

LE PRINCIPE D'INJECTION DE DÉPENDANCES



The screenshot shows a web browser window with the title 'Home Page - DemoDependencyInjection'. The address bar shows 'https://localhost:44306'. The page content includes a header with 'DemoDependencyInjection', 'Home', and 'Privacy' links. Below the header is a table with three columns: 'Services', 'HashCode from ViewBag', and 'HashCode from view Injection'. The table contains three rows of data for Singleton, Scoped, and Transient services. At the bottom of the page, there is a copyright notice: '© 2021 - DemoDependencyInjection - Privacy'.

Services	HashCode from ViewBag	HashCode from view Injection
Singleton	3456706	3456706
Scoped	56866013	56866013
Transient	42472704	4413806

© 2021 - DemoDependencyInjection - [Privacy](#)



UTILISER LES SESSIONS

ASP .NET 6.0 MVC

- Le « Session State »
- Activation et configuration des sessions
- « Session State » et « RGPD »
- Utiliser les variables de sessions
- Le service « IHttpContextAccessor »
- La classe « SessionManager »
- Stocker les sessions en base de données

UTILISER LES SESSIONS

ASP .Net 6.0 MVC

LE « SESSION STATE »

L'état de session est un scénario « ASP .NET Core » pour le stockage des données utilisateur pendant que l'utilisateur parcourt une application web.

L'état de session utilise un magasin tenu à jour par l'application afin de conserver les données entre les requêtes d'un client.

« ASP .NET Core » conserve l'état de session en fournissant un cookie au client qui contient un « id » de session, cet « id » de session :

- Est envoyé à l'application avec chaque demande.
- Est utilisé par l'application pour extraire les données de session.

Contrairement à l'« ASP .NET Mvc » du Framework classique, les sessions ne sont pas activées en « ASP .NET MVC Core ».

À la première connexion du navigateur, la session est créée et le cookie est envoyé au client avec son « id ».

Les sessions vides ne sont pas conservées. Quand une session n'est pas conservée, un nouvel « id » de session est généré pour chaque nouvelle requête et le cookie est réutilisé.

Les sessions cessent d'exister dans trois cas :

- Fermeture du navigateur
- Timeout (20 minutes par défaut)
- Utilisation de la méthode « Clear »

ACTIVATION ET CONFIGURATION DES SESSIONS

Pour activer la gestion des sessions dans notre application, nous devons aller dans le fichier « Startup.cs ».

Dans la méthode « ConfigureServices » il faut ajouter :

- « services.AddDistributedMemoryCache(); »
- « services.AddSession(...) »

Ensuite dans la méthode « Configure »

- « app.UseSession(); »

Le cache de mémoire distribuée (AddDistributedMemoryCache) est une implémentation fournie par le Framework qui stocke les éléments en mémoire.

Les éléments mis en cache sont stockés par l'instance d'application sur le serveur sur lequel l'application est en cours d'exécution.

```
public class Startup
{
    Masqué
    References
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddControllersWithViews();
        services.AddDistributedMemoryCache();
        services.AddSession();
    }

    References
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    {
        if (env.IsDevelopment()) ...
        else ...

        app.UseSession();

        app.UseHttpsRedirection();
        app.UseStaticFiles();
        app.UseRouting();
        app.UseAuthorization();
        app.UseEndpoints(endpoints => ...);
    }
}
```

ACTIVATION ET CONFIGURATION DES SESSIONS

```
services.AddSession(options => {  
    options.Cookie.Name = "AspNet5Mvc.Session";  
    options.Cookie.HttpOnly = true;  
    options.IdleTimeout = TimeSpan.FromMinutes(10);  
});
```

La méthode « AddSession » ajoute les services nécessaires à la gestion des sessions. Ces services utilisent un cookie pour suivre et identifier les demandes par navigateur.

Par défaut, ce cookie est nommé « .AspNetCore.Session »

Il est possible de paramétrer le service de sessions avec, entre autre, les propriétés suivantes :

Option	Description
Cookie.Name	Définit le nom du fichier contenant l'id de session
Cookie.HttpOnly	Indique si un cookie est accessible par un script côté client
IdleTimeout	Définit le temps avant expiration de la session. Chaque appel réinitialise le « Timeout »

« COOKIE » ET « RGPD »

Comme le « SessionState » utilise un cookie pour identifier les sessions, lorsque ce cookie devient essentiel au fonctionnement du site, nous devons faire attention aux règles du « RGPD ».

De ce fait, « ASP .NET Core » fournit des api et des modèles pour vous aider à répondre à certaines exigences du « RGPD ».

Cela se fait en quatre étapes :

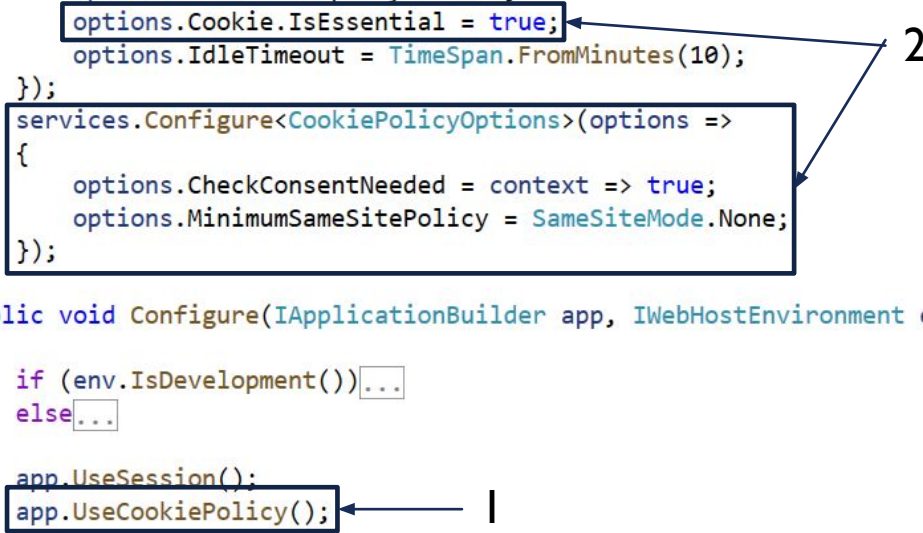
1. Définir que notre application va définir les polices (règles) concernant les cookie.
2. Configurer ces polices.
3. Créer le bandeau pour proposer à l'utilisateur d'accepter vos conditions d'utilisation.
4. Ajouter ce bandeau au « Layout » de votre site.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();
    services.AddDistributedMemoryCache();
    services.AddSession(options => {
        options.Cookie.Name = "AspNet5Mvc.Session";
        options.Cookie.HttpOnly = true;
        options.Cookie.IsEssential = true;
        options.IdleTimeout = TimeSpan.FromMinutes(10);
    });
    services.Configure<CookiePolicyOptions>(options =>
    {
        options.CheckConsentNeeded = context => true;
        options.MinimumSameSitePolicy = SameSiteMode.None;
    });
}

public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment()) ...
    else ...

    app.UseSession();
    app.UseCookiePolicy();

    app.UseHttpsRedirection();
    app.UseStaticFiles();
    app.UseRouting();
    app.UseAuthorization();
    app.UseEndpoints(endpoints => ...);
}
```



« COOKIE » ET « RGPD »

_CookieConsentPartial.cshtml

```
@using Microsoft.AspNetCore.Http.Features
@{
    var consentFeature = Context.Features.Get<ITrackingConsentFeature>();
    var showBanner = !consentFeature?.CanTrack ?? false;
    var cookieString = consentFeature?.CreateConsentCookie();
}
@if (showBanner)
{
    <div id="cookieConsent" class="alert alert-info alert-dismissible fade show" role="alert">
        Use this space to summarize your privacy and cookie use policy.
        <a asp-page="/Privacy">Learn More</a>.
        <button type="button" class="accept-policy close"
            data-dismiss="alert" aria-label="Close"
            data-cookie-string="@cookieString">
            <span aria-hidden="true">Accept</span>
        </button>
    </div>
    <script>
        (function () {
            var button = document.querySelector("#cookieConsent button[data-cookie-string]");
            button.addEventListener("click", function (event) {
                document.cookie = button.dataset.cookieString;
            }, false);
        })();
    </script>
}
```

3

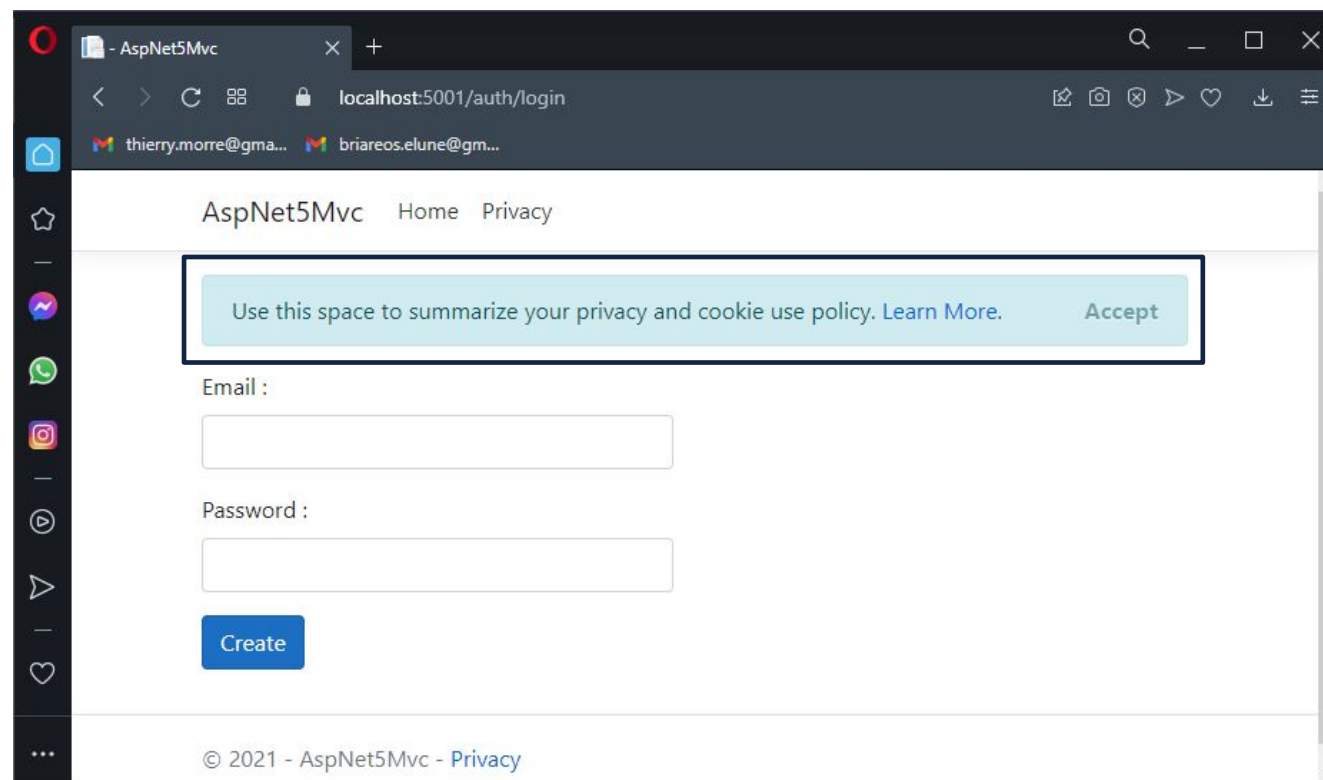
_Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>...</head>
<body>
    <header>...</header>
    <div class="container">
        <partial name="_CookieConsentPartial" />
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>

    <footer class="border-top footer text-muted">...</footer>
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

4

« COOKIE » ET « RGPD »



UTILISER LES VARIABLES DE SESSIONS

```
public class HomeController : Controller
{
    [ViewData]
    1 référence
    public string Title { get; set; }
    0 références
    public IActionResult Index()
    {
        HttpContext.Session.Set("ByteArray", new byte[0]);
        HttpContext.Session.SetString("key", "value");
        HttpContext.Session.SetInt32("MyInt32", 42);

        return View();
    }

    0 références
    public IActionResult Privacy()...
    Masqué pour des raisons de mise en page
    0 références
    public IActionResult Error()...
}
```

Pour utiliser les variables de session, nous devons utiliser la propriété « Session » qui est de type interface « ISession » présente dans le « HttpContext » du contrôleur ou via le « ViewContext.HttpContext » depuis la vue.

Les sessions de base ne peuvent contenir que des tableaux de bytes.

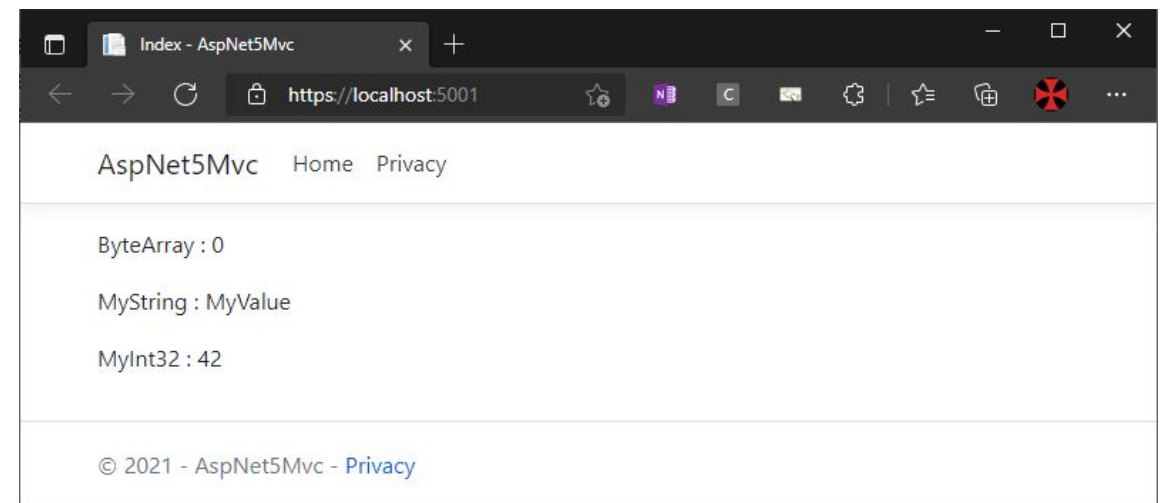
Cependant le type « SessionExtensions » de l'espace de noms « Microsoft.AspNetCore.Http » propose de stocker, en utilisant ses méthodes d'extensions, des entiers et des chaînes de caractères.

UTILISER LES VARIABLES DE SESSIONS

```
@using Microsoft.AspNetCore.Http

@{
    ViewBag.Title = "Index";
}

<p>@ViewContext.HttpContext.Session.Get("ByteArray").Length</p>
<p>@ViewContext.HttpContext.Session.GetString("MyString")</p>
<p>@ViewContext.HttpContext.Session.GetInt32("MyInt32")</p>
```



LE SERVICE « IHTTPCONTEXTACCESSOR »

Lorsqu'on utilise les variables de sessions, nous finissons toujours par tomber sur trois problématiques :

- Le nom des variables de session sont de type string et au moment de la compilation le contenu des chaînes de caractères n'est pas validé.
- Le noms des variables de session déjà utilisée et donc plus le site est gros et pour peu que nous soyons plusieurs à travailler sur le site connaître le nom des variables de sessions peut s'avérer difficile.
- Le type des variables de session, limité aux « byte[] », « int » et « string ».

Mais pour avoir accès aux variables de session, nous avons besoin d'accéder à l'« HttpContext ». Raison pour laquelle, nous allons ajouter un accesseur à cet « HttpContext » au niveau des services de l'application qui nous permet d'injecter un paramètre de type « IHttpContextAccessor » dans le constructeur de nos ressources.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();

    services.AddHttpContextAccessor();

    services.AddDistributedMemoryCache();
    services.AddSession(options => {
        options.Cookie.Name = "AspNet5Mvc.Session";
        options.Cookie.HttpOnly = true;
        options.Cookie.IsEssential = true;
        options.IdleTimeout = TimeSpan.FromMinutes(10);
    });
    services.Configure<CookiePolicyOptions>(options =>
    {
        options.CheckConsentNeeded = context => true;
        options.MinimumSameSitePolicy = SameSiteMode.None;
    });
}
```

LA CLASSE « SESSIONMANAGER »

```
public class SessionManager
{
    private readonly ISession _session;

    0 références
    public SessionManager(IHttpContextAccessor httpContextAccessor) {
        _session = httpContextAccessor.HttpContext.Session;
    }

    2 références
    public byte[] ByteArray {
        get { return _session.Get(nameof(ByteArray)); }
        set { _session.Set(nameof(ByteArray), value); }
    }

    2 références
    public string MyString {
        get { return _session.GetString(nameof(MyString)); }
        set { _session.SetString(nameof(MyString), value); }
    }

    2 références
    public int? MyInt32 {
        get { return _session.GetInt32(nameof(MyInt32)); }
        set { _session.SetInt32(nameof(MyInt32), value.Value); }
    }
}
```

Maintenant que nous nous sommes assuré d'accéder à au « HttpContext », nous pouvons définir une classe qui nous permet de :

- Centraliser les variables de sessions
- De typer ces variables
- De nommer ces variables avec nameof()

Il serait même possible de déclarer des types autres que le « SessionState » peut gérer en utilisant la sérialisation et la désérialisation.



Pour sérialiser et désérialiser, nous pouvons utiliser le type « JsonSerializer » de l'espace de noms « System.Text.Json »

LA CLASSE « SESSIONMANAGER »

```
public class SessionManager
{
    private readonly ISession _session;
    0 références
    public SessionManager(IHttpContextAccessor httpContextAccessor) {
        _session = httpContextAccessor.HttpContext.Session;
    }
    Masqué pour des raisons de mise en page
    6 références
    public IEnumerable<Article> Basket {
        get {
            if (_session.GetString(nameof(Basket)) is null)
                Basket = new List<Article>();
            return JsonSerializer.Deserialize<Article[]>(_session.GetString(nameof(Basket)));
        }
        private set
        {
            _session.SetString(nameof(Basket), JsonSerializer.Serialize(value));
        }
    }

    0 références
    public void AddArticle(Article article)
    {
        List<Article> articles = new List<Article>(Basket);
        articles.Add(article);
        Basket = articles;
    }
}
```

LA CLASSE « SESSIONMANAGER »

Il ne nous reste qu'à déclarer notre ressource en « Scoped » et à l'injecter dans nos contrôleurs et nos vues

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();

    services.AddHttpContextAccessor();
    services.AddScoped<SessionManager>();

    services.AddDistributedMemoryCache();
    services.AddSession(options => {
        options.Cookie.Name = "AspNet5Mvc.Session";
        options.Cookie.HttpOnly = true;
        options.Cookie.IsEssential = true;
        options.IdleTimeout = TimeSpan.FromMinutes(10);
    });
    services.Configure<CookiePolicyOptions>(options =>
    {
        options.CheckConsentNeeded = context => true;
        options.MinimumSameSitePolicy = SameSiteMode.None;
    });
}
```

```
@using AspNet5Mvc.Infrastructure
@Inject SessionManager sessionManager

@{
    ViewBag.Title = "Index";

    <p>ByteArray : @sessionManager.ByteArray.Length</p>
    <p>MyString : @sessionManager.MyString</p>
    <p>MyInt32 : @sessionManager.MyInt32</p>
}
```

```
public class HomeController : Controller
{
    private readonly SessionManager _sessionManager;

    0 références
    public HomeController(SessionManager sessionManager)
    {
        _sessionManager = sessionManager;
    }

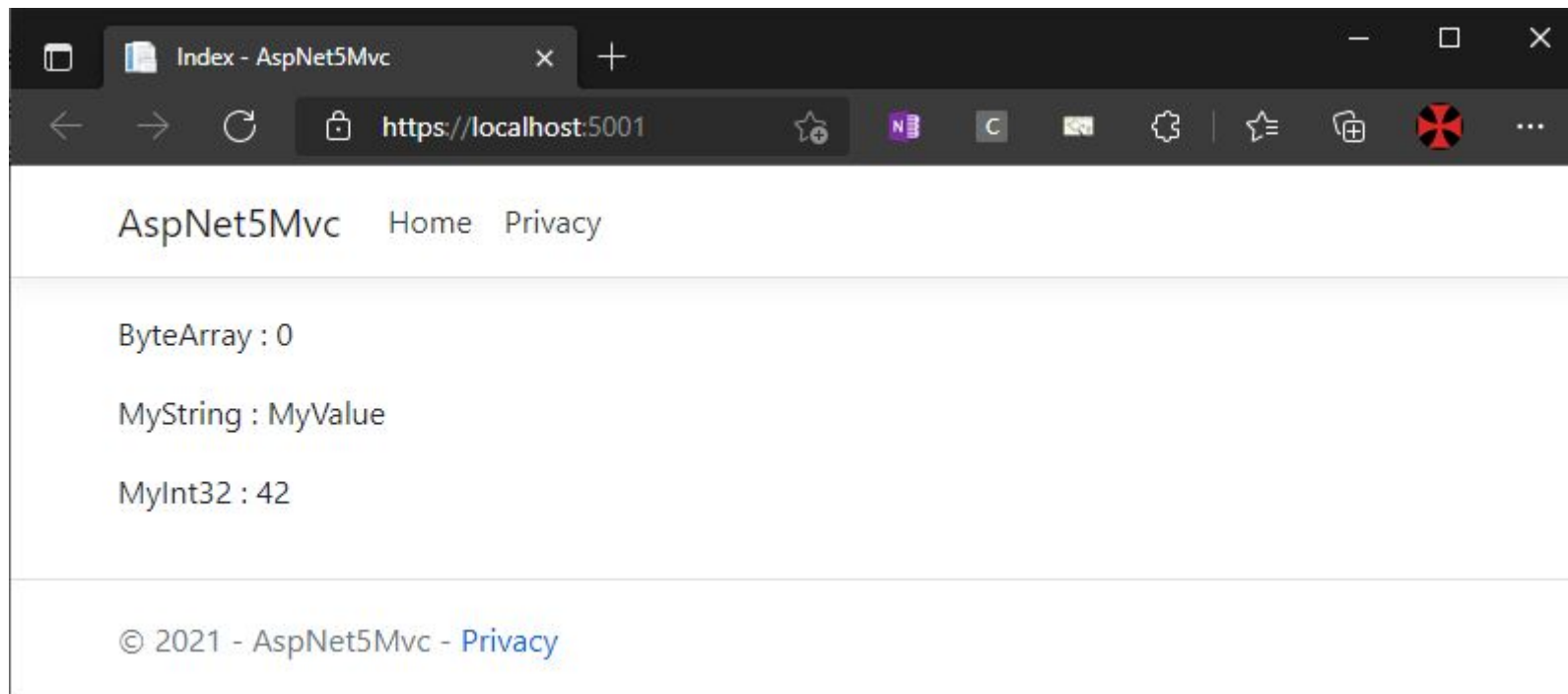
    0 références
    public IActionResult Index()
    {
        _sessionManager.ByteArray = new byte[0];
        _sessionManager.MyString = "MyValue";
        _sessionManager.MyInt32 = 42;

        return View();
    }
}
```

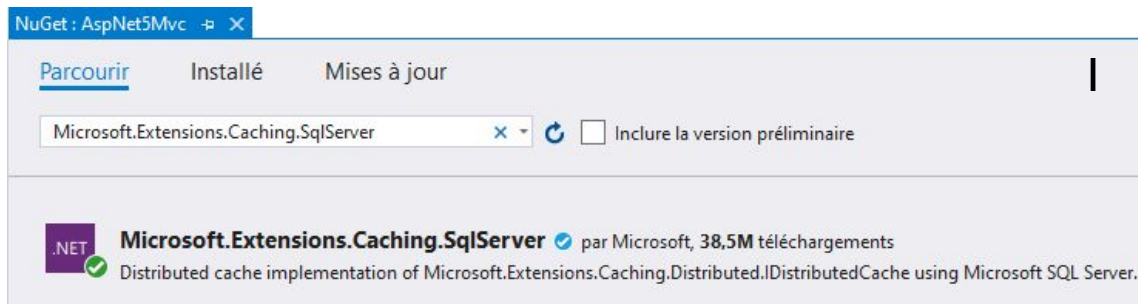


Mettre la ressource « SessionManager » en « Singleton » provoquera une exception car le « HttpContextAccessor » est disposé à chaque appel

LA CLASSE « SESSIONMANAGER »



STOCKER LES SESSIONS EN BASE DE DONNÉES



Le cache mémoire est pratique lorsque nous développons en test et en débogage.

En production, par contre nous devons assurer la pérennité des variables de sessions.

Il nous est donc possible de stocker nos sessions en base de données.

Pour y parvenir, nous devons :

1. Installer le package Nuget « Microsoft.Extensions.Caching.SqlServer »
2. Créer la base de données sur Sql Server
Create Database [DatabaseName];
3. Créer la table à l'aide de la commande dans une console
dotnet sql-cache create "connectionStringToDb" [Schema] [TableName]
4. Configurer les services dans la méthode « ConfigureServices » du fichier « Startup.cs ».

STOCKER LES SESSIONS EN BASE DE DONNÉES

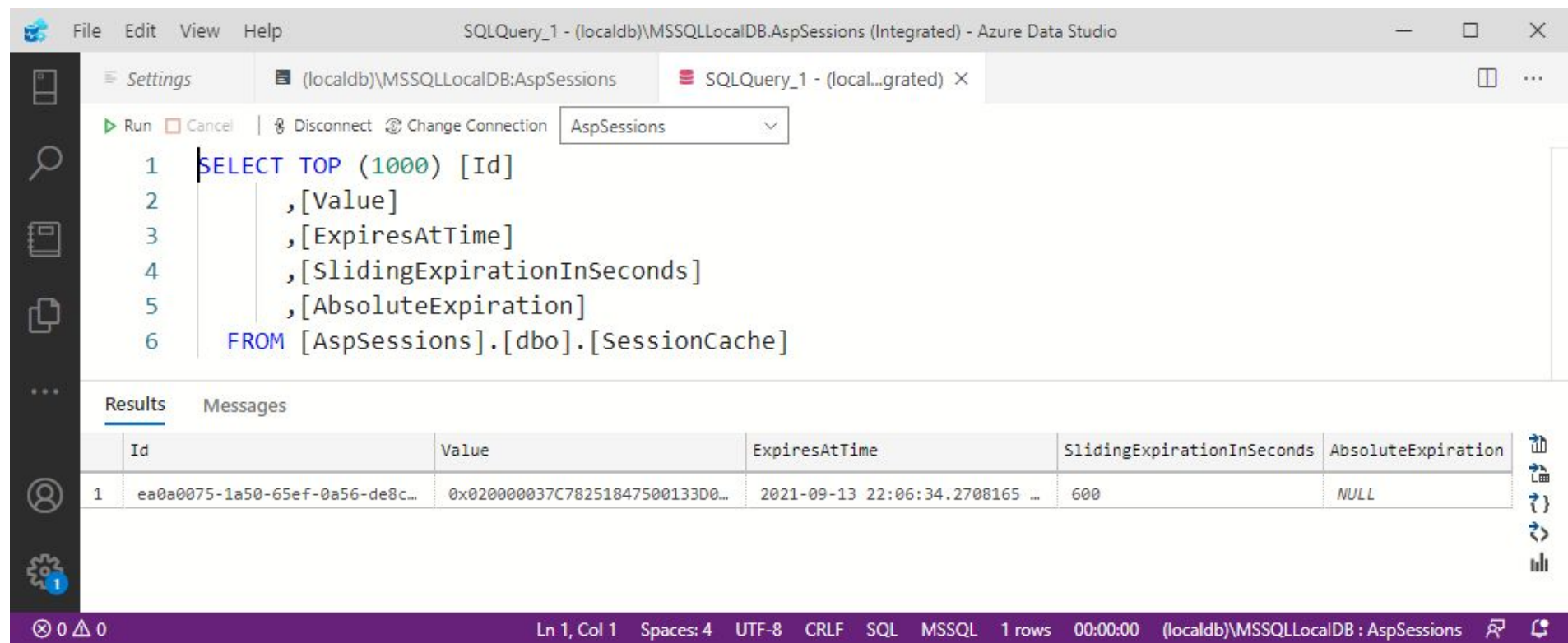
3

```
Developer PowerShell for VS 2019
PS E:\Users\Briareos\Sources\Repos> dotnet sql-cache create "Data Source=(localdb)\MSSQLLocalDB;
Initial Catalog=AspSessions;Integrated Security=True;" dbo SessionCache
Table and index were created successfully.
PS E:\Users\Briareos\Sources\Repos>
```

4

```
services.AddDistributedSqlServerCache(options =>
{
    options.ConnectionString = @"Data Source=(localdb)\MSSQLLocalDB;
Initial Catalog=AspSessions;Integrated Security=True;";
    options.SchemaName = "dbo";
    options.TableName = "SessionCache";
});
```

STOCKER LES SESSIONS EN BASE DE DONNÉES



The screenshot shows the Azure Data Studio interface. The top menu bar includes File, Edit, View, and Help. The title bar indicates the current query is 'SQLQuery_1 - (localdb)\MSSQLLocalDB:AspSessions (Integrated) - Azure Data Studio'. The main editor area contains a SQL query:

```
1 SELECT TOP (1000) [Id]
2     ,[Value]
3     ,[ExpiresAtTime]
4     ,[SlidingExpirationInSeconds]
5     ,[AbsoluteExpiration]
6 FROM [AspSessions].[dbo].[SessionCache]
```

Below the query editor, the 'Results' tab is active, displaying a table with the following data:

	Id	Value	ExpiresAtTime	SlidingExpirationInSeconds	AbsoluteExpiration
1	ea0a0075-1a50-65ef-0a56-de8c...	0x0200000037C78251847500133D0...	2021-09-13 22:06:34.2708165 ...	600	NULL

The bottom status bar shows 'Ln 1, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'SQL', 'MSSQL', '1 rows', '00:00:00', and the connection name '(localdb)\MSSQLLocalDB : AspSessions'.



RÉFÉRENCES

ASP .NET 6.0 MVC

- Microsoft :
Documentations (<https://docs.microsoft.com/fr-fr/aspnet/core/mvc/overview?view=aspnetcore-6.0>)